The CCME Release Guarantees

A formal model, with proofs, of the release computation, its publish protocol and its execution profile, with the executed campaigns

Semen Fediukovich

September 2026

Technical report accompanying `specs/ccme-spec/SPEC.md` 3.1.0-rc.2
at commit `252216ba` of `github.com/yohimik/dispat`

Abstract

Releasing interdependent packages is a distributed transaction without a coordinator or rollback: each publish is an irreversible write to an independent registry. CCME defines a resumable release algorithm and states eight guarantees: termination, completeness, version stability, exactly-once tag-ledger delivery, retry-invariant non-widening and convergence, channel discharge and one-way axis independence.

This report formalises the algorithm of the 3.1.0 release candidate and proves the guarantees under its explicit retry invariant. The candidate replaces the window test of earlier versions by delivery: a propagated contribution stays owed to a consumer until the source has released it and the consumer has released after that, which closes the orphan a consumer reaches by getting ahead of a failed provider, at the price of an owed window per dependency edge and a refusal (`E201`) where two releases on one commit could hide a debt. The report also proves the determinism and idempotency requirements of SPEC §17.2, the duality of SPEC §13.7b, and transcribes what the candidate adds around the computation: records read under the release lock (SPEC §13.2), the three-valued build readiness relation of SPEC §19.2a with the run durations it leads to, the distributed execution profile of SPEC §28 with the placement invariance and list-scheduling bounds it admits, and the repository-local windows of SPEC §27. The cost bounds of SPEC §§13.11, 20.7 are restated for the current text, including the single marker pass over history.

Two ledgers of obligations follow: the current departures of the reference implementation from the candidate, and the six failure classes found in CCME 1.0.0 with their minimal counterexamples, kept as provenance. A conformance table maps every result to the specification's vectors and to the integration test that stands behind it. The final section reports every executed campaign with its pinned binary and environment, including the two measurements of 22 September 2026 on which the execution profile rests: the kernel farm groups that met their public reference times on a second node, and the LLVM matrix released through two workers in less time than on one node, each once, with the reproduction kit that rebuilds the binaries byte for byte and recreates the machines.

Specification snapshot. The normative subject is `specs/ccme-spec/SPEC.md` as committed at `252216bab9bd4052ba181a2ceb47e21363b8eff3`, stamped 3.1.0-rc.2 and pending release as 3.1.0-rc.3 together with dispat 1.11.0-rc.4; its `DESIGN-HISTORY.md` entries of 21 and 22 September 2026 record what the engine implements. Where the report says *the current text* it means this commit. The historical audit of Section 10.2 concerns CCME 1.0.0 at `713f1a0b`; the released CCME 2.0.0 at `d3ce8ae6` is cited where a change since then matters.

Contents

1 Introduction **5**

1.1 The problem . . . 5
1.2 Why formalise them, and what changed . . . 6
1.3 Scope, and what this is not . . . 7
1.4 Implementation examples outside the model . . . 7
1.5 How to read it . . . 8
1.6 Licence . . . 8

2 The model **9**

2.1 History . . . 9
2.2 Workspace . . . 9
2.3 Bumps and versions . . . 10
2.4 State . . . 10
2.5 Units . . . 12
2.6 Runs . . . 14

3 Propagation **16**

3.1 The procedure, as written . . . 16
3.2 The traversal . . . 17
3.3 Phase 1: the channel axis . . . 19
3.4 Phase 2: channel resolution . . . 19
3.5 Phase 3: the bump axis . . . 19

4 The publish protocol **22**

4.1 Records under the lock . . . 22
4.2 Tag after, immediately, always, and once . . . 23
4.3 Order . . . 23
4.4 Build readiness . . . 24
4.5 Partial failure . . . 25
4.6 Reconciliation and manifest writes . . . 27

5 Execution: build readiness, distributed placement, and repository-local windows **27**

5.1 Build readiness durations . . . 28
5.2 Distributed execution . . . 30
5.3 Repository-local windows . . . 34

6 The eight guarantees **36**

6.1 Lemmas . . . 37

6.2 G1: termination . . . 39
6.3 G2: completeness . . . 39
6.4 G3: version stability . . . 40
6.5 G4: no double release . . . 42
6.6 G5: no blast-radius widening . . . 42
6.7 G6: convergence . . . 43
6.8 G7: channel discharge . . . 45
6.9 G8: axis independence . . . 45

7 Determinism, idempotency, resumption 46

8 The staleness dual 47

9 Cost 51
9.1 Windows . . . 53
9.2 Safe traversal reuse . . . 55
9.3 Hoisting . . . 56
9.4 Inverted channel resolution . . . 56
9.5 Deterministic order and failure closure . . . 57
9.6 The profile rows . . . 58
9.7 Space . . . 58
9.8 Parsing . . . 59
9.9 What none of this may change . . . 60

10 Proof obligations 60
10.1 Current obligations . . . 61
10.1.1 The owed window and `E201` are specified but not implemented . . . 61
10.1.2 Two releases on one commit: a stated boundary of completeness . . . 62
10.1.3 The version summary still contradicts the graduation rule . . . 62
10.1.4 One edge-kind set orders, blocks and refuses cycles . . . 63
10.1.5 The fleet recorder rejects a retry that annotates the same commit . . . 64
10.1.6 The failure category travels only from a distributed lock loss . . . 65
10.1.7 Profile departures and inapplicable vectors . . . 65
10.1.8 Textual defects noticed in passing . . . 66
10.2 Historical CCME 1.0.0 obligations . . . 66
10.2.1 The prerelease window never discharges . . . 67
10.2.2 Window-scoped suppression lapses, so the admitted set can grow . . . 68
10.2.3 The staleness dual omits the resolvability test . . . 69
10.2.4 Axis independence is one-directional . . . 70
10.2.5 Inherited channels read a baseline that release moves . . . 71

10.2.6 Bucketed traversal over-admits a unit's own sources 71

11 Conformance **72**

12 Evaluation: the executed campaigns **76**

12.1 Evidence boundaries and units 76

12.2 RQ1: the consumer-window mechanism 78

12.3 RQ2: retry and identity stability 79

12.4 RQ3: availability boundaries and CI 82

12.5 RQ4: planning cost 84

12.6 RQ5: distributed execution 85

12.6.1 The kernel farm 85

12.6.2 The LLVM pair 90

12.7 Threats to validity 95

12.8 Reproduction 96

A Notation **97**

1 Introduction

1.1 The problem

A workspace is a repository holding many independently versioned packages that depend on one another. Releasing it means choosing a new version for each package that needs one, writing those versions into manifests, uploading artefacts to registries, and recording what happened.

The recording is the hard part. Registries have no transactions. A publish is an irreversible write to a service the release tool does not control, several registries may be involved (npm, a container registry, a language-specific index), and there is no coordinator that can commit or abort them as a unit. A run that publishes seven packages is seven independent, individually fallible writes; it may finish having done four of them.

CCME[1] answers this the way a saga does. There is no external state file, no lock service beyond the release lock itself, and no registry polling. The release plan is a pure function of the repository at a fixed commit; each completed leg of the transaction is recorded by an annotated git tag written immediately after that leg succeeds; and recovery is re-execution: run the engine again and it recomputes the same transaction, skipping the legs already recorded.

That design only works if re-execution is well behaved, and SPEC §13.7c states the eight properties that say it is:

G1 termination
: propagation halts, whatever the declared depth.

G2 completeness
: a contribution admitted to a dependent survives in every later run until that dependent releases at or after a release of every owed source carrying it; there are no orphans, whether the provider failed or the consumer got ahead of it.

G3 version stability
: a package released late is released at the version it was originally planned at.

G4 no double ledger delivery
: a contribution does not survive its delivery in the tag ledger; a consumer that proceeded past a commit before its source released it is bumped once more when that release comes, and that is the delivery, not a second one. This says nothing unconditional about external effects.

G5 no blast-radius widening
: under the retry invariant, later targets are a subset of the first run's.

G6 restricted convergence
: under the retry invariant, repeated successful progress empties the initial plan within as many runs as it had publishable packages.

G7 channel discharge
: a prerelease channel is proposed to a package only until it is released on it.

G8 one-way axis independence
: suppressing a version bump does not suppress a channel change; resolved channels may suppress bumps.

[1] *Conventional Commits: Monorepo Extension*, version 3.1.0-rc.2 at commit `252216ba`, `specs/ccme-spec/SPEC.md` in the `dispat` repository. Referred to throughout as SPEC, cited by section number.

1.2 Why formalise them, and what changed

The specification proves each property briefly, while the algorithm is spread across prose and pseudocode. This report writes the algorithm down once, formally (Sections 2 to 4), and proves the guarantees inside that model (Section 6), along with the determinism and idempotency obligations of SPEC §17.2 (Section 7), the duality of SPEC §13.7b (Section 8), and the cost bounds and optimisation-equivalence claims of SPEC §§13.11, 20.7 (Section 9).

Earlier editions of this report formalised CCME 2.0.0, whose contribution was to state the cross-run retry invariant explicitly and to separate the train window from the undelivered window. The current text keeps both and changes the admission test of the bump axis: where 2.0.0 admitted a propagated contribution while the unit's commit lay in the dependent's undelivered window, the candidate admits it while some source of the unit has not *delivered* that commit to the dependent, that is, while the dependent has not released at or after a release of that source carrying it (SPEC §13.4a). The two tests agree for every dependent that has not released past the commit. They differ for a dependent that released past it before its provider did, which the candidate's own publish protocol now allows: a consumer with a cause of its own proceeds past a failed provider (SPEC §19.3) instead of being blocked. Under the old test such a consumer was never planned again once the provider published, an orphan reached from the consumer's side. Delivery closes it, and two further rules follow from it: a window per dependency edge that keeps the debt visible once the provider has released (SPEC §13.3), and a refusal, `E201`, where a provider would release at the very commit a consumer it still owes was released at, because two releases on one commit cannot be ordered afterwards. Sections 2, 3 and 6 transcribe and prove the candidate's form; proof obligation 132 states the one place where the ledger reports a loss instead of preventing it.

Around the computation the candidate adds what an engine that runs builds and publishes across machines needs, and Sections 4 and 5 transcribe it as definitions and propositions rather than as narrative: release records compared with the authoritative store under the lock before planning (SPEC §13.2); a create-only rule for release tags (SPEC §19.1); the three-valued build readiness relation of an edge, `none`, `build` or `publish`, which is execution policy and never changes a plan (SPEC §19.2a), with the run durations each value leads to; the distributed execution profile (SPEC §28), where the plan is fixed under the lock before any task is placed, outputs travel as verified first-class inputs, and publication and recording stay serialised per owner; and the repository-local windows of the polyrepository profile (SPEC §27). The single-history theorems are not extended to several histories here; the report says so where the objects are defined.

The reference implementation, dispat, implements the delivery admission and the reconciliation of a proceeding consumer, and does not yet implement the owed windows and `E201`. Section 10 keeps two ledgers: the current departures of the engine from the candidate, each with the scenario that exposes it, and the six failure classes found in CCME 1.0.0 with their minimal counterexamples, which 2.0.0 repaired and which remain regression evidence. Section 11 maps each result to the specification's conformance vectors and to the integration test that stands behind it.

Section 12 is new to this edition. It reports every executed campaign with its pinned binary and environment: the differential validations of the planner, the recovery and mixed-registry campaigns, the controlled fault matrix with its finite projection, the planning-cost cells, and the two measurements of 22 September 2026 on which the execution profile rests, the Linux kernel farm groups and the LLVM matrix, each once, with the kit that reproduces them.

1.3 Scope, and what this is not

The subject is the release computation of SPEC §13, the publish protocol of SPEC §19 and, as transcribed obligations, the execution rules of SPEC §§19.2a, 28; the guarantees are properties of the computation and the protocol together, and no separation of them is meaningful. Message syntax (SPEC §4–SPEC §8) is taken as given: units arrive already parsed, with their scope-sets resolved and their corrections applied, which is what SPEC §§13.4, 13.4b deliver to SPEC §13.5 onwards. The one exception is Proposition 130, which states the parser's linearity because SPEC §18.3 depends on it.

Five things a complete account would also cover are outside the theorems, and no theorem below should be read as saying anything about them: manifest range reconciliation (SPEC §9.4, SPEC §19.5), which happens at publish time and feeds back into the plan only through the published versions the protocol names; the diagnostic registry of SPEC §16, of which only the codes naming a suppression or a refusal are mentioned, as labels; the scheduling of the executor beyond the happens-before relations and the duration bounds Section 5 states; shared-version groups (SPEC §13.9a), whose member target floor and independent counter and channel axes are a specification contract this report does not prove; and the adapter and rollback protocols of SPEC §§25, 26, which the current text marks as pending implementation.

The report makes no novelty claim for the underlying patterns: sagas, idempotent consumers, breadth-first search and list scheduling. Its contribution is the formal model, corrected hypotheses, counterexamples, conformance obligations, and the record of what was measured.

1.4 Implementation examples outside the model

Two small implementation examples show how the executor can be composed with other controls; neither extends the theorem scope above. A developer using Windows locally and a Linux CI runner can share one release graph and lifecycle while choosing a shell explicitly. Keep these three files in the repository root:

```
# global.yaml: shared graph, scripts and lifecycle
scripts:
  build: cmake --build build
flow:
  build: [build]
packages:
  app:
    path: app

# linux.yaml: Linux overrides only
$ref: ./global.yaml
shell: ["/bin/sh", "-c"]

# windows.yaml: Windows overrides only
$ref: ./global.yaml
shell: ["powershell.exe", "-NoProfile", "-NonInteractive",
        "-Command"]
```

The platform files inherit the shared configuration and replace only `shell`. References resolve relative to the file containing them. Overrides replace whole keys, not nested fields: overriding `scripts` would replace that entire map. Individual scripts can instead reference separate files with `$ref`.

From Windows, preview with `dispat -config windows.yaml status`; on the Linux runner, use `dispat -config linux.yaml status`. After its release gates, CI can run `dispat -config linux.yaml` to execute the release. Where a compatible POSIX shell is available, one selector can choose the preview configuration:

```
dispat if OS=Windows_NT \
  --then 'dispat --config windows.yaml status' \
  --else 'dispat --config linux.yaml status'
```

`OS=Windows_NT` tests an environment value; it is not host detection. Here the non-Windows branch assumes a Linux runner. The evaluated releases execute `if` branches through `/bin/sh -c` without reading configuration; the child invocation then loads its platform shell. Thus the selector needs `/bin/sh`, even though the Windows configuration names PowerShell. A native Windows user can invoke the Windows configuration directly instead. The example assumes CMake is installed and `app/build` is already configured. It does not establish native Windows execution coverage.

A shared `dispat for Debug Release -do "dispat exec build-one"` can likewise pass each build configuration through `DISPAT_ITEM` to a named script. A configuration-free literal `for` loop also requires `/bin/sh`; platform-specific leaf commands must still be valid for the chosen shell. These are composition examples, not additional protocol guarantees.

For an agent-assisted release, complementary gates keep evidence attached to the inputs being published: a lock-free `dispat status -require-release -strict`; an all-package workspace suite with temporary links and cleanup that runs and is verified under `always()`; and package checks after version/lockfile reconciliation plus smoke tests of the exact built artefact before publish. These commands inspect or test inputs; they do not implicitly execute the computed release plan. Every wrapper must preserve exit status, and these checks neither start a release nor change the required authorization policy. HOL Guard PR [20] adds opt-in review when a Dispat release starts, but at its cited open, unmerged revision it is not an implementation or experimental validation of this whole gate stack; that integration remains in progress.

1.5 How to read it

Section 2 is definitional and dense; a reader who wants the results first can read Section 6 directly, following the macro names back to Section 2 as needed. Section 5 can be read on its own by anyone interested in what placement and the readiness relation can and cannot buy; Section 12 likewise, by anyone who wants the numbers and their provenance before the mathematics. Appendix A is the symbol table. Every definition names the specification section it transcribes, and every theorem names the guarantee it formalises, so the report can be read beside its source. Section 11 maps each result to the conformance vectors of SPEC Appendix B and to the test in the reference implementation that exercises it.

1.6 Licence

SPEC is licensed GPL-3.0-or-later, and this report reproduces its definitions and pseudocode; it is therefore distributed under the same licence. See `paper/LICENSE`.

2 The model

Everything in this report is stated over the objects defined here. Each definition transcribes a numbered section of SPEC and cites it. Where the transcription makes a choice the source leaves implicit, the choice is flagged in a remark rather than buried.

2.1 History

Definition 1 (commit history, SPEC §12.2)**.** The history is a finite directed acyclic graph on a set of commits $\mathcal{K}$, with an edge from a commit to each of its parents. A distinguished commit $h \in \mathcal{K}$ is *HEAD*. For $c \in \mathcal{K}$, $\mathrm{reach}(c) \subseteq \mathcal{K}$ is the set of commits reachable from c by following parent edges, including c itself. Write $c' \preceq c$ for $c' \in \mathrm{reach}(c)$ ("c' is an ancestor-or-self of c").

h is fixed for the whole report. The retry and convergence guarantees of SPEC §13.7c are stated at a fixed h; advancing it may legitimately add, remove, or redirect work. Structural termination and single-state determinism do not need that restriction.

Remark 2. Ancestry, not commit date or log order, is the only ordering used. This is what makes cancellation deterministic under merges and rebases (SPEC §10.4) and what lets the model ignore clocks entirely. One consequence is used without comment below: since h reaches every commit considered and the graph is acyclic, $h \preceq c$ holds for no $c \neq h$.

2.2 Workspace

Definition 3 (workspace graph, SPEC §13.1, SPEC §8.4)**.** $\mathcal{P}$ is the finite set of packages present at h. For a fixed set of edge kinds K_{prop} (the configured `propagation.kinds`), $E \subseteq \mathcal{P} \times \mathcal{P}$ contains (a, b) when a declares a dependency on b of a kind in K_{prop}. The *dependents* of p are $\mathrm{dep}(p) = \{d : (d, p) \in E\}$. The graph $(\mathcal{P}, E)$ is acyclic. Two further kind sets, K_{order} (`publish.orderKinds`) and K_{block} (`publish.blockingKinds`), select the edges of the publish order and of failure blocking (SPEC §19.2, SPEC §19.3); by default K_{prop} and K_{order} contain `optionalDependencies` and K_{block} does not (SPEC §14).

Acyclicity is an assumption the model may make because the engine enforces it: a cycle over the propagation or publish-ordering kinds is the repository-scoped error `E200` and aborts the run before any plan is computed. SPEC §13.1 justifies rejecting rather than accommodating cycles: no publish order exists for a cycle, so whichever member goes first declares a range on a version that does not yet exist, and if the run then fails that state is permanent.

Propagation travels *against* the dependency edges, from a changed package to the packages that consume it; that is, along $\mathrm{dep}(\cdot)$.

Example 4 (the running workspace, SPEC §22)**.** The running examples use the six-package workspace shared by many conformance vectors:

$$\mathcal{P} = \{\, \texttt{core},\ \texttt{cli},\ \texttt{ui},\ \texttt{api},\ \texttt{docs-site},\ \texttt{@acme/theme} \,\},$$

$$E = \{\, (\texttt{cli}, \texttt{core}),\ (\texttt{ui}, \texttt{core}),\ (\texttt{api}, \texttt{core}),\ (\texttt{docs-site}, \texttt{ui}),\ (\texttt{@acme/theme}, \texttt{ui}) \,\},$$

drawn in Figure 1, with the initial tag state

$$\sigma_0 = \{\ \texttt{core@1.4.2},\ \texttt{cli@2.0.0},\ \texttt{ui@0.9.1},\ \texttt{api@1.2.0},\ \texttt{@acme/theme@1.0.0}\ \}.$$

`docs-site` has never been released, so it has no baseline; it is private and publishes to an internal registry, which by SPEC §13.10a exempts it from nothing.

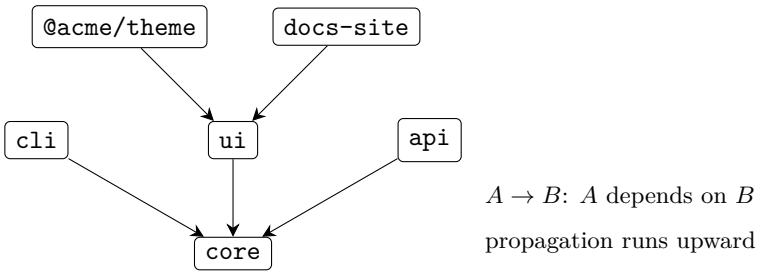


Figure 1: The running workspace of Example 4. Distances from $S = \{\texttt{core}\}$ are $\delta(S, \texttt{cli}) = \delta(S, \texttt{ui}) = \delta(S, \texttt{api}) = 1$ and $\delta(S, \texttt{docs-site}) = \delta(S, \texttt{@acme/theme}) = 2$.

Definition 5 (distance, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $d \in \mathcal{P}$, $\delta(S, d)$ is the least n for which there are $p_0 \in S$ and $p_1, \ldots, p_n = d$ with $p_{i+1} \in \mathrm{dep}(p_i)$ for all i, and ∞ if no such chain exists. Note $\delta(S, d) = 0$ exactly when $d \in S$. We say p *reaches* d when $\delta(\{p\}, d) < \infty$; equivalently, d depends on p transitively over K_{prop}.

2.3 Bumps and versions

Definition 6 (bumps, SPEC §7.1, SPEC §9.1)**.** $\mathbb{B} = \{\mathsf{none}, \mathsf{patch}, \mathsf{minor}, \mathsf{major}\}$ ordered $\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}$. Being a finite chain, $(\mathbb{B}, \leq)$ is a join-semilattice with least element none and join $\sqcup = \max$.

The whole of SPEC §9.1's "bumps accumulate" is the statement that the combining operator is this join. Two consequences that SPEC states separately are then immediate and are used without further comment: combination is associative, commutative and idempotent, so the order in which contributions are gathered cannot matter; and $x \sqcup y \geq x$, so propagation can only ever raise a package's bump, never lower or replace a direct decision.

Definition 7 (versions and channels, SPEC §11.1–SPEC §11.4)**.** $\mathcal{V}$ is the set of SemVer 2.0.0 versions with SemVer precedence. $\mathcal{C}h = \{\mathsf{stable}\} \cup \mathcal{N}$ for a set $\mathcal{N}$ of channel names. $\mathrm{ch}(v) = \mathsf{stable}$ if v carries no prerelease identifier, and otherwise the first (alphanumeric) prerelease identifier of v. $\mathrm{core}(v)$ is the `major.minor.patch` triple of v and $\mathrm{cnt}(v)$ the numeric counter of a prerelease. $\mathrm{apply}(v, b)$ applies bump b to v, with $\mathrm{apply}(v, \mathsf{none}) = v$.

2.4 State

Everything above is fixed by h and the configuration. Exactly one thing changes from run to run, and isolating it is what makes the guarantees provable as statements about a sequence.

Definition 8 (engine state, SPEC §13.2, SPEC §12.3)**.** A state σ is a finite set of release tags $(p, v, c) \in \mathcal{P} \times \mathcal{V} \times \mathcal{K}$ with $c \preceq h$. States are ordered by inclusion, $\sigma \leq \sigma'$ iff $\sigma \subseteq \sigma'$. From σ:

- $T_\sigma(p) = \{\, c : (p, v, c) \in \sigma \,\}$ is the set of *release commits* of p;
- $\mathrm{base}_\sigma(p)$ is the highest-precedence version among the tags of p in σ, undefined if there are none;
- $\mathrm{basec}_\sigma(p)$ is the commit of that tag;
- $\mathrm{sbase}_\sigma(p)$ is the highest-precedence non-prerelease version among them, and $\mathrm{scom}_\sigma(p)$ its commit.

Where $\mathrm{base}_\sigma(p)$ is undefined, $\mathrm{ch}(\mathrm{base}_\sigma(p)) = \mathsf{stable}$ by convention (SPEC §11.1): a never-released package is graduated by nothing and is put on a train by any directive naming it.

We say p is *tagged at* h in σ when $h \in T_\sigma(p)$.

The baseline functions additionally assume a unique highest-precedence reachable tag for each package and each stable baseline. States with tied precedence and different commits need a specified tie rule and are outside this proof's domain. Operational lock and temporary refs are not release tags and are not members of σ. In every state considered here a tag at h, where there is one, is the baseline: it was written by a run at h (Definition 25) at a version exceeding the baseline of that run (`E195`), so $h \in T_\sigma(p)$ implies $\mathrm{basec}_\sigma(p) = h$.

That release-tag states only ever grow is not an assumption about tidy operators; it is normative. SPEC §19.1 forbids moving or deleting a pushed tag and SPEC §12.4 makes tags authoritative over manifests, because a tag is the record that a leg of the transaction completed and an unwritten-back or rewritten record is exactly what makes a saga unrecoverable.

Definition 9 (train and undelivered windows, SPEC §13.3)**.**

$$W_\sigma(p) \;=\; \mathrm{reach}(h) \setminus \mathrm{reach}(\mathrm{scom}_\sigma(p)), \tag{1}$$

and $W_\sigma(p) = \mathrm{reach}(h)$ when p has no stable baseline in σ. The undelivered window is

$$W^\circ_\sigma(p) = W_\sigma(p) \setminus \mathrm{reach}(\mathrm{basec}_\sigma(p)),$$

with an empty subtracted set when p has no baseline tag.

$W_\sigma(p)$ is the train window used to aggregate a prerelease target from its stable base. $W^\circ_\sigma(p)$ is the fresh-admission window for direct work, corrections, channels, and propagated channels to dependent p. For a stable baseline they coincide; on a train they must not be conflated.

Definition 10 (delivery, SPEC §13.4a)**.** For packages P, D and a commit C,

$$\mathrm{dlv}_\sigma(P, C, D) \;\equiv\; \exists\, t \in T_\sigma(P) :\; C \preceq t \;\wedge\; t \preceq \mathrm{basec}_\sigma(D),$$

and $\mathrm{dlv}_\sigma(P, C, D)$ is false when D has no baseline. Read: D has released at or after a release of P that carries C.

Delivery is a relation between two packages' release positions, decided by ancestry alone. Releasing past a commit is not delivery: a D whose baseline reaches C but reaches no release of P carrying C has not had C delivered by P. Only the source's release, followed by the target's, is (SPEC §13.4a).

Definition 11 (owed window, SPEC §13.3)**.** For every edge $(D, P) \in E$ where D has a baseline,

$$W^{\mathrm{ow}}_\sigma(P, D) \;=\; \mathrm{reach}(h) \setminus \bigcup \{\, \mathrm{reach}(t) \;:\; t \in T_\sigma(P),\; t \preceq \mathrm{basec}_\sigma(D) \,\},$$

the commits reachable from h and from no release commit of P that $\mathrm{basec}_\sigma(D)$ reaches.

Lemma 12 (the owed window is the undelivered set)**.** *For $(D, P) \in E$ with D having a baseline, and every $c \preceq h$, $c \in W^{\mathrm{ow}}_\sigma(P, D)$ iff $\neg\mathrm{dlv}_\sigma(P, c, D)$.*

Proof. $c \notin W^{\mathrm{ow}}_\sigma(P, D)$ iff $c \in \mathrm{reach}(t)$ for some $t \in T_\sigma(P)$ with $t \preceq \mathrm{basec}_\sigma(D)$, which is Definition 10 verbatim. □

SPEC §13.3 adds the owed windows to the union of windows that SPEC §13.4 parses, so that a debt D is owed stays *visible* after D has released past the commit and P has released it: once P has, no window of P holds the commit, and if D released past it before P did, no window of D does either. The window makes a debt visible; whether it is owed is Definition 19's question, and the window admits nothing by itself.

Remark 13 (two cases SPEC §13.3 describes, and one it does not). SPEC §13.3 says that where the newest release of P that $\text{basec}_\sigma(D)$ reaches is P's own baseline, nothing is added. That is right: then $\text{reach}(\text{basec}_\sigma(P))$ is among the subtracted reaches, so $W^{\text{ow}}_\sigma(P, D) \subseteq W^{\circ}_\sigma(P) \subseteq W_\sigma(P)$, and this is the position of every consumer that never got ahead of its provider. It also says that where $\text{basec}_\sigma(D)$ reaches no release of P nothing is added, because D released before P's first release and $W_\sigma(D)$ already holds everything since. Definition 11 read literally gives $W^{\text{ow}}_\sigma(P, D) = \text{reach}(h)$ in that case, which is not "nothing"; and the prose's reason covers only commits after D's release, while a commit $c \preceq \text{basec}_\sigma(D)$ that P first released at a commit $\text{basec}_\sigma(D)$ does not reach is, by Definition 10, undelivered and owed, and lies in neither $W_\sigma(D)$ (for a stable D) nor $W_\sigma(P)$. The model follows the formula, which is the reading the guarantees need; the two sentences of SPEC §13.3 need reconciling, and that is reported as a specification obligation rather than resolved here.

Example 14 (windows)**.** Let the history be $c_0 \to c_1 \to c_2 = h$ with σ_0 of Example 4, where the tags sit at c_0. Then

$$W_{\sigma_0}(\texttt{core}) = \{c_1, c_2\}, \qquad W_{\sigma_0}(\texttt{docs-site}) = \{c_0, c_1, c_2\},$$

the second because a package with no stable baseline has the whole reachable history pending: SPEC §12.5 makes an unreleased package's first release account for everything, which is also why adopting the engine on an existing repository does not need a migration step. The owed window $W^{\text{ow}}_{\sigma_0}(\texttt{core}, \texttt{cli})$ subtracts $\text{reach}(c_0)$, the one `core` release that $\text{basec}_{\sigma_0}(\texttt{cli}) = c_0$ reaches, and equals $W_{\sigma_0}(\texttt{core})$: nothing is added for a consumer level with its provider. Now let $\sigma_1 = \sigma_0 \cup \{\texttt{cli@2.1.0} \text{ at } c_2\}$, the state after `cli` has proceeded past c_2 on a cause of its own while `core` failed (Example 39). Then $W^{\circ}_{\sigma_1}(\texttt{cli}) = \emptyset$, $W_{\sigma_1}(\texttt{cli}) = \emptyset$, and still $W^{\text{ow}}_{\sigma_1}(\texttt{core}, \texttt{cli}) = \text{reach}(c_2) \setminus \text{reach}(c_0) = \{c_1, c_2\}$, because $\text{basec}_{\sigma_1}(\texttt{cli}) = c_2$ reaches no `core` release newer than c_0. If `core` then releases at c_2, $W^{\text{ow}}(,)$ becomes empty as well: `core` has delivered c_2 to `cli`, or the ledger reads as if it had (Proposition 26).

Remark 15 (the window is measured from the last *stable* tag). This is deliberate and it serves SPEC §11.4: a package on a prerelease train must accumulate every commit since its last *stable* release, because the train's target version is recomputed from the stable baseline on every run so that a breaking change arriving mid-train moves the whole train. The consequence, which Section 10 returns to at length, is that a commit already published in a prerelease of p is still a member of $W_\sigma(p)$.

2.5 Units

Definition 16 (units, SPEC §4.4, SPEC §13.3, SPEC §13.4, SPEC §13.4b)**.** $\mathcal{U}$ is the set of units carried by commits in

$$\bigcup_{p \in \mathcal{P}} W_\sigma(p) \;\cup \bigcup_{(D,P) \in E,\ D \text{ has a baseline}} W^{\text{ow}}_\sigma(P, D),$$

after parsing (SPEC §20), scope resolution (SPEC §6) and the application of corrections (SPEC §13.4b). For $u \in \mathcal{U}$: $c_u \in \mathcal{K}$ is its commit; $\text{scope}(u) \subseteq \mathcal{P}$ its resolved scope-set; $\text{bump}(u) \in \mathbb{B}$ the bump its type and `!` map to; and its directives are

$$\text{pb}(u) \in \mathbb{B}, \quad \text{pd}(u) \in \mathbb{N} \cup \{\infty\}, \quad \text{ps}(u) \subseteq \mathcal{P}, \quad \text{pc}(u), \quad \text{pcd}(u), \quad \text{pcs}(u) \subseteq \mathcal{P},$$

where $\text{pb}(u)$ is the *effective* propagated bump: the value the unit states, or $\text{bump}(u)$ where it states `inherit`. The two axes default differently and the asymmetry is deliberate (SPEC §8.1). `Propagate` defaults to `patch`, not to `inherit`: a dependent reached by propagation has had a

dependency change under it but no change to its own public API, so patch is the honest signal, and `feat(core)^` therefore gives its consumers a patch rather than a minor. `Propagate-Channel` defaults to `inherit`, since a channel is not a quantity that can be understated. Both defaults apply only once the corresponding depth is non-zero, and both depths default to 0 (SPEC §8.3, SPEC §8.3a): a unit that says nothing releases its own packages and nothing else. $\mathrm{pc}(u)$ is a channel value: a name, `stable`, `inherit`, `none`, or a transition $\langle f \to t \rangle$.

Taking corrections as already applied is what SPEC §13.4b prescribes: it requires `Edits` and `Deletes` to be resolved into the tuple stream before cancellation runs, and notes that every later phase operates on the corrected stream, which downstream phases cannot distinguish from one that was authored that way.

Remark 17 (the corrected stream depends on σ). It does not follow, and SPEC §13.4b does not claim, that the stream is independent of the state. A correction reaches only *undischarged* work: it applies to a (package, unit) pair only while that package's window still contains the target commit, and once the package has released the target the correction is a no-op for it (`W209`, SPEC §7.4.2). So $\mathcal{U}$ is properly a function $\mathcal{U}_\sigma$, and a record suppressed by a `Deletes` for a package returns to the stream for that package once the package releases. This matters in exactly one place, and it is the same place that a lapsed cancellation matters: proof obligation 139. Everywhere else the two readings coincide, because a released package's own window is empty and its direct contribution is therefore already spent, so the notation $\mathcal{U}$ is kept unindexed below. The retry invariant of Section 6 fixes the corrected stream outright, which also fixes the parsed set.

Remark 18 (what the owed windows make visible, and what they do not). The owed windows of Definition 16 range over the *direct* edges of E. A debt, by Definition 10, is between a unit's source and any target the unit reaches, at any depth. For a target at distance 1 the two coincide: if P owes D and $(D, P) \in E$, then $c_u \in W^{\mathrm{ow}}_\sigma(P, D)$ by Lemma 12, and the unit is parsed. For a target at distance 2 or more it need not: with $D \to M \to P$, the commit c_u is parsed for D's sake only through $W()$ of some package or through $W^{\mathrm{ow}}(P, M)$ and $W^{\mathrm{ow}}(M, D)$, and once M has been delivered c_u by P and D has been delivered c_u by M (which says nothing about P's release reaching D), every one of those windows can be empty while $\mathrm{dlv}_\sigma(P, c_u, D)$ is still false. The model transcribes SPEC §13.3 as written; the theorems of Section 6 fix the parsed stream by the retry invariant and are unaffected. The gap is reported as a specification obligation rather than repaired here.

Definition 19 (suppression, source packages and owed sources, SPEC §13.4a)**.** $\mathrm{canc}(c, p)$ holds when c is an ancestor-or-self of some `cancel` commit whose resolved scope contains p (SPEC §10.3); $\mathrm{held}(p)$ holds when p's effective `Release-As` directive is `none` (SPEC §13.6a). Both are computed from the corrected stream, h, configuration, and release-tag state. They are constant across retries only where the retry invariant says those inputs remain fixed. Write

$$\mathrm{disch}_\sigma(p, c) \;\equiv\; c \notin W^{\circ}_\sigma(p),$$

and then

$$\mathrm{src}_\sigma(u) \;=\; \{\, p \in \mathrm{scope}(u) \;:\; \mathrm{disch}_\sigma(p, c_u) \;\vee\; \neg(\mathrm{canc}(c_u, p) \vee \mathrm{held}(p)) \,\}.$$

For a target D, the *owed sources* of u are

$$\mathrm{owed}_\sigma(u, D) \;=\; \{\, P \in \mathrm{src}_\sigma(u) \;:\; \neg\mathrm{dlv}_\sigma(P, c_u, D) \,\},$$

and P *still owes* D, written $\mathrm{owes}_\sigma(P, D)$, when $P \in \mathrm{owed}_\sigma(u, D)$ for some unit u with $\mathrm{adm}_\sigma(u, D)$ (Definition 37).

The disjunction in src() is the normative content of SPEC §13.4a: suppression applies only to *undischarged* work. Once p has published the version carrying u, the artefact its consumers are owed is public, and neither a later `cancel` (a no-op, `W170`) nor a later hold may retroactively strand those consumers. Proof obligation 139 shows that this clause, and not the traversal, is where **G5** is decided. The owed set is the other half of the same section: on the bump axis a unit propagates to D while $\mathrm{owed}_\sigma(u, D)$ is non-empty, and D's provenance names exactly the owed sources.

Lemma 20 (delivery implies discharge at the target)**.** *If* $\mathrm{dlv}_\sigma(P, c, D)$ *then* $c \notin W^\circ_\sigma(D)$. *Consequently, if* $c_u \in W^\circ_\sigma(D)$ *then* $\mathrm{owed}_\sigma(u, D) = \mathrm{src}_\sigma(u)$.

Proof. Delivery gives t with $c \preceq t \preceq \mathrm{basec}_\sigma(D)$, so $c \in \mathrm{reach}(\mathrm{basec}_\sigma(D))$, which Definition 9 subtracts from $W^\circ_\sigma(D)$. For the consequence, no source of u has delivered c_u to D, so none is removed from $\mathrm{src}_\sigma(u)$. □

So for a target that has not released past the commit, the owed set is the whole source set and SPEC §13.4a's delivery test reduces to the window test of CCME 2.0.0. The finer question arises only for a target that released past c_u before its source did: a consumer that proceeded on a cause of its own while the source failed (SPEC §19.3), or one released while the source was held (SPEC §13.6a). Under the polyrepository profile the comparison of Definition 10 is made in P's repository (SPEC §27); that profile is outside this report.

2.6 Runs

Definition 21 (plan)**.** Given σ, the release computation of SPEC §13 yields the bumps $\mathrm{dir}_\sigma(p)$ and $\mathrm{prp}_\sigma(p)$ (Definition 38), the channels $\mathrm{chan}_\sigma(p)$ (Definition 34), and $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$. The plan is

$$\Pi(\sigma) \;=\; \{\, p \in \mathcal{P} \;:\; \neg\mathrm{held}(p) \;\wedge\; (\mathrm{eff}_\sigma(p) \neq \mathsf{none} \;\vee\; \mathrm{chan}_\sigma(p) \neq \mathrm{ch}(\mathrm{base}_\sigma(p)) \;\vee\; p \text{ has an applicable exact } \texttt{Release-As} \text{ in } W^\circ_\sigma(p)) \,\},$$

each $p \in \Pi(\sigma)$ carrying the version $\nu_\sigma(p)$ of SPEC §13.9. The fresh bumps above decide whether work is newly admissible; they are not by themselves the prerelease target. On a train, the intended calculation aggregates the surviving corrected units in $W_\sigma(p)$ from the stable baseline, including work already delivered to an earlier prerelease. For graduation, the model applies the specific rule of SPEC §11.5: publish that same train target without a prerelease suffix. The generic summary in SPEC §13.9 is inconsistent with this rule; proof obligation 133 records the textual obligation.

Of the disjuncts, the direct bump, the channel change and the exact version all read $W^\circ_\sigma(p)$, and are therefore absent for a package tagged at h; only $\mathrm{prp}_\sigma(p)$ can put such a package in the plan (Lemma 84). That asymmetry is what SPEC §19.3's "cause of its own" is about.

Remark 22 ($\Pi(\sigma)$ is the released set, not the printed one). SPEC §13.9 computes a would-be version for a held package too, and SPEC §13.6a requires the engine to report it (`W154`) so that the number needed to lift the hold is available without hand computation. That reported entry is part of what an operator calls "the plan", but it is not a release: SPEC §13.10 excludes it. $\Pi(\sigma)$ above is the released set, which is why Theorem 98 needs no exception for holds and why the convergence check of SPEC §19.6 states one: the two are reading the same word differently, and only SPEC §19.6's reading has to be told about held packages.

Definition 23 (a cause of its own, SPEC §19.3)**.** For $p \in \Pi(\sigma)$ and a set $X \subseteq \mathcal{P}$ of packages that have failed or been blocked earlier in the run, p has a *cause of its own* relative to X, written $\mathrm{own}_\sigma(p, X)$, when at least one of the following holds: $\mathrm{dir}_\sigma(p) \neq$ `none`; $\mathrm{chan}_\sigma(p) \neq \mathrm{ch}(\mathrm{base}_\sigma(p))$; p has an applicable exact `Release-As` in $W^\circ_\sigma(p)$; or there is a unit u with $\mathrm{adm}_\sigma(u, p)$ and $\mathrm{owed}_\sigma(u, p) \not\subseteq X$, a contribution from a provider that is neither failed nor blocked, including one that published in an earlier run.

SPEC §19.3 lists a fresh direct bump, a channel change and a contribution from an unfailed provider; the model adds the exact-version disjunct of Definition 21, which the section does not name, because a package pinned by `Release-As` has a cause that is its own by any reading. The addition is flagged for the specification. The reference implementation follows the section as written: its test of a cause of its own is a fresh unit with a bump other than `none`, a channel change, or an update from a provider that did not fail, so a consumer whose only cause is an exact `Release-As` is blocked behind a failed provider there, and the theorems below read the engine with the disjunct removed.

Definition 24 (well-formed state, SPEC §19.3, `E201`)**.** $\mathrm{wf}(\sigma)$ holds when for every $P \in \Pi(\sigma)$ and every D with $\mathrm{basec}_\sigma(D) = h$ and $\mathrm{owes}_\sigma(P, D)$: $D \in \Pi(\sigma)$ and P precedes D in the publish order of Definition 44.

A run from a state that is not well-formed would release a package at the baseline commit of a consumer it still owes without that consumer releasing after it. Two releases on one commit have no order (SPEC §13.7b), so afterwards nothing could tell that the consumer came first: it would read as served and stay on the old version with nothing left to detect it. SPEC §19.3 therefore refuses such a run with `E201` before any publication, naming the pair; the remedy is to release the consumer in the same run, or the provider at a later commit, where the debt stays visible (Definition 11).

Definition 25 (run, SPEC §19.1–SPEC §19.3)**.** A run from σ is parameterised by a set $F \subseteq \Pi(\sigma)$ of publishes that fail. If $\neg\mathrm{wf}(\sigma)$ the run is refused: $\mathrm{run}(\sigma, F) = \sigma$. Otherwise the plan is processed in the publish order of Definition 44. Let X_p be the set of packages before p in that order that failed or were blocked. Then p is *blocked* (`W194`) when it transitively depends over K_{block}, in the full workspace graph, on a member of X_p and $\neg\mathrm{own}_\sigma(p, X_p)$; it *fails* when it is not blocked and $p \in F$; and it is *published* otherwise. Write $B(\sigma, F)$ for the blocked set and $S(\sigma, F) = \Pi(\sigma) \setminus (F \cup B(\sigma, F))$ for the published set. The run yields

$$\mathrm{run}(\sigma, F) \;=\; \sigma \;\cup\; \{\, (p, \nu_\sigma(p), h) \;:\; p \in S(\sigma, F) \,\}.$$

The run *reports `E201` for* D when $h \in T_\sigma(D)$, $D \in F$, and some $P \in S(\sigma, F)$ has $\mathrm{owes}_\sigma(P, D)$.

Four features of this definition carry weight later.

F is chosen adversarially and independently on each run. The guarantees are claims about *every* failure schedule, not about a well-behaved one; a proof that appeals to what usually fails proves nothing about a system whose purpose is to survive what does not usually happen.

Every tag is written at h, never at the commit the work arrived in (SPEC §19.1). This is what makes Lemma 78 a two-case statement rather than an argument about which commits a tag covers, and what makes Lemma 80 a statement about one commit.

A package is tagged if it is released, whatever its publish target, including targets of `none` that upload nothing (SPEC §13.10a). Proposition 101 shows this is forced: the alternative is a permanent hole in **G6**.

Blocking is not the transitive closure of failure. A dependent with a cause of its own *proceeds*, its manifests reconciled to what its providers have published by then (SPEC §19.5), and what

the failed provider owed it stays owed and arrives as a catch-up when that provider publishes. This is the consumer-ahead shape of SPEC §13.7a, and it is why $S(\sigma, F)$ may contain a package that was already tagged at h.

Proposition 26 (the one loss, SPEC §19.3)**.** *Let* $\mathrm{wf}(\sigma)$, $\sigma' = \mathrm{run}(\sigma, F)$, *and let the run report* `E201` *for* D *because of* $P \in S(\sigma, F)$. *Then for every unit* u, $P \notin \mathrm{owed}_{\sigma'}(u, D)$, *although* D *published nothing after* P. *If* $\mathrm{owed}_{\sigma'}(u, D) = \emptyset$ *for every* u *with* $\mathrm{adm}_{\sigma}(u, D)$, *then* $D \notin \Pi(\sigma')$, *and no state reachable from* σ' *by runs at this or any later* h *plans the contribution again.*

Proof. D was not released, so $\mathrm{basec}_{\sigma'}(D) = \mathrm{basec}_{\sigma}(D) = h$, and $h \in T_{\sigma'}(P)$. For any u, $c_u \preceq h \preceq \mathrm{basec}_{\sigma'}(D)$ with $t = h$, so $\mathrm{dlv}_{\sigma'}(P, c_u, D)$ and P is not owed. If every admitted unit's owed set is now empty, $\mathrm{prp}_{\sigma'}(D) = \mathsf{none}$ (Definition 38); $W^{\circ}_{\sigma'}(D) = \emptyset$ since $\mathrm{basec}_{\sigma'}(D) = h$, so no other disjunct of Definition 21 holds. A later state adds tags, which keeps the witness $t = h$, and a later h changes neither $T(P)$ nor $\mathrm{basec}(D)$ while D is unreleased; so $\mathrm{dlv}(P, c_u, D)$ persists and the owed set stays empty until D itself releases, which nothing in the ledger now asks for. □

This is the one place where the ledger loses an obligation. The run reports it with its remedy, an explicit `Release-As` on the consumer at a later commit (SPEC §8.6), because no later plan can compute the debt: two tags on one commit have no order. Everywhere else the protocol refuses before the fact (Definition 24) or catches up after it (Theorem 87). The theorems of Section 6 state where they exclude this event.

Definition 27 (run sequence)**.** A *run sequence* at h is $\sigma_1 \leq \sigma_2 \leq \cdots$ with $\mathrm{wf}(\sigma_1)$ and $\sigma_{k+1} = \mathrm{run}(\sigma_k, F_k)$ for some $F_k \subseteq \Pi(\sigma_k)$. By Definition 25 it is monotone: $\sigma_k \subseteq \sigma_{k+1}$, and $T_{\sigma_{k+1}}(p) \setminus T_{\sigma_k}(p) \subseteq \{h\}$ for every p. Lemma 97 shows that under the retry invariant every σ_k is then well-formed, so no run of the sequence is refused.

3 Propagation

This section transcribes SPEC §9.2, the three-phase procedure that assigns each package a propagated channel, resolves every package's channel, and then assigns propagated bumps. The phase order is forced: phase 3 reads what phase 2 produces, and SPEC §9.2 forbids fusing them.

3.1 The procedure, as written

The pseudo-code is reproduced from SPEC §9.2 unchanged, so that a reader can diff it against `SPEC.md`. The definitions that follow are its clauses, one by one.

```
propagate(units, graph, held, W):
    edges = graph.restrictedTo(config.propagation.kinds)          # ğ8.4

    # ---------- shared, computed once ----------
    # sourcePackages is a pure function of the unit (ğ13.4a) and both passes read it.
    # Computing it inside each pass evaluates it twice for every unit carrying both axes.
    src = { u: sourcePackages(u) for u in units }                 # ğ13.4a

    # ---------- phase 1: channel axis ----------
    chan = {}                                  # package -> (channel, commit, unitIndex)
    for u in units:
        if u.channelDepth == 0:                    continue       # ğ8.3a: default
        if u.propagateChannel == none:             continue
```

```
        sources = src[u]
        if sources is empty:                          continue
        c      = commitOf(u)                            # loop-invariant
        pscope = resolve(u.propagateChannelScope)       # ğ8.5a: resolved once,
        origin = UNCOMPUTED                            # memoized on first admitted target
        for (d, level) in reach(edges, sources, u.channelDepth):  #   never per target
            if d not in pscope:                         continue
            if c not in Wfresh(d):                           continue # admission, ğ13.4a
            if cancelledFor(c, d):                      continue # ğ13.5a
            if u.propagateChannel == 'inherit' and origin == UNCOMPUTED:
                origin = originChannel(u)
            v = propagatedChannelFor(u, d, origin)          # ğ9.3; origin computed once
            if v is NONE:                               continue
            chan[d] = newerOf(chan[d], (v, u))              # W160 on conflict

    # ---------- phase 2: channel resolution ----------
    channel = resolveChannels(chan, units)              # ğ13.8

    # ---------- phase 3: bump axis ----------
    prop = {}                                  # package -> bump
    prov = {}                                  # package -> set of origin packages
    for u in units where bumpOf(u) != none:
        b = (u.propagate == 'inherit') ? bumpOf(u) : u.propagate
        if b == none or u.depth == 0:                continue
        sources = src[u]
        if sources is empty:                          continue
        c       = commitOf(u)
        pscope  = resolve(u.propagateScope)             # ğ8.5: once per unit
        # ğ9.3a resolvability depends only on the unit's sources, so it is decided here,
        # once, and reduces to membership in the distinct source-channel set per target.
        srcChan   = { channel[P] : P in sources }       # ğ9.3a
        anyStable = 'stable' in srcChan
        for (d, level) in reach(edges, sources, u.depth):
            if d not in pscope:                         continue
            owed = { P in sources : not delivered(P, c, d) }    # admission, ğ13.4a
            if owed is empty:                           continue
            if cancelledFor(c, d):                      continue
            if not (anyStable or channel[d] in srcChan):  continue # ğ9.3a, W208
            prop[d] = max(prop[d], b)
            prov[d] = prov[d] | owed
    return prop, prov, chan
```

The two axes admit differently, and the difference is the subject of this report's revision. Phase 1 tests `c not in Wfresh(d)`: a dependent is reached on the channel axis if and only if it has not itself released past the commit carrying the unit. Phase 3 tests `owed is empty`: a dependent is reached on the bump axis if and only if some source of the unit has not yet *delivered* that commit to it (Definition 10). The two coincide except for a dependent that released past the commit before its source did (Lemma 20), and `prov[d]` records exactly the owed sources.

3.2 The traversal

Definition 28 (reach, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $\ell \in \mathbb{N} \cup \{\infty\}$,

$$R(S, \ell) \;=\; \{\, d \in \mathcal{P} \setminus S \;:\; \delta(S, d) \le \ell \,\},$$

and each $d \in R(S, \ell)$ carries the level $\delta(S, d)$.

```
reach(edges, sources, depth):
    frontier = sources
```

```
seen     = set(sources)                    # a unit never reaches its own packages
out      = {}                              # package -> level
for level in 1 .. depth:                   # 'all' => until frontier is empty
    next = {}
    for p in frontier:
        for d in edges.dependentsOf(p):
            if d in seen: continue
            seen.add(d); next.add(d)
    if next is empty: break
    for d in next: out[d] = level
    frontier = next
return out
```

Lemma 29 (the traversal computes Definition 28)**.** *On termination,* `out` *has domain* $R(S, \ell)$ *and* `out`$[d] = \delta(S, d)$.

Proof. Standard breadth-first search from the set S, seeded at level 0. By induction on `level`: `frontier` at the start of iteration n is exactly $\{d : \delta(S, d) = n - 1\} \setminus S$ for $n > 1$ and S for $n = 1$, because `seen` contains precisely the packages at distance $\leq n - 1$ (together with S at level zero), and a package first enqueued at iteration n has a chain of length n from S and none shorter. Membership of `seen` is tested before insertion and never cleared, so each package is assigned at most one level, its least. The loop runs for ℓ iterations or until the frontier is empty. Seeding `seen` $= S$ removes S from the output, which is Definition 28's $\mathcal{P} \setminus S$. □

Three properties recorded by SPEC §9.2 are visible in this statement and are used repeatedly below. Depth is shortest-path, so a package reachable at both depth 1 and depth 3 is included by `+1`. Depth is measured from the originating source set as a whole and never recomputed from an intermediate package, so a unit written `+1` reaches exactly the direct consumers of its own packages in every run, whatever released in between. And a package reached by several paths is visited once, so a diamond does not double-count.

Example 30 (depth, on Figure 1)**.** Take $S = \{$`core`$\}$, the source set of a unit scoped `(core)`:

Written	$R(S, \ell)$	Note
`feat(core)`	$\emptyset$	$\ell = 0$, the default
`feat(core)^`	{`cli`, `ui`, `api`}	direct consumers
`feat(core)+2`	$R(S, 1)$ ∪ {`docs-site`, `@acme/theme`}	two edges out
`feat(core)^^`	the same set	$\ell = \infty$; the graph is only two deep

Now take the two-package source set $S' = \{$`core`, `ui`$\}$, as a unit scoped `(core,ui)` would have. Then $R(S', 1) = \{$`cli`, `api`, `docs-site`, `@acme/theme`$\}$: `ui` is *absent* although it is a dependent of `core` at distance 1, because Definition 28 removes the whole of S'. The unit bumps `ui` directly, and a package is never both a direct and a propagated target of the same unit. Proposition 122 is about exactly this clause.

Remark 31 (self-exclusion is per unit)*.* $R(S, \ell)$ excludes all of S, not merely the source a given path started from. This is what stops a unit from propagating to a package it already bumps directly. It is also the clause that makes the bucketed traversal of SPEC §13.11 unsound if transcribed naively; see Proposition 122.

3.3 Phase 1: the channel axis

Definition 32 (proposed channel, SPEC §9.3)**.** For a unit u and package d, let $cur = \mathrm{ch}(\mathrm{base}_\sigma(d))$ (stable if d has no baseline). Then $\mathrm{pcf}_\sigma(u,d) \in \mathcal{C}h \cup \{\bot\}$ is:

- if $\mathrm{pc}(u)$ is a transition $\langle f \to t\rangle$: $\bot$ if $f = t$ (`W207`); $\bot$ if cur does not match f, where $f = *$ matches any $cur \neq$ stable and otherwise f matches only $cur = f$; else $target = t$;
- otherwise $target$ is $\mathrm{pc}(u)$, or the origin channel of u where that is `inherit`; and if $target =$ stable while $cur \neq$ stable, the result is $\bot$ (`W200`: a propagated channel never graduates implicitly);
- in either case, if $target = cur$ the result is $\bot$ (`W199`); otherwise it is $target$.

The origin channel of u is the channel u's own `Channel` directive assigns to its sources if it carries one, and otherwise $\mathrm{ch}(\mathrm{base}_\sigma(p))$ for its sources p, taking the value of the byte-wise least name if they disagree (`W160`).

The final clause complements the target $W^\circ(a)$dmission gate: it compares $target$ against the package's *baseline* channel rather than against anything computed in the current run. This no-op check is the other premise of **G7**, and Theorem 102 is its consequence.

Definition 33 (phase 1 output, SPEC §9.2)**.** $\mathrm{prop}\chi_\sigma(d)$ is the value $\mathrm{pcf}_\sigma(u,d)$ of the $\prec$-greatest unit u (newest commit first, then last unit within a commit, SPEC §11.6) satisfying: $\mathrm{pcd}(u) \neq 0$; $\mathrm{pc}(u) \neq$ `none`; $\mathrm{src}_\sigma(u) \neq \emptyset$; $d \in R(\mathrm{src}_\sigma(u), \mathrm{pcd}(u))$; $d \in \mathrm{pcs}(u)$; $c_u \in W^\circ_\sigma(d)$; $\neg\mathrm{canc}(c_u, d)$; and $\mathrm{pcf}_\sigma(u,d) \neq \bot$. It is undefined if no unit qualifies.

The channel axis keeps $c_u \in W^\circ_\sigma(d)$ as its admission because a channel is carried by the units and needs no release of the source (SPEC §13.4a, **G7**); it has no notion of delivery.

3.4 Phase 2: channel resolution

Definition 34 (channel resolution, SPEC §13.8)**.** For every $p \in \mathcal{P}$, whether or not it will be released, let $base = \mathrm{ch}(\mathrm{base}_\sigma(p))$ and let $D_\sigma(p)$ be the value proposed by the $\prec$-greatest unit u with $p \in \mathrm{scope}(u)$, $c_u \in W^\circ_\sigma(p)$, $\neg\mathrm{canc}(c_u, p)$, that carries a `Channel` directive proposing something for p (a transition matching $base$, or a value differing from $base$; a directive proposing nothing is not a competitor and does not count towards `W186`). Then

$$\mathrm{chan}_\sigma(p) \;=\; \begin{cases} D_\sigma(p) & \text{if defined,} \\ \mathrm{prop}\chi_\sigma(p) & \text{else if defined,} \\ base & \text{otherwise.} \end{cases}$$

A direct directive beats every propagated one regardless of age; among direct directives, and separately among propagated ones, the newest commit wins and then the last unit within it. A package named by nothing keeps its baseline channel, which is why an established train needs no directives to stay together.

3.5 Phase 3: the bump axis

Definition 35 (resolvability, SPEC §9.3a)**.** $\mathrm{sch}_\sigma(u) = \{\mathrm{chan}_\sigma(p) : p \in \mathrm{src}_\sigma(u)\}$ and

$$\mathrm{rsv}_\sigma(u,d) \;\equiv\; \text{stable} \in \mathrm{sch}_\sigma(u) \;\vee\; \mathrm{chan}_\sigma(d) \in \mathrm{sch}_\sigma(u).$$

A bump is a claim that the dependent has something new to pick up, and across a channel boundary the claim is false: a stable consumer resolves a beta-releasing dependency by its stable range exactly as before, so a republished consumer would contain byte-for-byte what its predecessor contained. Suppression is reported per suppressed pair as `W208`. Note that `srcChan` in the pseudo-code is computed over the whole source set, not over the owed sources; Section 8 returns to this.

Example 36 (a caret that reaches and does not bump: SPEC vector 95)**.** The unit `feat(core)^%beta: x` on Figure 1. The `%` puts `core` on `beta`; the `^` reaches $\{\texttt{cli}, \texttt{ui}, \texttt{api}\}$. Phase 2 leaves the three consumers where their baselines put them, on stable, so

$$\mathrm{sch}_\sigma(u) = \{\texttt{beta}\}, \qquad \mathsf{stable} \notin \mathrm{sch}_\sigma(u), \qquad \mathrm{chan}_\sigma(\texttt{cli}) = \mathsf{stable} \notin \mathrm{sch}_\sigma(u),$$

and Item (A6) fails for every one of them: three `W208` diagnostics, and $\Pi(\sigma) = \{\texttt{core}\}$ alone. The caret was honoured and reached its targets; what it would have delivered is not resolvable by them. Writing `feat(core)^%beta++1` instead moves the same three onto `beta` in phase 1, so $\mathrm{chan}_\sigma(\texttt{cli}) = \texttt{beta} \in \mathrm{sch}_\sigma(u)$ and all four packages release, which is the form that keeps a train installable as a set.

Definition 37 (admission, SPEC §9.2, SPEC §13.4a)**.** For $u \in \mathcal{U}$ and $d \in \mathcal{P}$, $\mathrm{adm}_\sigma(u, d)$ holds iff all of:

(A1) $\mathrm{bump}(u) \neq \mathsf{none}$, $\mathrm{pb}(u) \neq \mathsf{none}$, $\mathrm{pd}(u) \neq 0$;
(A2) $\mathrm{src}_\sigma(u) \neq \emptyset$ and $d \in R(\mathrm{src}_\sigma(u), \mathrm{pd}(u))$;
(A3) $d \in \mathrm{ps}(u)$;
(A4) $\mathrm{owed}_\sigma(u, d) \neq \emptyset$;
(A5) $\neg\mathrm{canc}(c_u, d)$;
(A6) $\mathrm{rsv}_\sigma(u, d)$.

Definition 38 (propagated and direct bumps, SPEC §13.6, SPEC §13.7)**.**

$$\mathrm{prp}_\sigma(d) = \bigsqcup\{\, \mathrm{pb}(u) \;:\; u \in \mathcal{U},\ \mathrm{adm}_\sigma(u, d) \,\},$$
$$\mathrm{dir}_\sigma(p) = \bigsqcup\{\, \mathrm{bump}(u) \;:\; p \in \mathrm{src}_\sigma(u) \cap \mathrm{scope}(u),\ c_u \in W^\circ_\sigma(p) \,\},$$

both empty joins being none. Then $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$.

Item (A4) is the load-bearing clause of the entire design, and it is worth being explicit about what it is and is not. It tests the *target's* position against its *sources'* releases: d is admitted while some source has not delivered c_u to it. It does not test the source's window, and it does not test the target's window either. SPEC §13.7a shows what each of the other readings costs. With retention governed by the source's window, a dependent whose publish failed while its provider's succeeded is silently orphaned, permanently one patch behind a dependency it declares, with no diagnostic, because from the engine's point of view nothing is pending. With admission governed by the target's window, the rule of CCME 2.0.0, the same orphan is reached from the other side: a dependent that proceeded past the commit on a cause of its own while its provider failed is owed nothing once the provider publishes, and keeps the range it reconciled against the old version. By Lemma 20 the delivery test agrees with the window test on every target that has not released past c_u; it differs only on a target that has, where it asks the finer question. Every guarantee in Section 6 rests on Item (A4) being this test, and Theorem 87 is where that is used.

Example 39 (the orphan, from both sides: SPEC vectors 66 and 80d)**.** Both shapes use the history of Example 14, $c_0 \to c_1 \to c_2 = h$, with the tags of σ_0 at c_0 and the unit $u =$

`feat(core)^: streaming reader` at $C = c_2$. So $\text{scope}(u) = \{\texttt{core}\}$, $\text{bump}(u) = \textsf{minor}$, $\text{pb}(u) = \textsf{patch}$ (the default of Definition 16) and $\text{pd}(u) = 1$.

Provider fails to deliver; the consumer waits (vector 66). Run 1 evaluates Definition 37 with σ_0. All of `cli`, `ui`, `api` lie in $R(\{\texttt{core}\}, 1)$, none is cancelled, all are on `stable` as is `core`, and $C \in W^\circ_{\sigma_0}(d)$ for each, so by Lemma 20 $\text{owed}_{\sigma_0}(u, d) = \{\texttt{core}\}$, all three are admitted and

$$\Pi(\sigma_0) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}, \qquad \nu_{\sigma_0}(\texttt{core}) = 1.5.0, \quad \nu_{\sigma_0}(\texttt{cli}) = 2.0.1.$$

Suppose `core`, `ui` and `api` publish and `cli` fails, so

$$\sigma_1 = \sigma_0 \cup \{\texttt{core@1.5.0},\ \texttt{ui@0.9.2},\ \texttt{api@1.2.1}\} \quad \text{at } C.$$

Run 2 is where the readings of Item (A4) separate. Under delivery, the normative reading, $C \not\preceq \text{basec}_{\sigma_1}(\texttt{cli}) = c_0$, so $\neg\text{dlv}_{\sigma_1}(\texttt{core}, C, \texttt{cli})$ and $\text{owed}_{\sigma_1}(u, \texttt{cli}) = \{\texttt{core}\}$: `cli` is admitted. Under the target's window, the reading of CCME 2.0.0, $C \in W^\circ_{\sigma_1}(\texttt{cli}) = \{c_1, C\}$: `cli` is admitted. Under the source's window, the trap, $C \notin W_{\sigma_1}(\texttt{core}) = \emptyset$: u reaches nobody. The first two agree, as Lemma 20 says they must for a `cli` that has not released past C. For `ui`, which has, $\text{dlv}_{\sigma_1}(\texttt{core}, C, \texttt{ui})$ holds with $t = C \preceq C = \text{basec}_{\sigma_1}(\texttt{ui})$: nothing is owed, and `ui` is out. Under the normative rule $\Pi(\sigma_1) = \{\texttt{cli}\}$ at the version run 1 planned, marked `W193`, and $\Pi(\sigma_2) = \emptyset$. Under the trap $\Pi(\sigma_1) = \emptyset$ and `cli` is never released, on this run or any later one, with no diagnostic anywhere.

Consumer gets ahead of its provider (vector 80d). Let C carry a second unit $u' = \texttt{feat(cli): y}$ beside u. Then $\text{dir}_{\sigma_0}(\texttt{cli}) = \textsf{minor}$, and run 1 plans $\Pi(\sigma_0) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}$ with $\nu_{\sigma_0}(\texttt{cli}) = 2.1.0$. Suppose `core` fails. `ui` and `api` have no cause of their own relative to $X = \{\texttt{core}\}$ and are blocked (`W194`); `cli` has one, its direct minor, and *proceeds* (Definition 23): it is published at 2.1.0 with its manifest reconciled to `core`'s published baseline 1.4.2, and tagged at C. So

$$\sigma_1 = \sigma_0 \cup \{\texttt{cli@2.1.0 at } C\}, \qquad W^\circ_{\sigma_1}(\texttt{cli}) = \emptyset.$$

Run 2. `core` is planned by its direct minor. For `cli`, the window reading says $C \notin W^\circ_{\sigma_1}(\texttt{cli})$ and admits nothing: `cli` keeps the range it reconciled against 1.4.2, is never planned again, and is one minor behind a dependency it declares, silently. The delivery reading asks whether `core` has delivered C to `cli`: $T_{\sigma_1}(\texttt{core}) = \{c_0\}$ and $C \not\preceq c_0$, so $\text{owed}_{\sigma_1}(u, \texttt{cli}) = \{\texttt{core}\}$, `cli` is admitted with $\text{prp}_{\sigma_1}(\texttt{cli}) = \textsf{patch}$, and

$$\Pi(\sigma_1) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}, \qquad \nu_{\sigma_1}(\texttt{cli}) = 2.1.1,$$

ordered after `core` and blocked if `core` fails again. σ_1 is well-formed (Definition 24): `core` would be released at $C = \text{basec}_{\sigma_1}(\texttt{cli})$ and still owes `cli`, and `cli` is in the plan after it. Once both are tagged, $\text{dlv}_{\sigma_2}(\texttt{core}, C, \texttt{cli})$ holds with $t = C$, nothing is owed, and $\Pi(\sigma_2) = \emptyset$. The plan of run 2 equals the plan of run 1 although run 1 published a package; that is the observation Theorem 98 has to accommodate.

Example 40 (two tags on one commit: `E201`, vector 80d continued)**.** Continue the second shape from σ_1. Suppose run 2 publishes `core`, `api` and `ui` and `cli` fails. Then $\sigma_2 = \sigma_1 \cup \{\texttt{core@1.5.0}, \texttt{api@1.2.1}, \texttt{ui@0.9.2}\}$ at C, and now $\text{dlv}_{\sigma_2}(\texttt{core}, C, \texttt{cli})$ holds with $t = C \preceq C = \text{basec}_{\sigma_2}(\texttt{cli})$: the ledger reads `cli` as served, although `cli@2.1.1` was never published and `cli@2.1.0` still names 1.4.2. $\text{owed}_{\sigma_2}(u, \texttt{cli}) = \emptyset$, $\Pi(\sigma_2) = \emptyset$, and no later plan at any commit will compute the debt, because two tags on one commit have no order. This is Proposition 26: run 2 reports `E201` for the pair, and the remedy is an explicit `Release-As` on `cli` at a later commit. The other route to the same state is prevented rather than reported: a run 2 in which `cli` sat out, held or deselected, would release `core` at `cli`'s baseline commit without `cli` following, and SPEC §19.3 refuses it before any publication.

Had `core` instead published at a later commit c_3 in a run `cli` sat out, C would stay in $W^{\text{ow}}(\texttt{core}, \texttt{cli}) = \text{reach}(c_3) \setminus \text{reach}(c_0)$ (Definition 11) and the next run would plan `cli`'s catch-up.

Remark 41 (catch-up is not a mode). There is no separate catch-up pass in this model, and none in the specification. Catch-up is what the algorithm does when the ordinary rule is evaluated against the right position (SPEC §13.7a). A "catch-up release" is a label for a release whose entire cause is a propagation from a package not itself in the run's plan (`W193`); the engine does nothing different to produce it, and vector 80d's second run is not one, since `core` is in that plan. Catch-up does not widen depth, resurrect a cancelled contribution, release a held package, change a planned version, or propagate onward from the catching-up package; each of which is a property of Definition 37 rather than a rule about catch-up.

4 The publish protocol

The guarantees are joint properties of the computation and the protocol that writes the state the next computation reads. SPEC §19 says so explicitly: they do not survive an implementation that gets this section wrong. This section transcribes the obligations the proofs use, in the order a run meets them: the comparison of release records under the lock, tagging, publication order, build readiness, partial failure, reconciliation, and manifest writes. The durations these obligations lead to, and their distributed and multi-repository forms, are the subject of Section 5.

4.1 Records under the lock

Definition 8 takes the state σ to be the set of release tags reachable from h. SPEC §13.2 says whose tags: the authoritative store's, not the checkout's.

Definition 42 (records under the lock, SPEC §13.2)**.** A write-capable run plans from the release records of the authoritative store it records to, not from whatever its checkout fetched. After it holds every release lock it needs and before it fixes the input of SPEC §13.2, it compares, for each participating repository, the release records of the store with the records it is about to plan from. A stored record whose commit is reachable from the planned head and which the planning input lacks is incomplete history, `E196`. A stored record that names another commit than the planning input's record of the same package and version is `E191`. Neither difference is repaired by planning the package as unreleased, and the records are not refreshed after the comparison: a run that fetches does so before it and plans from what it then holds. Only what SPEC §12.1 parses as a release tag of a workspace package is a record here; a ref the implementation moves by design, its lock, and a name that merely resembles a tag are not. A store whose records cannot be read is `E196`. Read-only planning takes no lock and makes no comparison; its result describes the checkout. The comparison reads one record inventory per repository, $O(T)$ each, and asks one ancestry question per record the input lacks, which is none in the ordinary case.

The lock alone serialises runs and isolates nothing: two runs that never overlap still plan the same version when the second one's checkout predates the first one's records. The comparison is what the lock makes sufficient, because no other coordinated run can add a record between the comparison and this run's own. Complete history makes reachability decidable locally, since a commit the checkout does not hold cannot be reachable from its head. A run under an unsafe lock bypass (`W331`) still compares; nothing then holds other runs off after the

comparison, so the exposure is narrowed and not closed. Under SPEC §27 the inventory is read per owning repository and cross-repository consumer positions are reconstructed as Section 5.3 describes; commit identifiers of different repositories are never compared as one ancestry relation.

4.2 Tag after, immediately, always, and once

Definition 43 (tagging, SPEC §19.1)**.** For each released package: the tag is created at h; *after* that package's publish succeeds, never before; *immediately* after it, not batched to the end of the run; for every released package whatever its publish target, including targets that upload no artefact at all; and *once*. A release tag is created, never replaced. Writing it to the authoritative store fails when the store already holds that tag name, unless the stored tag names the same commit for the same package and version, which is the retry of an uncertain write (Definition 55). A forced ref update is never used for a release tag, and a tag is not moved or deleted once pushed. A ref the implementation moves by design is not a release record and is outside this rule. For a target of `none` there is no publish step: the tag is written once the package's manifest write succeeds.

Each clause excludes a specific failure. Tagging before publishing makes the version unrecoverable, because Definition 9 will treat it as released. Batching tags to the end of a run makes a mid-run failure republish everything already published, which most registries reject outright. Tagging only what uploads an artefact is the subject of Proposition 101. The create-only clause is what makes Definition 8 monotone in fact and not only by convention: a record, once written, is never overwritten by a later writer, which is also what SPEC §28.6 relies on when an orchestrator records a publication it authorised before losing its lock (Definition 70).

4.3 Order

Definition 44 (publish order, SPEC §19.2)**.** Let G be the workspace graph restricted to `publish.orderKinds` over *all* packages. Publication proceeds in a topological order of G, ties broken by byte-wise least name, filtered to the plan at the end.

Proposition 45 (the order exists, is total, and is deterministic)**.** *Definition 44 is well defined and yields the same sequence on every machine and in every conforming implementation.*

Proof. G is acyclic by Definition 3, so a topological order exists. Ties are broken by a total order on names (byte-wise comparison of distinct names), so the order is total and independent of any container's iteration order. The filter is applied to a sequence, preserving it. □

Remark 46 (order over the whole graph, then filter). Inducing the graph on the planned packages *first* is a defect, not an optimisation. With $A \to B \to C$ where B has no bump this run (the common case in any incremental release), the induced subgraph on $\{A, C\}$ has no edge at all, so A may publish against a C that has not published yet. The constraint is real even though B is not released, because A still resolves C through B at install time. This is why the model orders over $\mathcal{P}$ and filters last, and why an implementation detecting a violation must raise `E197`.

Order matters for a reason no later run can repair. A dependent published before its dependency carries a range reconciled against the dependency's old version, and both land at the same commit, so no ancestry test can tell afterwards that anything is wrong (SPEC §13.7b). The inconsistency is undetectable after the fact, which is why it is prevented rather than diagnosed. `E197` is scoped to Definition 44 and not to propagation: every workspace dependency

edge between two planned packages constrains the order, whether or not either package's bump arrived by propagation, because a dependent planned for its own unrelated `feat` still resolves its dependency at install time (SPEC §22, vector 80a). An `E197` that fires only for propagated pairs is under-reporting.

4.4 Build readiness

Definition 44 orders publication. An engine that also runs the packages' builds has a second question for every edge between two packages that build in this run: what the consumer's build reads of the provider.

Definition 47 (build readiness relation, SPEC §19.2a)**.** For every workspace dependency edge $C \to P$ between two packages that build in this run, the edge carries a declared *build readiness relation* with one of three values, ordered `none` < `build` < `publish`:

- `publish`: C's build reads P as an installer resolves it, the published artefact and its record, and starts after P is published and recorded (Definition 43);
- `build`: C's build reads what P's build leaves in the workspace, and starts after P's build has succeeded;
- `none`: C's build reads nothing that P's build or publication produces in this run, and waits for nothing of P.

C's build includes every preparation step of C that can read the provider, lockfile regeneration in particular; manifest reconciliation (Definition 56) reads planned versions only and waits for nothing. Each precondition implies the one before it, because a package builds before it publishes. Under all three values P publishes before C does: Definition 44 is unchanged, is not a fourth value, and cannot be declared away. The relation is *declared, never inferred*: an engine cannot check that a build reads nothing, so it neither infers `none` nor makes it a default. It is *policy, not plan*: it never changes which packages release or at which versions, is no part of the input of SPEC §13 or of a plan digest, and two runs of one plan under different relations record the same releases when every task succeeds; under SPEC §28 it travels with the run's resolved configuration and a node's own settings cannot change it. It is *ordered over the whole graph*: for two packages C and P that build in this run, a dependency path from C to P with no `none` edge on it places P's build before C's, whether or not the packages between them are in the plan, and a `none` edge ends the constraint of every path through it.

The reference implementation reads the relation at the consumer's own edge only, and carries build and publish *order* through a non-building intermediate by a memoised reach (`release/stagerelation.go`), never the intermediate's relation value; that is the reading taken here for a `publish` edge behind a package that does not build, and the question is flagged for the specification (Section 10).

`none` therefore means build in parallel and publish in order, which is the relation of a deployment order: infrastructure before the application that runs on it, a schema before the service that reads it. Vector 80c of SPEC §22 is the whole-graph clause: with `app` → `ui` → `core` where `app` and `core` build and `ui` does not, `core`'s build precedes `app`'s when both edges are `build`, and no build constraint exists when either edge is `none`, while `core` publishes before `app` in every row. Adding build edges only between a package and its direct providers in the plan loses the first row, exactly as inducing the publish graph on the plan loses vector 80a.

Definition 48 (task graph, SPEC §19.2a)**.** The *task graph* of a run has a node build(X) for every package X that builds and a node publish(X) for every $X \in \Pi(\sigma)$, and the edges (arrows denote happens-before): build(X) → publish(X); publish(P) → publish(C) whenever

P precedes C in Definition 44 along a dependency path; build(P) $\to$ build(C) for every dependency path from C to P with no `none` edge, both building; and publish(P) $\to$ build(C) for every edge $C \to P$ declared `publish`.

Proposition 49 (no new cycle, SPEC §19.2a)**.** *For every choice of relations the task graph is acyclic. Weakening a relation only removes edges.* ***`E197`*** *cannot arise from a relation.*

Proof. Let r number the packages in a topological order of the workspace graph with providers first, so that $r(P) < r(C)$ whenever C depends on P over a dependency path. Give each task the key $(r(X), s)$ with $s = 0$ for a build and $s = 1$ for a publication, ordered lexicographically. Every edge of Definition 48 strictly increases the key: build(X) $\to$ publish(X) raises s at fixed r, and the three remaining forms raise r, since each runs along a dependency path from a consumer C to a provider P with $r(P) < r(C)$. A cycle would return to its starting key, which a strictly increasing key cannot do. The only edges a relation contributes are of the third and fourth forms; `publish` contributes both, `build` the third, `none` neither, so a weaker value contributes a subset. The publication order of Definition 44 reads no relation, so a violation of it (`E197`) cannot be caused by one. □

Failure is the business of Definition 50. Under `build` and `none`, C may be prepared and built against P's planned version before the outcome of P's publication is known. That is sound because Definition 50 then either blocks C or, where C has a cause of its own, reconciles its manifest to P's published version before C publishes and leaves P's contribution owed (Definition 19): a manifest naming a version that was never published is never published itself. A build that embedded the planned version is rebuilt, or C is blocked; a build finished for a blocked consumer confers nothing, is not published, discharges no obligation, and the next run builds again or reuses it under the identity rules of SPEC §28.5. Under SPEC §28 a `build` edge is the local-output dependency and a `publish` edge the registry-availability edge of Definition 69; a `none` edge carries no output into the consumer's build task, and the requirement that task inputs include every build dependency is met because the declaration says there is none.

4.5 Partial failure

Definition 50 (own causes, blocking, and proceeding, SPEC §19.3)**.** Let $F \subseteq \Pi(\sigma)$ be the packages whose publish failed. A package $d \in \Pi(\sigma)$ that transitively depends over `publish.blockingKinds`, through the *full* workspace graph, on a member of F or on a blocked package is *blocked* (`W194`) when it has no cause of its own, and *proceeds* otherwise. A cause of its own is a fresh direct bump, a channel change, or a contribution from a provider that is neither failed nor blocked, including one that published in an earlier run. A blocked package is planned but not attempted in this run. A proceeding package publishes at its planned version with its manifests reconciled to what its providers have published (Definition 56); what the failed provider owed it stays owed (Definition 19) and arrives as a catch-up (`W193`) when that provider publishes. Packages that depend on no failed or blocked package continue to publish: an unrelated subtree is not punished for another's failure. The run reports what published, what failed and what was blocked, and exits non-zero.

The closure traverses packages that are not in the plan: a package with no bump this run is still a path from a dependent to a failed dependency (vector 80a). Ordering and blocking use different edge-kind sets, and the asymmetry is deliberate: `orderKinds` includes optional dependencies, because an optional dependency should still be available before its dependent, while `blockingKinds` excludes them, because a missing optional dependency does not break

an install and a missing required one does. The pseudo-code of SPEC §19.3 is transcribed verbatim (the section sign is mapped to § for the typewriter font and nothing else is changed):

```
run(plan):
    published, failed, blocked = {}, {}, {}
    for P in publishOrder(plan, graph):            # a flat sequence, §19.2
        bad = transitiveBlockingDeps(P, graph) & (failed | blocked)   # full graph, §19.3
        if bad and ownCauses(P, plan, bad) is empty:
            blocked[P] = W194; continue
        reconcileRanges(P, published)              # §9.4, §19.5: providers as published by now
        try:
            if publishTarget(P) != none:           # §13.10a
                publishTo(publishTarget(P), P, plan[P].version)   # or adopt, §19.4
            tag(P, plan[P].version, HEAD)          # §19.1: always, immediately
            published[P] = plan[P].version
        except PublishError as e:
            failed[P] = e
            if config.publish.onFailure == 'abort': return report(...)
    return report(published, failed, blocked)
```

`publishOrder` returns a flat, totally ordered sequence, not levels, so `run` iterates it directly. Publishing strictly one at a time is the normative reading; an implementation may publish concurrently only if it preserves the same happens-before relation and reports in the sequence order regardless (SPEC §17.2). On re-run the plan is recomputed from tags: packages already tagged fall out by SPEC §13.6, packages that failed or were blocked remain candidates by Definition 19, and a dependent that proceeded returns as a catch-up once the failed provider publishes.

Remark 51 (the blocked set of Definition 25). Definition 25 takes blocking in exactly this sense: its blocked set is the closure of the failed set restricted to packages without a cause of their own relative to what failed or was blocked before them in the publish order (Definition 23), and a proceeding dependent belongs to its published set at its planned version. The proofs of Section 6 that quantify over $F \cup B$ use only that a package in $F \cup B$ gained no tag. What a proceeding dependent is still owed is the subject of Definition 19 and of the delivery rule it transcribes, not of the blocking closure.

Definition 52 (the owed-commit rule, SPEC §19.3)**.** A package P is not released at the baseline commit of a consumer D it still owes (Definition 19) unless D is in the plan and is published after P in this run. Such a run is refused with `E201` before any publication, naming the pair; the remedy is to release the consumer in the same run, or the provider at a later commit, where the debt stays visible in the owed window of SPEC §13.3. If D is published after P and fails, the same state arises, and `E201` reports it with its remedy, an explicit `Release-As: <version>` on the consumer (SPEC §8.6), because no later plan can compute the debt.

Definition 24 states the rule as a well-formedness condition on the state a run starts from, and Definition 25 refuses a run from a state that violates it. Two releases on one commit cannot be ordered afterwards (SPEC §13.7b). A consumer that sat the run out, held, deselected or excluded by any other means, would then read as served: the delivery test of SPEC §13.4a finds a release of P on a commit that D's baseline reaches, namely D's own baseline commit, and nothing is left to detect that D came first. The rule prevents that state rather than diagnosing it.

Example 53 (an infrastructure leg)**.** Nothing in Definitions 25 and 43 requires a publish leg to be a registry upload. A `terraform apply` may be such a leg: its build stage imports the

explicitly configured live resources and writes a plan, and its publish stage applies that plan before the release tag is written. The tag records that this version of the configuration completed successfully. It is not Terraform resource state, does not prove that every live resource was imported, and does not make applying an arbitrary configuration externally idempotent. The edge from the application to the infrastructure package is the case Definition 47 names: the application's build reads nothing the apply produces, so the edge is declared `none`, the two builds overlap, and only the application's publication waits for the infrastructure publish. Example 64 gives the durations.

Remark 54 (recorded-success abstraction). Definition 25 models confirmed successful legs by their tags. It proves properties of that release ledger, not convergence of an external registry or infrastructure state. A publish may take effect before its tag is recorded; then the repository alone is insufficient and Definition 55 or a publish-target-specific idempotence contract is required. Terraform likewise needs its own state or an explicit, complete import boundary before a plan can safely describe existing resources.

4.6 Reconciliation and manifest writes

Definition 55 (reconciliation, SPEC §19.4)**.** When a publish fails because the version already exists, the implementation verifies by registry digest or provenance attestation that the published artefact is the one this run would have published; if it matches, the publish counts as successful and the tag is written (`W196`); if it does not match, or identity cannot be established, the run stops with `E198`.

This is the one place where the model's "a publish either succeeds or fails" abstraction meets a third outcome, that it already happened, and Definition 55 maps it back onto success. Verification is what makes that safe: without it, adopting a version published by something other than this run would enrol a foreign artefact into the release lineage, which is a supply-chain vulnerability rather than a convenience.

Definition 56 (manifest writes, SPEC §19.5)**.** Before a released package publishes, its manifest is updated so that it declares no range excluding the version each workspace dependency has *published* by the time the package is reconciled: the dependency's planned version when it is in this run's plan and has published, and its current baseline otherwise (`W197` when that baseline came from an earlier run). A dependency that is in the plan but failed or was blocked has published nothing new; a dependent that proceeds past it (Definition 50) is reconciled to the baseline, and `W193` follows when the dependency publishes. Because the graph is acyclic and Definition 44 publishes every dependency before its dependents, the planned and published versions coincide at the moment each package is reconciled whenever the dependency published; reconciliation may therefore be computed once up front from the plan and corrected, before the dependent's publish, only for the providers that did not publish. A manifest naming a version that was never published is never published, whatever was computed up front. The writes are part of the publish step, not the plan, and are not committed back in a way that itself parses as a bump-producing unit.

5 Execution: build readiness, distributed placement, and repository-local windows

Section 4 states what must happen before what. This section states what that costs in wall time, where the tasks may run, and which history each window ranges over when the packages

live in several repositories. Its first subsection is informative in the sense of SPEC §13.11: the propositions bound durations of schedules that respect Definition 47, and no conforming implementation is required to attain them. Its second and third subsections transcribe the two optional profiles of SPEC §28 and SPEC §27 as definitions and state the one invariance each profile promises. Neither extends the theorems of Section 6, which are stated for a single history.

5.1 Build readiness durations

SPEC §19.2a promises a happens-before relation and no duration; SPEC §13.11 nevertheless says what each relation buys when nothing else limits the run. Throughout, package i has a build of duration b_i and a publication of duration p_i, both fixed; a task starts as soon as its predecessors in the task graph of Definition 48 have finished and a slot is free; and one relation is declared on every edge.

Proposition 57 (the longest path, SPEC §13.11)**.** *With unlimited build and publication slots a run lasts exactly the longest path L of its task graph, measured in task durations. A weaker relation never lengthens it.*

Proof. The task graph is acyclic (Proposition 49), so its tasks can be taken in a topological order. With unlimited slots a task starts at the greatest finish time of its predecessors, or at 0 if it has none, and by induction along that order its finish time is the longest path ending at it. The run ends at the greatest finish time, which is the longest path of the graph. A weaker relation contributes a subset of the edges (Proposition 49), so every path of its task graph is a path of the stronger one's, and the longest is no longer. □

Proposition 58 (a chain, SPEC §13.11)**.** *Let packages $1, \dots, n$ form a chain in which package $k+1$ consumes package k, under one relation throughout. With unlimited slots the run lasts*

$$\sum_{i=1}^{n}(b_i + p_i) \quad \textit{under } \texttt{publish}, \qquad \max_{1\le k\le n}\Big(\sum_{i=1}^{k} b_i + \sum_{i=k}^{n} p_i\Big) \quad \textit{under } \texttt{build},$$

$$\max_{1\le k\le n}\Big(b_k + \sum_{i=k}^{n} p_i\Big) \quad \textit{under } \texttt{none}.$$

Proof. Under `publish`, build$(k+1)$ waits for publish(k), so the $2n$ tasks form one path and the sum is its length. Under `build`, the builds form the path build$(1) \to \cdots \to$ build(n) and build(k) ends at $B_k = \sum_{i\le k} b_i$; the publication of k waits for build(k) and for publish$(k-1)$, so its finish E_k satisfies $E_k = \max(B_k, E_{k-1}) + p_k$ with $E_0 = 0$. By induction $E_k = \max_{j\le k}(B_j + \sum_{i=j}^{k} p_i)$: the case $k = 1$ is $B_1 + p_1$, and the step is $\max(B_k + p_k,\ \max_{j<k}(B_j + \sum_{i=j}^{k-1} p_i) + p_k)$. Under `none` every build starts at 0 and build(k) ends at b_k; the same recurrence with b_k in place of B_k gives the third form. In each case the run lasts E_n by Proposition 57. □

Corollary 59 (equal durations)**.** *With $b_i = b$ and $p_i = p$ for all i, the three forms are $n(b+p)$, $\max(nb+p,\ b+np)$ and $b+np$. `build` saves $(n-1)\min(b,p)$ over `publish`, and `none` saves a further $(n-1)\max(b-p, 0)$, so the second saving exists only where builds outlast publications.*

Proof. Under `build` the maximand $kb+(n-k+1)p$ is affine in k, so the maximum is at $k = 1$ or $k = n$. Under `none` the maximand $b + (n-k+1)p$ is greatest at $k = 1$. The differences follow by cases on $b \ge p$. □

Proposition 60 (fan-out and fan-in, SPEC §13.11)**.** *For one provider* 0 *and consumers* $i \geq 1$*, each consuming* 0 *and nothing else, the run lasts*

$$b_0 + p_0 + \max_i(b_i + p_i), \qquad b_0 + \max_i\big(\max(b_i, p_0) + p_i\big), \qquad \max_i\big(\max(b_i,\ b_0 + p_0) + p_i\big)$$

under `publish`*,* `build` *and* `none` *respectively. For providers* $i \geq 1$ *of one consumer* c*, each consumed by* c *and nothing else, it lasts*

$$\max_i(b_i + p_i) + b_c + p_c, \qquad \max\big(\max_i b_i + b_c,\ \max_i(b_i + p_i)\big) + p_c, \qquad \max\big(b_c,\ \max_i(b_i + p_i)\big) + p_c.$$

Proof. Fan-out. Under `publish`, consumer i's build starts at $b_0 + p_0$ and its publication follows its build, giving $b_0 + p_0 + b_i + p_i$; take the maximum. Under `build`, its build starts at b_0 and ends at $b_0 + b_i$, and its publication waits for that and for publish(0) at $b_0 + p_0$, giving $b_0 + \max(b_i, p_0) + p_i$. Under `none`, its build ends at b_i and its publication waits for that and for $b_0 + p_0$. Fan-in. Under `publish`, c's build starts when the last provider has published, at $\max_i(b_i + p_i)$. Under `build`, it starts at $\max_i b_i$ and ends b_c later, and c's publication waits for that and for every provider's publication. Under `none`, c's build ends at b_c and only its publication waits for the providers. Each is the longest path of Proposition 57. □

Proposition 61 (one publication lane, SPEC §13.11)**.** *Where one owner's publications are serialised (*SPEC *§27.2,* SPEC *§28.6) in the order* $1, \ldots, n$ *of Definition 44, let* e_k *be the instant the build of package* k *ends. Under every relation the run lasts*

$$\max_{1 \leq k \leq n} \Big(e_k + \sum_{i=k}^{n} p_i\Big),$$

which is at least $\sum_i p_i$ *whatever the relation, and a weaker relation only lowers the* e_k*.*

Proof. Publication k starts at $\max(e_k, E_{k-1})$, since it waits for its own build and for the lane, and the lane order agrees with Definition 44, so every provider of k has published by E_{k-1}. The recurrence is that of Proposition 58 with e_k in place of B_k, and the same induction gives the maximum. The term $k = 1$ is $e_1 + \sum_i p_i \geq \sum_i p_i$. For the last claim, add the lane edges $\text{publish}(k-1) \to \text{publish}(k)$ to the task graph; they follow the publish order, which is a topological order of the workspace graph, so the key of Proposition 49 still increases along every edge and the graph stays acyclic. With a fixed lane order every task starts at the greatest finish of its predecessors, and by induction along a topological order each finish time under a subset of the edges is at most the finish time under the full set; e_k is such a finish time. □

Proposition 62 (budgets, SPEC §13.11)**.** *Let there be* m_b *build slots and* m_p *publication slots, and write* $W_b = \sum_i b_i$*,* $W_p = \sum_i p_i$ *and* L *for the longest path of the task graph. No schedule finishes before* $\max(L,\ W_b/m_b,\ W_p/m_p)$*. A schedule that never leaves a slot of a stage idle while a task of that stage is ready finishes within* $L + W_b/m_b + W_p/m_p$*, which is within three times the best possible. Under budgets a weaker relation can lengthen such a schedule, but never the best one.*

Proof. The lower bound: a schedule contains every path, so it lasts at least L; it performs W_b units of build work on m_b slots, so at least W_b/m_b; and likewise W_p/m_p. The upper bound: let the schedule end at T and let t_1 be a task finishing at T. For $j \geq 1$, while t_j has predecessors, let t_{j+1} be one that finishes last among them; this gives a chain $t_q \to \cdots \to t_1$ whose total duration is at most L. Consider an instant of $[0, T)$ at which no task of the chain is running. It lies between the finish of some t_{j+1} and the start of t_j (or before the start of t_q),

and at that instant t_j is ready, since its latest predecessor has finished, and has not started; by hypothesis every slot of t_j's stage is then busy. The instants at which all m_b build slots are busy total at most W_b/m_b, and those at which all m_p publication slots are busy at most W_p/m_p. Hence $T \leq L + W_b/m_b + W_p/m_p$, and each summand is at most the lower bound. For the last claim, a schedule feasible under a stronger relation is feasible under a weaker one, so the best weaker schedule is no longer; that a particular greedy schedule can be longer is Example 63. □

The bound is Graham's argument for list scheduling under precedence constraints [12, 13], applied to two stages with separate budgets; SPEC §13.11 does not claim it tight for this shape of graph.

Example 63 (Graham's anomaly, SPEC §13.11)**.** Take two build slots and one publication slot, packages $A = (2, 3)$ and $B = (1, 2)$ where B consumes A, and an unrelated $C = (8, 3)$, with ready tasks taken in the order A, B, C. Under `build`, B's build is not ready at 0: A builds in $[0, 2)$ and C in $[0, 8)$ on the two slots; B builds in $[2, 3)$; A publishes in $[2, 5)$, B in $[5, 7)$ once A has published, and C in $[8, 11)$ once its build ends. The run lasts 11. Under `none`, B's build is ready at 0 and takes the second slot: A builds in $[0, 2)$ and B in $[0, 1)$, so C builds only in $[1, 9)$; A publishes in $[2, 5)$, B in $[5, 7)$ and C in $[9, 12)$. The run lasts 12: the consumer's early build took the slot the long build would have had. A list that preferred C to B at 0 would finish at 11 under `none` as well, which is the point: the length depends on the list, not on the relation, and it is why SPEC §19.2a promises a happens-before relation and no duration.

Example 64 (an infrastructure chain)**.** Let the infrastructure leg of Example 53 be consumed by a backend consumed in turn by a frontend, with (b, p) in minutes of $(4, 6)$ for the infrastructure, $(9, 3)$ for the backend and $(12, 2)$ for the frontend. By Proposition 58, under `publish` the run lasts $10 + 12 + 14 = 36$ minutes; under `build` it lasts $\max(4 + 11,\ 13 + 5,\ 25 + 2) = 27$; under `none` it lasts $\max(4 + 11,\ 9 + 5,\ 12 + 2) = 15$. The publications still run in order in every case, and the frontend's publication still ends the run.

5.2 Distributed execution

SPEC §28 is an optional execution profile that changes no message syntax and asserts no speedup. The definitions below transcribe its obligations; the one proposition is the invariance it promises.

Definition 65 (nodes and roles, SPEC §28.1)**.** An execution node is a machine or runner executing the same release engine and configuration schema; it is distinct from a repository peer. Every node supports two roles, and one engine provides both. The default role is *orchestrator*. An explicit *worker* refuses release initiation, including indirect initiation from a delegated hook, and executes authorised tasks in an existing run; it never substitutes a locally recomputed plan, acquires competing release ownership, dispatches to another pool, or independently finalises a release. An orchestrator-capable node that accepts a delegated task has only worker authority for it. The node on which CI starts the release is the run's orchestrator, of which there is one per run; it owns lock acquisition, the fixed planning input, task allocation, result admission and release finalisation, and may execute tasks locally under the same rules as a worker. Workers may be added to every history mode; execution links are not fleet links and alter neither source discovery nor release scope.

Definition 66 (lock, snapshot, plan, assign, SPEC §28.3)**.** Before creating a write-capable run the orchestrator identifies every participating repository, acquires all release locks in one

stable total order, verifies ownership, and revalidates discovery under those locks; a changed participant set releases the partial acquisition and acquires the complete set again. Discovery and compatibility inspection before locking are read-only, and an unsafe lock bypass is refused. The run input $\mathcal{I}$ contains the fixed repository-qualified heads and complete relevant history, the release records of the authoritative stores as compared under the locks (Definition 42), the graph, the resolved semantic configuration, the explicit release options, and the withdrawal inventory and receipts of SPEC §26. The semantic release plan is the pure function $\mathrm{Plan}(\mathcal{I})$. Node placement configuration is not semantic input: worker count, placement, completion order, wall-clock time, run identifiers and transport branch names do not change $\mathrm{Plan}(\mathcal{I})$. A plan digest is computed over a canonical serialisation of semantic input and plan content, and transient execution identity is bound separately. The orchestrator derives a task graph from the plan, including preparation, tests, builds, transfers, publications and recording gates; a task is placed only on a compatible node and only when its exact prerequisites are available; the schedule need not be deterministic. Where a package's stages may be placed is execution policy declared beside Definition 47 and, like it, no part of the plan. Every reconciliation that writes a shared manifest or lockfile is an explicit task under the orchestrator's authority, serialised per shared file; each admitted result is one prepared input state, builds fan out only from an admitted state, and every build binds the state it consumed. An assignment identifies the run, plan digest, task, attempt, ownership generation, repository-qualified source object identifiers, effective configuration and command environment, toolchain and platform requirements, input receipts, permitted writes and outputs; workers authenticate the assigning orchestrator and the assignment's integrity; duplicate delivery is recognised; only one attempt is authorised to perform a given external side effect; and a task receipt is execution evidence, never a release record.

Proposition 67 (placement invariance)**.** *For a fixed input $\mathcal{I}$, the plan $\Pi(\sigma)$, the versions $\nu_\sigma(\cdot)$ and the publish order of Definition 44 do not depend on the placement of tasks, on the number of nodes, or on the order in which tasks complete.*

Proof. By Theorem 106 the plan, the versions and the planned publish order are functions of $(\sigma, h, \mathrm{config})$ alone. $\mathcal{I}$ fixes all three: σ is the set of release records compared under the locks (Definition 42), h is the fixed repository-qualified head, and the configuration is the run's resolved semantic configuration. Nothing a node contributes reaches those arguments: the node-startup settings of SPEC §28.2 (role, capacity, worker links) do not override package policy, planned versions, commands or inputs; the build readiness relation and the placement policy are execution policy outside the plan (Definition 47, Definition 66); and placement, node count and completion order are excluded from $\mathcal{I}$ by Definition 66. A function of fixed arguments takes one value. Vectors 3 and 5 of SPEC §28.8 are this statement. □

Definition 68 (Git synchronisation branches, SPEC §28.4)**.** The orchestrator provisions isolated task worktrees and temporary coordination branches named `dispat-worker-<id>-<workinfo>`, created atomically under `refs/heads/`, where `<id>` is the node the branch is addressed to and `<workinfo>` carries a date, a kind label and at least 128 bits of randomness. The name is an untrusted routing hint that lets a node list the branches addressed to it; the authenticated manifest, not the name, identifies node, run, task and attempt, and a node rejects a manifest whose binding disagrees with where it was found. Each acknowledged checkpoint identifies an exact full object identifier, and a consumer reads only from that object, never from a tip that can still move; a tip fetched but not read or verified has not been seen. Ref updates and deletions check the expected previous object identifier and attempt ownership, so that an update racing another party's update fails instead of overwriting it. Transport credentials do not authorise a worker to update native release branches, tags or fleet settlement refs. A source snapshot may travel without the history behind it, as a parentless commit or bare

tree, because a task's commands may not read history from a transport checkout. Transport commits are never release intent, a window boundary, a successful release or an admitted native head; they are neither merged into a release branch nor made ancestors of release tags, and only validated release-file changes enter native recording through the ordinary transaction. Artefact provenance binds the exact source revision and admitted release-file state used for the build.

Definition 69 (outputs as first-class inputs, SPEC §28.5)**.** A provider's revision does not make its ignored or generated outputs available on another machine, so a conforming implementation transports the outputs a local-output dependency needs, declared ignored and generated files included; it neither reclones and rebuilds the provider closure on every consumer node nor pins the connected graph to one worker. A successful build emits a manifest binding its outputs to the run, plan, task, attempt, repository-qualified source revision, admitted release-file state, package and version, toolchain and platform, and input and output digests. Identity is by content: consumers verify the authenticated manifest, content integrity and compatibility before their stage becomes ready; installation is atomic at the task-input boundary; a completed build publishes one admitted output set for its exact attempt; and a missing, stale, incomplete or incompatible output fails the prerequisite and is never substituted with registry contents or a new build under a different input. Bytes travel through the coordination branch or through immutable content-addressed bundles whose references and digests are in it; a reference alone is not a transfer. For package edge $B \to A$ under the `build` relation, readiness is the task precedence build(A) $\to$ export(A) $\to$ verify/materialise(A at B's node) $\to$ build(B); under `publish` it requires A's publication and every applicable recording gate, which local bytes never replace; under `none` the edge carries no output and the two builds are independent tasks. A provider a consumer reaches only across `none` edges is no input of that consumer and is not prepared for it: the input closure ends at a `none` provider. Task inputs include every required build dependency, including packages needing no new release and build-only edges outside the propagation kinds; the task graph is acyclic, and an unschedulable cycle is an execution-configuration error, never a reason to change release intent. Verified outputs may be reused across runs only under a new admission receipt bound to the current run, plan and task attempt.

Definition 70 (publication, recording and recovery, SPEC §28.6, SPEC §28.2)**.** The orchestrator alone admits results and authorises publication, after the ordinary test, dependency, settlement and relevant-input checks; it serialises publication and recording within one owner, a single-history monorepo included, while isolated builds within that owner may overlap and independent owners keep their publication concurrency. A worker never moves release branches, creates release tags, settles peer links or finalises the fleet. Authorisation to publish is explicit, single-use and bound to one task attempt: it is issued only after the package's `beforePublish` hook has completed on the executing node and after the relevant-input and lock-ownership checks, the publisher does not start its command before it has observed the authorisation for its own attempt, and an authorisation is never reused by another attempt or for another effect. An authorisation may be bounded in time, with a start instant after which the publisher does not begin and a deadline at which the executing node ends the command itself; the bound limits when an effect may begin and how long it runs, is not an expiry of the release lock, and authorises no takeover. The orchestrator retains and verifies lock ownership for the lifetime of all authorised effects, and assignments and results are bound to its ownership generation. After lock loss no new effect starts: no further assignment and no further authorisation. Recording an effect authorised before the loss is not a new effect and is still attempted, as the create-only record of Definition 43, because a publication left unrecorded is a wedge the next owner can clear only by Definition 55, and a create-only record cannot overwrite what a successor wrote. A timed-out worker is not proof that its publisher stopped: before owner-

ship is handed over, cancellation is acknowledged with the phase the attempt stopped in, or a fence is enforced where the effect occurs, or the outcome is reconciled; a generation field alone fences no registry command, and no takeover expires ownership while the previous process can still act. Capacity returns only on evidence: an in-flight attempt retains its reservation until completion, acknowledged cancellation or safe fencing, a timeout alone frees nothing, and an assignment no node has claimed holds no capacity and may be withdrawn and offered again. A worker that fails before publication is retried under a new attempt without discharging the package; a publication that may have succeeded is reconciled by Definition 55 before another attempt, since neither a missing reply nor a missing tag proves failure; only the durable source release record discharges the obligation. Locks are released only after authorised operations have quiesced or been fenced, in reverse acquisition order; if that cannot be proved the run fails holding its exclusion and no record of the uncertain package, which the next run treats as pending. An attempt never authorised to perform an external effect is fenced by revoking its coordination ref; an authorised publisher is not. An abandoned run is findable from its lock, and recovery revokes the run's coordination refs, settles every authorised attempt without a result, and only then removes the lock.

Example 71 (the required example, SPEC §28.7)**.** In one large JavaScript repository let `assets` $\Rightarrow$ {`ui`, `docs`} $\Rightarrow$ `app` denote task precedence: `ui` and `docs` consume `assets`, and `app` consumes both; the package edges of Definition 3 point the other way. Worker A builds `assets` once and exports its required outputs. Workers B and C receive verified outputs and build `ui` and `docs` concurrently. The assigned `app` worker receives both output sets and builds `app`. Publication and source recording then obey Definition 44 and the owner lane of Definition 70. Neither B nor C rebuilds `assets`, and the `app` worker rebuilds neither closure. The example works without converting the monorepo to a polyrepo or publishing private intermediate packages solely for transport.

Remark 72 (the performance boundary, SPEC §28.7)*.* The required gain is the distribution and reuse of dependent builds with parallel ready branches; universal speedup is not a semantic guarantee. Git provides source and result transport; it does not schedule tasks, transfer omitted build products, guarantee compatible environments or make effects exactly-once. A strictly serial chain retains its critical path, and transfer and setup overhead can outweigh saved compute on small jobs. Before an implementation claims a performance gain it compares the same pinned fixture locally and on several nodes, reporting cold and warm setup, per-node resources, transfer bytes and time, task execution counts, total work, wall time, repetitions and variability, and artefact equivalence, with the fixture and environment identified; skipped or failed trials are not speedups, and neither source-only dispatch nor duplicated prerequisite compilation is a distributed-build success.

Proposition 73 (list scheduling under placement, SPEC §13.11)**.** *Let W_k be the summed duration of a run's command tasks, L_k the duration of its longest precedence chain, and s the task slots usable at once, the lesser of the summed node capacities and the applicable run-wide stage limits. No schedule finishes before* $\max(L_k,\ W_k/s)$*. When slots are interchangeable and transfers cost nothing, a placement that never leaves a usable slot idle while a compatible task is ready finishes within $W_k/s + (1-1/s)\,L_k$, which is at most $(2-1/s)$ times the best possible. On a chain $L_k = W_k$ and no number of nodes helps.*

Proof. The lower bound is that of Proposition 62 with one stage. For the upper bound take the chain $t_q \to \cdots \to t_1$ of that proof. At every instant either a chain task runs or all s slots are busy, and in the latter case the work done off the chain is at most $W_k - L'$ where $L' \le L_k$ is the chain's length; so the off-chain time is at most $(W_k - L')/s$ and the makespan at most $L' + (W_k - L')/s = W_k/s + (1-1/s)L' \le W_k/s + (1-1/s)L_k$. Each of W_k/s and L_k is at most

the optimum, giving the factor. This is Graham's bound for list scheduling under precedence constraints [12, 13]. On a chain every task is on the longest path, so $L_k = W_k$ and the lower bound is W_k for every s. □

Neither premise holds in general: nodes differ in platform and speed, and an output transfer delays the consumer it feeds. SPEC §13.11 therefore states no bound once transfers count and puts the measurement of Remark 72 in its place. The inequality still says what to expect: once $s \geq W_k/L_k$ the lower bound is L_k, so the chain and not the pool is the limit, and more nodes can only close the gap between a greedy schedule and L_k, which is below a factor of two.

Implementation status. The design history of SPEC dated 2026-09-22 records that the dispat release engine implements the profile in every history mode over the Git mailbox transport of Definition 68: every message is a signed tree on a branch, every transition one compare-and-swap ref update, a worker polls, and nothing connects to a worker. Builds and publications run on workers, a publication is delegated only by an explicit placement, a space that logs in to its registry publishes on the orchestrator, the orchestrator is one more node of the pool chosen last, and a provider the run does not release is prepared once per run when a consumer reads its outputs. Outputs travel as result trees on the branches, with no bundle service, which is vector 17 of SPEC §28.8. Not implemented, so their conditional vectors do not apply: reuse of verified outputs across runs (vector 20) and the rollback profile of SPEC §26 (vector 26); the durable mailbox ref and the history-free snapshot that SPEC §28.4 permits are likewise unimplemented, so a snapshot descending from the planned head carries the repository's whole history to a mailbox that starts empty. Departures from the profile as written: the platform check of SPEC §28.2 runs before dispatch for the packages a run releases and, for a prepared provider, at its placement; and the `native-recording-or-lock` class is attached where a distributed run loses its lock and not yet at the older recording and lock sites. The same history records that the owed windows of SPEC §13.3 and `E201` (Definition 52) are not yet implemented, so a consumer that sits out the run in which its provider releases the owed commit is still stranded in that engine. What was measured, on which fixtures, is reported in Section 12.

5.3 Repository-local windows

SPEC §27 plans one package graph whose packages are owned by several linked Git repositories. It changes where history and release records are read and nothing else: the grammar, the bump lattice, the windows, holds, cancellation, corrections, channel rules, the dependency graph and the partial-publication rules of Section 4 are unchanged. This subsection defines the objects the profile adds and states what it promises; it does not extend the single-history theorems of Section 6 to several histories.

Definition 74 (fleet snapshot and commit identity, SPEC §27.2)**.** Before parsing history the engine resolves one immutable map from repository identity to full commit object identifier, the fixed fleet snapshot; every participating repository is an initialised, non-shallow checkout at its pinned commit, and a missing, shallow, moved or ambiguous participant is `E330`. A commit identity is a pair (repository, full object identifier); two equal identifier byte strings in different repositories are incomparable. Commit dates, author dates, modification times, tag creation dates and fetch order create no ancestry or precedence. The engine reads each repository's reachable DAG once for the snapshot, preserves native parent order, and manufactures no synthetic fleet DAG or parent edge. Ancestry, changed paths, direct pending

windows, cancellation barriers, corrections and `Reverts` are evaluated in the repository that owns the commit.

Definition 75 (repository-local windows, SPEC §27.5, SPEC §27.6, SPEC §27.11)**.** For a source repository r with fixed head h_r, write $\mathrm{reach}_r(x)$ for the ancestors-or-self of a commit x of r, and

$$A_r(x) \;=\; \mathrm{reach}_r(h_r) \setminus \mathrm{reach}_r(x), \qquad A_r(\bot) \;=\; \mathrm{reach}_r(h_r).$$

For a consumer package D and a repository r, let $s_{D,r}$ be the revision of r that D's last stable release incorporated and $b_{D,r}$ the revision of r that D's newest release of any channel incorporated, each $\bot$ where no such release exists. D's train and fresh windows over r are

$$W(D;\, r) \;=\; A_r(s_{D,r}), \qquad W^{\circ}(D;\, r) \;=\; A_r(s_{D,r}) \cap A_r(b_{D,r}).$$

When r owns D, the boundaries are the commits of D's own release tags, and the windows are those of Definition 9. When r does not own D, the boundaries are the positions D's releases prove in r, and only these prove them: a control release checkpoint whose release-commit message names D's exact tag, changes D's gitlink to the commit peeled from that tag, and pins the revision of each other repository the release incorporated (SPEC §27.6); an explicit `repositoryBaselines` tuple $(D, \text{release tag}, r, \text{revision})$, with stable and prerelease tags as separate entries (SPEC §27.6); or, in the linked peer topology, the route of fleet links recorded hop by hop from the tree of the release commit that names D's tag (SPEC §27.11). A matching object identifier alone proves nothing. Missing or ambiguous evidence is `E333`, never a guess and never a choice by date or proximity. Admission of a unit u carried by a commit of r at a consumer D remains $c_u \in W^{\circ}(D;\, r)$: D's last release had not yet incorporated the qualified revision. A source unit resolves directly only packages of its own repository and propagates through the one combined graph; delivery (SPEC §13.4a) compares D's boundary and the release commits of the provider in the provider's repository. A window over r is a function of r, h_r and one boundary revision of r and of nothing else, so packages of any owners share an immutable train window over r exactly when their stable boundary there is the same qualified revision, and a fresh window exactly when both boundaries are.

The identity $W^{\circ}(D;\, r) = A_r(s_{D,r}) \cap A_r(b_{D,r})$ is the fresh-window identity of Proposition 117, read per repository; it is what lets Section 9 key every window by one boundary revision. Newest-wins precedence between competing directives is defined within one repository's DAG and within the control history; source revisions of different repositories have no such order, and where the existing rule requires a single winner among incomparable revisions the engine fails with `E334` rather than choosing by date, traversal order, repository name or identifier (SPEC §27.4). An explicit control directive can resolve the choice, evaluated against the gitlinks its own commit pinned, its causal snapshot, so that it cannot rewrite work the control revision had not yet observed; in the linked peer topology no repository addresses the whole fleet and the conflict is refused outright.

Definition 76 (recording order, SPEC §27.7, SPEC §27.11)**.** The combined publish sequence is that of Definition 44 across all repositories, and publication remains non-atomic. Each successful package receives its immutable release tag in its owning repository at the ordinary tag-after-publish point of Definition 43. Where a source release commit and push are enabled, the source release is recorded durably first, and only after its branch and tags are reachable from the configured remote may a control gitlink or a peer's pin advance to it: a pushed pointer never names a commit nobody else can fetch. A control checkpoint, when enabled and non-empty, follows the source record and never precedes it; no empty commit is forced and no checkpoint is required. A failure to record or durably push a tag is `E335`; prior successes remain successful and dependents are protected by Definition 50. If the source record succeeds

and the control checkpoint then fails, `E335` names the repository, the full revision and the tag and requires explicit repair; the next run remains `E330` while the checkout and the committed gitlink are unpinned, and the engine neither auto-commits the repair, nor forces a checkpoint, nor republishes the source, nor infers success from the worktree. A successful source tag is never deleted, moved or duplicated to make a checkpoint look atomic, and no rollback is inferred. In the linked peer topology the route evidence is settled by ordinary commits before publication, deepest hop first, all or nothing per package (`E333` otherwise), and each settlement is an admitted transition of the repository it was written in. The fleet lock covers the control repository, every active source and every standalone package as one fleet (`E336` otherwise), is taken in one stable total order over lock identities, and is released in reverse.

Bounds. SPEC §27.9 requires the engine to index the control gitlink transitions once per fixed control snapshot, not once per consumer or provider pair; to store each parsed (repository, commit) record once; to share window membership only under the keys of Definition 75; and to bound its reachability and ancestry caches, because a cache of every queried pair can grow quadratically in one repository's history even when that history is walked once. With the notation of Section 9, input traversal is $O(G + \sum_q (H_q + A_q))$, window reachability is $O(\sum_q (H_q + A_q) \cdot \mathrm{bw}(m_q) + I_w)$ with one marker pass per repository, and in the linked peer topology boundary resolution is $O(Q + X \cdot Y)$ lookups over B tree reads, with settlement in at most $Q - 1$ commits and at most two in a star. Publication revalidation adds $O(\sum_q J_q(1 + T_q))$ comparison work, with J_q the fleet or package checks repository q takes part in and T_q its relevant tag refs, and retaining every package's input closure costs memory linear in the total number of repository memberships across those closures, at most $P \cdot Q$. The profile makes no globally linear end-to-end claim and permits no synthetic ancestry shortcut.

Remark 77 (what this subsection does not prove). Lemma 78, and every theorem of Section 6 that rests on it, is stated for one history with one h. Under Definition 75 a consumer has one window per repository its plan reads, each measured from a boundary its own release proves there, and a release of the consumer moves every one of those boundaries at once only when the evidence of Definition 76 is written. Restating the retry invariant over the fleet snapshot and proving the eight guarantees per repository is a separate piece of work; this report defines the objects, and SPEC §27 promises that the single-history rules apply unchanged inside each repository.

6 The eight guarantees

Throughout this section, h and the configuration are fixed and $\sigma_1 \le \sigma_2 \le \cdots$ is a run sequence (Definition 27). Four hypotheses are stated once and used by name.

(H1) stable inherited-channel state
every baseline channel read by `Propagate-Channel: inherit` remains unchanged through the sequence.

(H2) no suppressed source is released
for every unit u and every $p \in \mathrm{scope}(u)$ whose contribution to u is suppressed at σ_1, whether by cancellation, by a hold (Definition 19), or by a correction (Remark 17), p is not released at h in the sequence.

(R) owed sources reach their target
for every u and d with $\mathrm{adm}_\sigma(u, d)$, every $P \in \mathrm{owed}_\sigma(u, d)$ reaches d (Definition 5), and $\delta(\mathrm{owed}_\sigma(u, d), d) = \delta(\mathrm{src}_\sigma(u), d)$.

(K) propagation edges order and block
$K_{\text{prop}} \subseteq K_{\text{order}}$ and $K_{\text{prop}} \subseteq K_{\text{block}}$.

The *retry invariant* of SPEC §13.7c fixes, for every outstanding unit, the corrected tuple stream, source sets, graph, scope and depth predicates, target baselines, and resolved source/target channel admission. The only permitted state changes are baseline tags written for packages that successfully release. Thus it entails the constancies used below; (H1) and (H2) name two historically problematic components, not a weaker substitute for the full invariant. One consequence the invariant states in its own terms is that $\mathrm{dlv}(,,)$ only grows (**G5**); Lemma 80 derives it.

(H1) and (H2) are explicit parts of the invariant. They are named because the 1.0.0 proof audit isolated them in counterexamples (proof obligations 139 and 142); they are not sufficient on their own. The fresh window of CCME 2.0.0 resolves the prerelease-discharge defect of proof obligation 138, while retries still require every other input listed above to remain fixed. Every theorem below states when it needs that full invariant.

(R) and (K) are not in SPEC. (R) holds automatically for a unit with a single source, since an admitted singleton is owed and reaches d by Item (A2), and for a target that has not released past c_u, where the owed set is the whole source set (Lemma 20) so that only the reaching clause is at issue. It can fail for a multi-source unit whose delivered source reaches the target and whose owed source does not, and Remark 112 shows what that costs. (K) holds for the default K_{order} and fails for the default K_{block}, which excludes `optionalDependencies` (Definition 3); Remark 100 shows what that costs. Both are flagged for the specification.

6.1 Lemmas

Lemma 78 (window antitonicity)**.** *Let* $\sigma' = \mathrm{run}(\sigma, F)$ *with* $\mathrm{wf}(\sigma)$. *For every* $p \in \mathcal{P}$*:*

$$W_{\sigma'}(p) = \begin{cases} \emptyset & \text{if } p \text{ published a non-prerelease version in the run,} \\ W_\sigma(p) & \text{otherwise,} \end{cases}$$

and

$$T_{\sigma'}(p) = \begin{cases} T_\sigma(p) \cup \{h\} & \text{if } p \in S(\sigma, F), \\ T_\sigma(p) & \text{otherwise.} \end{cases}$$

In particular $W_{\sigma'}(p) \subseteq W_\sigma(p)$*, and along a run sequence* $W_{\sigma_j}(p) \subseteq W_{\sigma_k}(p)$ *for* $j \geq k$*. For the undelivered window, every successful release of* p*, stable or prerelease, gives* $W^\circ_{\sigma'}(p) = \emptyset$*; otherwise it is unchanged.*

Proof. By Definition 25 the run adds only tags of the form $(p, \nu_\sigma(p), h)$ for $p \in S(\sigma, F)$, which is the claim about $T()$. If p is not among them, its tag set is unchanged, hence so are $\mathrm{scom}_\sigma(p)$ and its window. Suppose p is among them and $\nu_\sigma(p)$ is not a prerelease. Then $\nu_\sigma(p)$ exceeds $\mathrm{base}_\sigma(p)$ in SemVer precedence, and therefore exceeds $\mathrm{sbase}_\sigma(p)$: SPEC §13.9 computes it as $\mathrm{apply}(\mathrm{sbase}_\sigma(p), b)$ for the graduation and stable cases, and the guards `E185`, `E195`, `E153` and `E156` reject any computed or pinned version that fails to exceed the baseline. So the new tag is the highest-precedence non-prerelease tag of p, giving $\mathrm{scom}_{\sigma'}(p) = h$ and $W_{\sigma'}(p) = \mathrm{reach}(h) \setminus \mathrm{reach}(h) = \emptyset$. If $\nu_\sigma(p)$ is a prerelease, it is not a candidate for sbase() at all, so scom() and the train window are unchanged. Its tag is nevertheless the new baseline at h, so $W^\circ()$ is empty. The final claim follows by induction along the sequence. □

Remark 79. The third case of the proof, a prerelease release leaving the window untouched, is not an oversight in Definition 9 but its stated purpose (Remark 15). Admission neverthe-

less uses the empty undelivered window. Proof obligation 138 records the 1.0.0 defect that motivated this separation.

Lemma 80 (delivery is monotone)**.** *Let* $\sigma' = \mathrm{run}(\sigma, F)$ *with* $\mathrm{wf}(\sigma)$*. For all packages* P, D *and commits* $C \preceq h$*:*

(a) $\mathrm{dlv}_\sigma(P, C, D)$ *implies* $\mathrm{dlv}_{\sigma'}(P, C, D)$*;*
(b) if $h \notin T_{\sigma'}(D)$ *then* $\mathrm{dlv}_{\sigma'}(P, C, D)$ *iff* $\mathrm{dlv}_\sigma(P, C, D)$*;*
(c) if $h \in T_{\sigma'}(D)$ *then* $\mathrm{dlv}_{\sigma'}(P, C, D)$ *iff some* $t \in T_{\sigma'}(P)$ *has* $C \preceq t$*: iff* P *has released* C *at all.*

Consequently, under the retry invariant, $\mathrm{owed}_{\sigma'}(u, D) \subseteq \mathrm{owed}_\sigma(u, D)$ *for every unit* u*, with equality whenever* $h \notin T_{\sigma'}(D)$*; and along a run sequence* $\mathrm{dlv}_{\sigma_j}(P, C, D)$ *is non-decreasing in* j *and* $\mathrm{owed}_{\sigma_j}(u, D)$ *is non-increasing.*

Proof. (a): $T_\sigma(P) \subseteq T_{\sigma'}(P)$, and $\mathrm{basec}_{\sigma'}(D)$ is either $\mathrm{basec}_\sigma(D)$ or h, which reaches $\mathrm{basec}_\sigma(D)$; so a witness t for σ is one for σ'. (b): D gained no tag, so $\mathrm{basec}_{\sigma'}(D) = \mathrm{basec}_\sigma(D) =: b$ with $b \neq h$, and every tag the run added is at h; $h \preceq b$ would put h in $\mathrm{reach}(b)$, so $b \preceq h$ and acyclicity forbid it. Hence no added tag is a witness and the witness set is unchanged. (If D has no baseline both sides are false.) (c): $\mathrm{basec}_{\sigma'}(D) = h$ (Definition 8) and every release commit is an ancestor-or-self of h, so the second conjunct of Definition 10 is vacuous. The consequences restate (a) and (b) over $\mathrm{src}(u)$, which the retry invariant fixes, and induction along the sequence gives the last claim. □

Lemma 81 (outstanding-input stability)**.** *Assume the retry invariant. For every outstanding unit* u *and unreleased target* d*, its corrected tuple, source set, graph and scope/depth predicates, target baseline, resolved source and target channels, and channel-eligibility decision are the same in every retry.*

Proof. This is the retry invariant specialised to one unit and target. H1 prevents an inherited baseline channel from drifting; H2 prevents a successful source release from changing discharge and hence the source set. The remaining constancies are separate clauses of the invariant and cannot be derived from H1 and H2 alone. □

Lemma 82 (admission restriction)**.** *Under the retry invariant, admission of an outstanding unit* u *at a target* d *can change only through Item (A4), that is through* $\mathrm{owed}_\sigma(u, d)$*, which is non-increasing along the sequence. It becomes empty exactly when* d *is tagged at* h *and every source of* u *has released* c_u*.*

Proof. All other clauses of Definition 37 are fixed by the retry invariant (Lemma 81). The monotonicity is Lemma 80; the characterisation of emptiness is Lemma 80(b) (an untagged d keeps its owed set) and (c). □

Remark 83 (why H1 is part of the invariant). Without H1, the origin channel of an `inherit` unit can change when a source baseline moves from a train to stable. That can change an unreleased dependent's resolved channel and planned version between retries. Proof obligation 142 preserves the 1.0.0 counterexample; SPEC prevents it by fixing inherited baseline-channel state in the retry invariant.

Lemma 84 (a planned package tagged at h)**.** *Let* $h \in T_\sigma(D)$ *and* $D \in \Pi(\sigma)$*. Then* $\mathrm{dir}_\sigma(D) =$ none, $\mathrm{chan}_\sigma(D) = \mathrm{ch}(\mathrm{base}_\sigma(D))$*,* D *has no applicable exact* `Release-As`*, and some unit* u *has* $\mathrm{adm}_\sigma(u, D)$*. For every such* u *and every* $P \in \mathrm{owed}_\sigma(u, D)$*:*

(i) no release of P *reaches* c_u*; in particular* $h \notin T_\sigma(P)$*;*

(ii) $c_u \in W^\circ_\sigma(P)$*;*
(iii) $\neg\mathrm{canc}(c_u, P)$, $\neg\mathrm{held}(P)$, $\mathrm{dir}_\sigma(P) \geq \mathrm{bump}(u) \neq \mathsf{none}$*, hence* $P \in \Pi(\sigma)$*;*
(iv) if $\mathrm{wf}(\sigma)$*,* P *precedes* D *in the publish order.*

Proof. $\mathrm{basec}_\sigma(D) = h$ gives $W^\circ_\sigma(D) = W_\sigma(D) \setminus \mathrm{reach}(h) = \emptyset$. Definition 38, both candidate sets of Definition 34 and the exact-version disjunct of Definition 21 all require a commit in $W^\circ_\sigma(D)$, so the first three claims hold and $D \in \Pi(\sigma)$ forces $\mathrm{prp}_\sigma(D) \neq \mathsf{none}$: some unit is admitted. (i): by Lemma 80(c), P owed means no $t \in T_\sigma(P)$ has $c_u \preceq t$; h reaches c_u. (ii): $\mathrm{scom}_\sigma(P)$ and $\mathrm{basec}_\sigma(P)$ are release commits of P when defined, so neither reaches c_u, and $c_u \preceq h$; by Definition 9 $c_u \in W^\circ_\sigma(P)$. (iii): by (ii) $\neg\mathrm{disch}_\sigma(P, c_u)$, so $P \in \mathrm{src}_\sigma(u)$ (Definition 19) forces $\neg(\mathrm{canc}(c_u, P) \vee \mathrm{held}(P))$. Then $P \in \mathrm{src}_\sigma(u) \cap \mathrm{scope}(u)$ and $c_u \in W^\circ_\sigma(P)$ put $\mathrm{bump}(u)$ into the join of Definition 38, and $\mathrm{bump}(u) \neq \mathsf{none}$ by Item (A1); an unheld package with a non-none bump is in the plan. (iv): $P \in \Pi(\sigma)$, $\mathrm{basec}_\sigma(D) = h$ and $\mathrm{owes}_\sigma(P, D)$; Definition 24 orders P before D. □

The lemma is the formal content of SPEC §19.3's "a dependent with no cause of its own": a package already tagged at h can be in the plan for one reason only, an owed provider that is itself in the plan, untagged, and ordered before it.

6.2 G1: termination

Theorem 85 (**G1**, termination)**.** *Every traversal of Definition 28 halts after at most* $|\mathcal{P}|$ *insertions into* `seen` *and* $O(|\mathcal{P}| + |E|)$ *work, regardless of the declared depth, including* ∞ *(`+*`), and regardless of whether the graph is acyclic. Propagation performs at most* $2|\mathcal{U}|$ *such traversals, so it halts, in* $O(|\mathcal{U}| \cdot (|\mathcal{P}| + |E|))$ *traversal work.*

Proof. A package is inserted into `seen` only after a test that it is absent, and `seen` is never cleared, so there are at most $|\mathcal{P}|$ insertions. Only inserted packages (and the initial sources, inserted at seeding) ever enter `next` and hence `frontier`, so each package is expanded at most once, and the edges scanned across the whole traversal are therefore at most $\sum_p |\mathrm{dep}(p)| = |E|$. The loop terminates either by exhausting ℓ iterations or, when $\ell = \infty$, by the empty-frontier break, which must occur within $|\mathcal{P}|$ iterations since each non-empty frontier consumes at least one distinct unvisited package. No step of this argument uses acyclicity: a cycle merely means some edge leads to a package already in `seen`, and that edge is skipped. The outer loops of SPEC §9.2 run over the finite set $\mathcal{U}$ once per axis; the per-target work of phase 3 (the owed set and the resolvability test) is finite and adds no traversal. □

Remark 86. Acyclicity (Definition 3) is therefore not what makes propagation terminate; `seen` is. Acyclicity is needed for the publish order (Proposition 45), not here. This matters because it means an implementation that loosens the `E200` check does not silently gain a non-terminating traversal; it gains an unorderable publish phase, which is a different and louder failure.

6.3 G2: completeness

Theorem 87 (**G2**, completeness: no orphans)**.** *Assume the retry invariant. If* $\mathrm{adm}_{\sigma_1}(u, D)$*, then for every* j *either* $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ *or* $\mathrm{adm}_{\sigma_j}(u, D)$*, and in the latter case* $\mathrm{prp}_{\sigma_j}(D) \geq \mathrm{pb}(u)$*. Moreover* $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ *holds only if* D *is tagged at* h *in* σ_j *and every* $P \in \mathrm{src}_\sigma(u)$ *has a release reaching* c_u*. A release by a source alone does not discharge the contribution, and neither does a release by* D *at a commit reaching no release of some source carrying* c_u*.*

Proof. Take the conjuncts of Definition 37 in turn. Item (A1) depends only on the unit. Item (A3) and Item (A5) depend only on h and the configuration. Item (A2) is $D \in R(\mathrm{src}_{\sigma_j}(u), \mathrm{pd}(u))$, and by the retry invariant the source set is unchanged, as are the graph and the depth, so the traversal is literally the same computation. Item (A6) is constant by Lemma 81. Item (A4) is the one clause that varies, and it is exactly the disjunction in the statement. So whenever $\mathrm{owed}_{\sigma_j}(u, D) \neq \emptyset$ every conjunct holds, and $\mathrm{prp}_{\sigma_j}(D)$, a join over the admitted units (Definition 38) with u among them, is at least $\mathrm{pb}(u)$.

For the "moreover": $\mathrm{owed}_{\sigma_1}(u, D) \neq \emptyset$ by Item (A4) at σ_1, so some $P \in \mathrm{src}_\sigma(u)$ has gone from undelivered to delivered between σ_1 and σ_j. If $h \notin T_{\sigma_j}(D)$, Lemma 80(b) applied along the sequence says nothing changed; so D is tagged at h in σ_j, and then Lemma 80(c) makes $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ equivalent to every source of u having a release reaching c_u. The two final sentences are the contrapositive read against the same lemma: a source's release leaves $h \notin T(D)$ untouched, and a release of D at h with some source still lacking a release reaching c_u leaves that source owed. □

The theorem is the formal content of "there are no orphans", and it now covers both shapes of SPEC §13.7a. A provider that publishes while a consumer's publish fails leaves the consumer's owed set intact (Lemma 80(b)), so the consumer is planned again until it releases. A consumer that proceeds past the commit on a cause of its own while its provider fails is tagged at h with the provider unreleased; by Lemma 80(c) the provider is still owed, and the consumer is planned again, ordered after the provider (Lemma 84), until both have released. Note where the proof would break under the readings of Item (A4) that Example 39 contrasts. Against the source's window, the provider's successful release would empty *that* window, the unit would cease to be a propagation source, and the waiting consumer would never be released. Against the target's window, the proceeding consumer's own release would empty *that* window, and the consumer that got ahead would never be planned again. One membership test, evaluated against the right relation, is the whole difference between a resumable release and a silently lossy one.

Remark 88 (the discharge that is not a delivery). The "moreover" clause is satisfied, and the contribution leaves the plan, in one situation where nothing was delivered: D already tagged at h when the sequence starts, its owed provider P publishing at h in a run in which D then fails. Proposition 26 describes it; the run reports `E201` and the theorem does not claim otherwise. Where D is not tagged at h in σ_1 the situation cannot arise, since then $h \notin T_{\sigma_j}(D)$ until D itself releases in the sequence.

6.4 G3: version stability

Theorem 89 (**G3**, version stability)**.** *Assume the retry invariant. If $d \in \Pi(\sigma_1)$, d is not released in runs $1..k-1$, and no run $i < k$ reports `E201` for d (Definition 25), then $d \in \Pi(\sigma_k)$ and $\nu_{\sigma_k}(d) = \nu_{\sigma_1}(d)$. The third hypothesis is vacuous when $h \notin T_{\sigma_1}(d)$.*

Proof. First, $\mathrm{owed}_{\sigma_k}(u, d) = \mathrm{owed}_{\sigma_1}(u, d)$ for every unit u. If $h \notin T_{\sigma_1}(d)$ then, d having gained no tag, $h \notin T_{\sigma_k}(d)$ and Lemma 80(b) gives equality run by run; this is also why the third hypothesis is then vacuous, since reporting `E201` for d requires $h \in T_{\sigma_i}(d)$. If $h \in T_{\sigma_1}(d)$, then $\mathrm{basec}_{\sigma_i}(d) = h$ throughout, and by Lemma 80(c) the owed set can shrink only when some $P \in \mathrm{owed}_{\sigma_i}(u, d)$ receives a tag at h in run $i < k$, that is $P \in S(\sigma_i, F_i)$. Since $\mathrm{owed}_{\sigma_i}(u, d) \neq \emptyset$ and the other clauses are constant, $\mathrm{adm}_{\sigma_i}(u, d)$, so $\mathrm{owes}_{\sigma_i}(P, d)$. Run i published something, so it was not refused and $\mathrm{wf}(\sigma_i)$ holds (Definition 25); hence $d \in \Pi(\sigma_i)$ after P. d was not released in run i, and it was not blocked, because $P \notin X_d$ gives it a cause of its own (Definition 23); so $d \in F_i$, and run i reports `E201` for d, which the hypothesis excludes.

Run	State σ	Plan $\Pi(\sigma)$	Published	Empty?
1	σ_0 (Example 4)	`core` 1.5.0, `cli` 2.0.1, `ui` 0.9.2, `api` 1.2.1	`core`, `ui`, `api`; `cli` fails	no
2	$\sigma_1 = \sigma_0 \cup 3$ tags	`cli` 2.0.1	nothing; `cli` fails again	no
3	$\sigma_2 = \sigma_1$	`cli` 2.0.1	`cli`	no
4	$\sigma_3 = \sigma_2 \cup$ {`cli@2.0.1`}	$\emptyset$	(none)	yes

Table 1: The waiting consumer of Example 91. Theorem 87: `cli` is in every plan until it publishes, its owed set {`core`} unchanged by Lemma 80. Theorem 89: it is at `2.0.1` in all of them. Theorem 92: `core`, `ui` and `api` leave after run 1. Theorem 94: nothing new ever enters. Theorem 98: the plan shrinks to empty. Run 2 publishes nothing, which Theorem 98 treats as the hard-error case rather than the partial-failure one.

Now d gained no tag, so $\text{base}_{\sigma_k}(d) = \text{base}_{\sigma_1}(d)$ and $\text{sbase}_{\sigma_k}(d) = \text{sbase}_{\sigma_1}(d)$, and by Lemma 78 its windows are unchanged. Then $\text{dir}_{\sigma_k}(d) = \text{dir}_{\sigma_1}(d)$, because Definition 38 reads the window, $\text{scope}(u)$, and $\text{src}_\sigma(u)$, the last constant by the retry invariant. And $\text{prp}_{\sigma_k}(d) = \text{prp}_{\sigma_1}(d)$: every clause of Definition 37 is constant for every unit, Item (A4) by the first paragraph and the rest by Lemma 81. Hence $\text{eff}_{\sigma_k}(d) = \text{eff}_{\sigma_1}(d)$. Its resolved and baseline channels, changed-provider status, and applicable exact `Release-As` cause are also fixed by the retry invariant. Thus whichever disjunct of Definition 21 put d in the initial plan still holds. For a fixed version function, SPEC §13.9 reads unchanged stable and current baselines, resolved channel, applicable exact version, and corrected train records. Those inputs are fixed for an unreleased target by the retry invariant, so its result is unchanged. This proves replay stability. The graduation core itself follows the specific train-target rule of SPEC §11.5; proof obligation 133 records the inconsistent SPEC §13.9 summary rather than treating that intent as absent. □

Remark 90 (the prerelease case survives, for a different reason). For a target d planned on a prerelease channel, the version is core(·)-plus-counter per SPEC §11.4, and the counter is read from $\text{base}_\sigma(d)$, which is unchanged precisely because d was *not* released. This preserves the counter, but extending Theorem 89 to prerelease targets still requires invariant resolved channels and admitted contributions, including H1's channel-stability component. The prerelease problem of proof obligation 138 is about packages that *were* released, and is a convergence failure, not a stability failure.

Theorem 89 is as much an operational property as a formal one: after a partial failure, the re-run publishes the same numbers the operator already reviewed. It is what makes a resumed release a continuation rather than a fresh decision. It says nothing about a package that *was* released: a consumer that proceeded is planned again at a patch on its new baseline (`cli@2.1.1` in Example 39), which is the version its catch-up is owed, not the one it was first planned at.

Example 91 (four theorems in two tables)**.** Table 1 continues the first shape of Example 39 through an adversarial schedule: run 1 loses `cli`, run 2 loses it again, run 3 succeeds. Table 2 runs the second shape, where `cli` proceeds. Each column is a theorem. `docs-site` and `@acme/theme` never appear, at any distance 2, in any run: the unit said `^`, and Theorem 94 forbids a later run from reconsidering that.

Run	State σ	Plan $\Pi(\sigma)$	Published	Progress?
1	σ_0, C carrying u and u'	`core` 1.5.0, `cli` 2.1.0, `ui` 0.9.2, `api` 1.2.1	`cli` proceeds; `core` fails; `ui`, `api` blocked	yes
2	$\sigma_1 = \sigma_0 \cup \{$`cli@2.1.0`$\}$	`core` 1.5.0, `cli` 2.1.1, `ui` 0.9.2, `api` 1.2.1	all four	yes
3	$\sigma_2 = \sigma_1 \cup 4$ tags	$\emptyset$	(none)	

Table 2: The proceeding consumer of Example 91, SPEC vector 80d. Theorem 87: `cli` stays admitted after its own release because `core` has not delivered C to it. Theorem 89: `ui` and `api` keep their numbers. Theorem 92: after run 2 nothing is owed. Theorem 98: run 1 published a package and the plan did not shrink; what it did was tag `cli` at h for the first time, which is the measure the proof uses.

6.5 G4: no double release

Theorem 92 (**G4**, no double ledger delivery)**.** *Assume the retry invariant. If D is released in run k and, after run k, every $P \in \mathrm{src}_\sigma(u)$ has a release reaching c_u, then $\mathrm{adm}_{\sigma_j}(u, D)$ is false for every $j > k$.*

Proof. $h \in T_{\sigma_{k+1}}(D)$, so by Lemma 80(c) every source of u has delivered c_u to D and $\mathrm{owed}_{\sigma_{k+1}}(u, D) = \emptyset$. The owed set is non-increasing (Lemma 80), so it stays empty and Item (A4) fails at every later σ_j. The full train window may retain c_u for aggregate prerelease version arithmetic; it cannot re-admit the contribution. □

A D that proceeded past c_u before a source released it does not meet the hypothesis: that source still lacks a release reaching c_u, D is admitted again (Lemma 80(c)), and it is bumped once more when the release comes. That later release is the delivery, not a second one. The sufficient condition of CCME 2.0.0, "released at a commit whose ancestry contains c_u", is no longer sufficient on its own, and Example 39 shows why it must not be.

Corollary 93 (exactly-once ledger discharge)**.** *Under the retry invariant, a contribution admitted to D remains admitted until D is tagged at h with every source of u having released c_u (Theorem 87), and is never re-admitted after that (Theorem 92). If the sequence delivers it, it is therefore discharged in the tag ledger exactly once. D may be published more than once meanwhile, a consumer that proceeded being the case, but the discharging release is the last of those. The single exception is the reported loss of Proposition 26, where the ledger records a discharge that no release of D performed.*

This is the idempotent-consumer property of the saga, obtained without a deduplication table: the git tags are the record, and delivery is the query against them. This is not an exactly-once guarantee for external side effects. A publish can succeed before its tag is recorded; recovery across that gap requires the verified adoption of SPEC §19.4 or an idempotent publisher. The ledger proof alone cannot prevent a repeated external operation.

6.6 G5: no blast-radius widening

Theorem 94 (**G5**, no blast-radius widening)**.** *Assume the retry invariant. For every unit u and every $j \geq 1$,*

$$\{\, d : \mathrm{adm}_{\sigma_j}(u, d) \,\} \;\subseteq\; \{\, d : \mathrm{adm}_{\sigma_1}(u, d) \,\}.$$

Proof. Compare the conjuncts of Definition 37, as in Theorem 87, but now in the direction of restriction. Items (A1), (A3) and (A5) are constant. Item (A2) is constant by the retry invariant: the traversal has the same source set, the same depth and the same graph, all fixed at h. Item (A6) is constant under the retry invariant. Item (A4), $\mathrm{owed}_{\sigma_j}(u,d) \neq \emptyset$, is antitone in j by Lemma 80: $\mathrm{dlv}(,,)$ only grows. A conjunction of constant and antitone predicates is antitone. □

Remark 95 (why this is a security property). SPEC §18.1 depends on Theorem 94. If the admitted set could grow between runs, an attacker able to induce a publish failure (by exhausting a registry quota, say) could enlarge the set of packages a commit releases *after* that commit had been reviewed and merged. The theorem says the achievable effect of inducing a failure is confined to delaying a subset of what was already going to be released. Delivery preserves this: a target it admits that the window test would not is one that released past c_u on its own account while within the unit's reach, and the unit's traversal, scope and depth are what they were, so nothing outside the unit's declared blast radius is reached.

Lemma 96 (the plan is non-increasing)**.** *Assume the retry invariant. Then* $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$ *for every* j*, hence* $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$*.*

Proof. Take $p \in \Pi(\sigma_{j+1})$; $\mathrm{held}(p)$ is constant, so p is not held at σ_j either. If p was released in run j, then $W^{\circ}_{\sigma_{j+1}}(p) = \emptyset$ (Lemma 78) and by Lemma 84 its only cause is a unit u with $\mathrm{adm}_{\sigma_{j+1}}(u,p)$; by Theorem 94 $\mathrm{adm}_{\sigma_j}(u,p)$, so $\mathrm{prp}_{\sigma_j}(p) \neq \mathsf{none}$ and $p \in \Pi(\sigma_j)$. If p was not released, its windows and tags are unchanged, so $\mathrm{dir}_{\sigma}(p)$ and the exact-version disjunct are unchanged, $\mathrm{prp}_{\sigma}(p)$ can only have shrunk (Theorem 94 applied to each unit), and $\mathrm{chan}_{\sigma}(p)$ and $\mathrm{ch}(\mathrm{base}_{\sigma}(p))$ are unchanged under the retry invariant; so every disjunct of Definition 21 that holds at σ_{j+1} held at σ_j. □

What the lemma no longer gives is strict decrease. A package released in run j can be in $\Pi(\sigma_{j+1})$: it proceeded past a commit its provider has not delivered. Theorem 98 therefore counts something else.

Lemma 97 (well-formedness is preserved)**.** *Assume the retry invariant, (R), and* $K_{\mathrm{prop}} \subseteq K_{\mathrm{order}}$*. If* $\mathrm{wf}(\sigma_j)$ *then* $\mathrm{wf}(\sigma_{j+1})$*. Hence every state of a run sequence is well-formed and no run of it is refused.*

Proof. Let $P \in \Pi(\sigma_{j+1})$, $\mathrm{basec}_{\sigma_{j+1}}(D) = h$ and $\mathrm{owes}_{\sigma_{j+1}}(P,D)$ through a unit u. By Theorem 94 $\mathrm{adm}_{\sigma_j}(u,D)$, and by Lemma 80 $P \in \mathrm{owed}_{\sigma_j}(u,D)$, so $P \in \Pi(\sigma_j)$ (Lemma 96) and $\mathrm{owes}_{\sigma_j}(P,D)$. Since $\mathrm{owed}_{\sigma_{j+1}}(u,D) \neq \emptyset$ and $\mathrm{held}(D)$ is constant, $D \in \Pi(\sigma_{j+1})$ once $D \in \Pi(\sigma_j)$ or $\mathrm{prp}_{\sigma_{j+1}}(D) \neq \mathsf{none}$; the latter holds outright. For the order: by (R) P reaches D over K_{prop}, hence over K_{order}, and the topological order of Definition 44 places P before D. The second sentence is induction from $\mathrm{wf}(\sigma_1)$ (Definition 27). □

6.7 G6: convergence

Say that run j *makes progress* when it publishes a package p with $h \notin T_{\sigma_j}(p)$: a package tagged at h for the first time.

Theorem 98 (**G6**, convergence)**.** *Assume the retry invariant, and suppose every run whose plan is non-empty makes progress. Let* $n = |\Pi(\sigma_1)|$*. Then* $\Pi(\sigma_{n+1}) = \emptyset$*.*

A refused run (Definition 25) publishes nothing, so the hypothesis entails that no run with a non-empty plan is refused; Lemma 97 says when that is automatic.

Proof. First, $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$, so $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$ for every j (Lemma 96).

Let $\mu_j = |\{\, p \in \Pi(\sigma_1) : h \notin T_{\sigma_j}(p) \,\}|$, the planned packages not yet tagged at h. Tags only accumulate, so μ is non-increasing, and $\mu_1 \leq n$. If run j makes progress, the package it tags for the first time lies in $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$, so $\mu_{j+1} < \mu_j$. And if $\Pi(\sigma_j) \neq \emptyset$ then $\mu_j \geq 1$: take $p \in \Pi(\sigma_j)$; either $h \notin T_{\sigma_j}(p)$, or by Lemma 84 there is $P \in \Pi(\sigma_j) \subseteq \Pi(\sigma_1)$ with $h \notin T_{\sigma_j}(P)$. So while the plan is non-empty each run strictly decreases a quantity that is at least 1 and at most n; the plan is therefore empty by run $n+1$, and once empty it stays empty by the first paragraph. □

The old measure, $|\Pi(\sigma_j)|$, fails here: in Table 2 run 1 publishes `cli` and $|\Pi(\sigma_2)| = |\Pi(\sigma_1)|$. The bound n survives because every package is tagged at h for the first time at most once, and a plan cannot be non-empty without containing such a package. The bound is restricted to the invariant domain. Publishing a suppressed source or changing channel admission may expose a finite follow-up obligation; SPEC §19.6 requires re-planning and renewed review, not an unconditional global-convergence claim. Held packages remain excluded.

Proposition 99 (when publishing anything is progress)**.** *Assume the retry invariant, (R) and $K_{\text{prop}} \subseteq K_{\text{block}}$. Then every well-formed run that publishes at least one package makes progress. Under these hypotheses Theorem 98 therefore holds with its hypothesis weakened to "every run whose plan is non-empty publishes at least one package".*

Proof. Let run j publish a non-empty set, and let p be the first published package in the publish order. Suppose $h \in T_{\sigma_j}(p)$. By Lemma 84, p's only causes are units u with $\text{adm}_{\sigma_j}(u, p)$, and every $P \in \text{owed}_{\sigma_j}(u, p)$ is in $\Pi(\sigma_j)$, untagged at h, with $\text{dir}_{\sigma_j}(P) \neq \mathsf{none}$, and precedes p in the order. No such P was published, since p is the first; none was blocked, since a direct bump is a cause of its own; so every such P failed and lies in X_p. Then $\text{owed}_{\sigma_j}(u, p) \subseteq X_p$ for every admitted u, and with the first three disjuncts of Definition 23 absent, $\neg\text{own}_{\sigma_j}(p, X_p)$. By (R) some owed P reaches p over K_{prop}, hence over K_{block}, so p transitively depends on a member of X_p and is blocked: a contradiction with its being published. Hence $h \notin T_{\sigma_j}(p)$ and the run makes progress. □

Remark 100 (a non-blocking propagation edge). Under the default configuration K_{prop} contains `optionalDependencies` and K_{block} does not, so (K) fails and Proposition 99 does not apply. The weaker hypothesis then does not suffice. Let p optionally depend on P, let one commit carry `feat(P)^`, and let P fail on every run. In run 1, p is not blocked, since the edge is not in K_{block}, and it has no cause of its own; it proceeds anyway, is reconciled to P's old baseline, and is tagged at h. In run 2 it is owed P's release (Lemma 80(c)), is planned after P, and, P failing again, is published again with nothing new. Every run publishes a package; no run makes progress; the plan never empties. The count n of Theorem 98 bounds the runs that make progress, not the runs that publish, and a spec-level reading of "successful-progress run" must say so. The republished p carries nothing its predecessor lacked, which is a second cost of the same configuration. Both are flagged for the specification. The reference implementation does not exhibit the scenario: it blocks over every declared dependency kind and treats an update from a failed provider as no cause (proof obligation 134), so the caught-up consumer of vector 80d is planned again each run and blocked with `W194` until its provider publishes, each such run exiting non-zero; the remark concerns the specification's default kinds, not the engine.

Proposition 101 (universal tagging is necessary, SPEC §13.10a)**.** *Assume the retry invariant. Suppose the protocol were weakened so that a released package p whose publish target uploads no artefact is versioned and has its manifest written, but is not tagged. Then if $\text{eff}_{\sigma_1}(p) \neq \mathsf{none}$, $p \in \Pi(\sigma_j)$ for every j, and Theorem 98 fails.*

Proof. p gains no tag in any run, so $h \notin T_{\sigma_j}(p)$ for all j; by Lemma 78 $W^{\circ}_{\sigma_j}(p) = W^{\circ}_{\sigma_1}(p)$, and by Lemma 80(b) $\mathrm{owed}_{\sigma_j}(u,p) = \mathrm{owed}_{\sigma_1}(u,p)$ for every unit. Hence $\mathrm{dir}_{\sigma_j}(p) = \mathrm{dir}_{\sigma_1}(p)$ and, every clause of Definition 37 being constant, $\mathrm{prp}_{\sigma_j}(p) = \mathrm{prp}_{\sigma_1}(p)$. The plan's first disjunct holds at every σ_j, so $\mu_j \geq 1$ for all j in the proof of Theorem 98 and no run empties the plan. □

This is why SPEC §13.10a tags whatever is released, including private, internal and artefact-less packages, and why the convergence check of SPEC §19.6 (`E199`) admits exactly one exclusion, held packages. A permanent exception to convergence would not be a caveat to document; it would disable the one check that detects the orphan failure of SPEC §13.7a, and disable it precisely for the internal-only packages where nobody is watching a registry.

6.8 G7: channel discharge

Theorem 102 (**G7**, channel discharge)**.** *Suppose d is released in run k on channel ch, so that* $\mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$ *for $j > k$. Then for every unit u and every $j > k$,* $\mathrm{pcf}_{\sigma_j}(u,d) \neq ch$*: the channel d has arrived on is never proposed for it again.*

Proof. Write $cur = \mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$; this is read from d's tag, not from anything computed in the run. Every branch of Definition 32 ends in the test "if $target = cur$, return $\bot$". A result of ch would require $target = ch = cur$, which that test rejects. The transition branch fails one step earlier where it applies: having arrived at t, d no longer matches an $f \neq t$, and for $f = *$ with $t = \mathsf{stable}$ the match requires $cur \neq \mathsf{stable}$, which fails. □

Corollary 103 (channel-axis ledger discharge)**.** *After any release of d, $W^{\circ}()$ excludes every delivered channel directive (Lemma 78), and Definition 33 admits none of them. The no-op and transition checks of Theorem 102 independently prevent re-proposing the channel already recorded in the baseline. These are complementary checks; the baseline comparison is not a substitute for fresh-work admission.*

The two axes discharge differently. The channel axis discharges through the target's undelivered window, with the no-op check on top; the bump axis discharges through delivery (Theorem 92), which on a target that has not released past the commit is the same window and on one that has is stricter. Under the retry invariant both survive a partial failure unchanged. This also marks the boundary of what Theorem 102 gives. It says the channel d arrived on is not re-proposed; it does *not* say nothing is proposed. The historical CCME 1.0.0 counterexample in proof obligation 138 used its unsplit window to let a second, older directive propose a different channel on every run. The fresh-window gate, normative since CCME 2.0.0, discharges that directive after release.

6.9 G8: axis independence

Theorem 104 (**G8**, axis independence)**.** *The channel assignment* $\mathrm{chan}_{\sigma}(\cdot)$ *is a function of σ, h and the configuration that does not read any quantity computed by phase 3. Consequently, altering the bump directives* $\mathrm{pb}(u), \mathrm{pd}(u), \mathrm{ps}(u)$ *of any set of units leaves* $\mathrm{chan}_{\sigma}(\cdot)$ *unchanged, and in particular a `W208` suppression removes no channel assignment and causes no package to lose a channel change it would otherwise have had.*

Proof. By Definition 33, phase 1 reads the units' channel directives, $\mathrm{src}_{\sigma}(u)$, the traversal, $W^{\circ}_{\sigma}(\cdot)$, $\mathrm{canc}(\cdot,\cdot)$, and, through Definition 32, $\mathrm{base}_{\sigma}(\cdot)$. By Definition 34, phase 2 reads phase 1's output and the same fixed quantities. None of these is a phase-3 output: $\mathrm{prp}_{\sigma}(\cdot)$, $\mathrm{eff}_{\sigma}(\cdot)$,

the admission predicate and $\text{owed}_\sigma(\cdot,\cdot)$ appear nowhere in Definitions 32 to 34. So $\text{chan}_\sigma(\cdot)$ is determined before phase 3 runs and is invariant under any change confined to phase-3 inputs. □

The dependency between the axes is thus a single directed edge, from channels to bumps, and Theorem 104 is the statement that there is no edge back. Two operational consequences follow, both worth stating because they are what a plan reader needs: a `W208` in a plan means a caret did not reach, not that a channel directive failed; and a package may legitimately appear in a plan with a channel change and no bump (`W202`), or with a bump and no channel change, and neither shape indicates a defect.

Remark 105 (the converse does not hold). CCME 1.0.0 stated **G8** as "suppressing a bump under SPEC §9.3a never suppresses a channel, *and vice versa*". SPEC states only the one-way result above. The old proof observed that the passes share only $\text{chan}_\sigma(\cdot)$, which phase 3 reads and never writes. That argument establishes exactly the direction proved above. The converse is false, and intentionally so: a channel suppressed by `W200` leaves $\text{chan}_\sigma(d)$ at its baseline value, and a bump from sources on another channel is then suppressed by Item (A6). Proof obligation 141 states the correction.

7 Determinism, idempotency, resumption

SPEC §17.2 states three obligations that are usually treated as informal quality properties. They are provable in this model, and two of them are corollaries of Section 6.

Theorem 106 (determinism)**.** $\Pi(\sigma)$*, the versions* $\nu_\sigma(\cdot)$*, the planned publish order and the ordered planning diagnostic list are functions of* $(\sigma, h, \text{config})$ *alone. Two conforming implementations, on two machines, at two times, produce identical plan output. Concurrent task completion, logs, and other operational events are outside this claim.*

Proof. Every quantity in Sections 2 and 3 is defined by a closed expression in σ, h and the configuration: Definitions 9 and 19 from ancestry and tags, Definition 10 and the owed windows of Definition 16 from the same; Definition 16 from commit messages and the configured types, separator and limits; Definition 28 from the graph at h; Definitions 32 to 35 and 37 from those; Definitions 21 and 44 from those. None mentions wall-clock time, commit dates, tag creation order, or the run history.

Three points admit a choice and each is resolved by a stated total order. Conflicting propagated channels and disagreeing origin channels resolve to the byte-wise least package name (`W160`, Definition 32). Competing directives resolve by SPEC §11.6 precedence: newest commit first, then last unit within the commit, a total order on units, since commits carry a topological index and units an index within their commit. Publish-order ties resolve by byte-wise least name (Proposition 45). SPEC §17.2 additionally requires every collection reaching the output to be sorted by package name, then commit topological index, then unit index, so no unordered container's iteration order is observable. A function of fixed inputs with all ties broken by total orders is single-valued. □

Theorem 107 (resumption after partial failure)**.** *Assume the retry invariant and* $\text{wf}(\sigma_1)$*. Let* $\sigma_2 = \text{run}(\sigma_1, F)$ *with blocked set* $B = B(\sigma_1, F)$ *and published set* $S(\sigma_1, F)$*, and suppose the run reports* `E201` *for no package. Then*

$$\Pi(\sigma_2) \;=\; F \cup B \cup A, \qquad A = \{\, p \in S(\sigma_1, F) \;:\; \text{owes}_{\sigma_2}(P, p) \text{ for some } P \in F \cup B \,\},$$

and $\nu_{\sigma_2}(p) = \nu_{\sigma_1}(p)$ *for every* $p \in F \cup B$*.*

Proof. $\subseteq$: $\Pi(\sigma_2) \subseteq \Pi(\sigma_1)$ by Lemma 96, and every member of $\Pi(\sigma_1)$ lies in F, in B or in $S(\sigma_1, F)$. Take $p \in \Pi(\sigma_2) \cap S(\sigma_1, F)$. It is tagged at h in σ_2, so by Lemma 84 its only cause is a unit u with $\mathrm{adm}_{\sigma_2}(u, p)$, and every $P \in \mathrm{owed}_{\sigma_2}(u, p)$ is in $\Pi(\sigma_2) \subseteq \Pi(\sigma_1)$ and untagged at h, hence not published in run 1, hence in $F \cup B$; so $p \in A$. $\supseteq$: each $p \in F \cup B$ was planned in run 1 and not released, and the run reported `E201` for nothing, so Theorem 89 with $k = 2$ gives $p \in \Pi(\sigma_2)$ at the same version. Each $p \in A$ has an admitted unit with a non-empty owed set, so $\mathrm{prp}_{\sigma_2}(p) \neq \mathsf{none}$, and $\mathrm{held}(p)$ is constant and false; so $p \in \Pi(\sigma_2)$. □

The set A is new with delivery: the consumers that proceeded past a commit on a cause of their own while a provider failed or was blocked, and are owed that provider's release. They return to the plan at a version their catch-up computes, not at the version they were first planned at (Theorem 89 says nothing about a released package), and they are ordered after the provider (Lemma 84). Under the window test of CCME 2.0.0, A was empty, because a published package could not be planned again at the same h; that is the orphan of Example 39, second shape.

Theorem 107 is the exact statement of what the operator is promised after a failed run under the retry invariant: the next run does the rest of the same job, at the same numbers for what never released and at the owed catch-up for what proceeded, without coordinator state to clean up. An ambiguous external publish outcome still requires SPEC §19.4 reconciliation. It is also what SPEC §19.6 checks. After a run in which nothing failed or was blocked, $F \cup B = \emptyset$, so $A = \emptyset$ and the re-planned set must be empty; a non-empty one is `E199`. Inside the retry invariant it contradicts Theorem 107; outside it, SPEC §19.6 reports the changed admission inputs for renewed review.

Corollary 108 (idempotency)**.** *Running the engine twice with no intervening commits and no publish yields the same plan. Under the retry invariant, running it after a fully successful publish from a well-formed state yields the empty plan.*

Proof. The first is Theorem 106: the state is unchanged, so the plan is. The second is Theorem 107 with $F = \emptyset$: then $B = \emptyset$, no `E201` is reported (it needs a failed package), and $A = \emptyset$ because nothing is left to owe; so $\Pi(\sigma_2) = \emptyset$. Equivalently, every $p \in \Pi(\sigma_1)$ is tagged at h in σ_2 together with every package it was owed by, so Lemma 84 leaves it no cause; the invariant excludes both input drift and newly visible catch-up. □

Remark 109 (what determinism forbids)*.* Theorem 106 is the reason SPEC §13.11 may permit optimisation of the computation but not of its result. An implementation may cache identical traversals, invert loops, cache lexical parses and publish concurrently; it may not introduce parallelism whose scheduling affects the iteration order of any collection that reaches the output. Section 9 records the precise conditions under which such transformations preserve those observables.

8 The staleness dual

SPEC §9.2 walks *down* from the packages a commit changed. An audit command wants the opposite walk: given a package, which of its dependencies is it behind, and by how much? SPEC §13.7b gives that walk and asserts that the two agree, calling disagreement an implementation bug. This section proves that the audit is sound against SPEC §9.2 unconditionally, that it is complete under two conditions the current text does not state, and exhibits the disagreements outside them. The pseudo-code is reproduced from SPEC §13.7b unchanged, except that the arrow and ellipsis glyphs in the comment on `dist`, which the monospace font lacks, are set as `->` and `....`

```
staleSources(D):
    edges = graph.restrictedTo(config.propagation.kinds)  # ğ8.4: the same edges as ğ9.2
    channel = resolveChannelsForAudit()                    # same phase-2 result as ğ9.2
    dist  = shortestPathsUp(D, edges)   # P -> edge count of the shortest path P -> ... -> D
    out   = {}
    for u in units:                                        # every retained tuple, ğ13.4
        if bumpOf(u) == none:                              continue
        b = (u.propagate == 'inherit') ? bumpOf(u) : u.propagate    # as ğ9.2
        if b == none or u.depth == 0:                      continue
        sources = sourcePackages(u)                        # ğ13.4a
        if sources is empty:                               continue
        if D in sources:                                   continue   # ğ9.2 seeds seen = sources
        sources = owed(u, D)                               # ğ13.4a: what D is still owed
        if sources is empty:                               continue   # every source delivered
        reaching = { P in sources : P in dist }
        if reaching is empty:                              continue
        if not resolvableBy(sources, D, channel):     continue   # ğ9.3a
        level = min({ dist[P] for P in reaching })         # measured from the whole source set
        if level > u.depth:                                continue
        if D not in resolve(u.propagateScope):             continue
        if cancelledFor(commitOf(u), D):                   continue
        for P in reaching:
            out |= { (P, u, level, b) }
    return out
```

Three things changed from the CCME 2.0.0 audit. The loop runs over every retained tuple, not over the units of $W^{\circ}(D)$: a commit D has released past can still be owed to it by a source that released it later, and iterating the target's window alone misses exactly the consumers that got ahead of a provider, which are the ones the audit exists to find after a partial failure. `sources` is rebound to $\mathrm{owed}(u, D)$ before the distance and resolvability tests, so the rows name the owed sources, as `prov[D]` does in SPEC §9.2. And `resolvableBy` is tested before the level test.

Definition 110. $\mathrm{stale}_{\sigma}(d) = \{(p, u, \ell, b)\}$ is the set the procedure returns. For the rows of a unit u, p ranges over the owed sources of u that reach d, $\ell = \delta(\mathrm{owed}_{\sigma}(u, d), d)$, the least distance from an owed source, and $b = \mathrm{pb}(u)$. Write $\mathrm{sch}_{\sigma}(u)' = \{\mathrm{chan}_{\sigma}(P) : P \in \mathrm{owed}_{\sigma}(u, d)\}$ for the channel set the audit tests, and $\mathrm{rsv}_{\sigma}(u, d)'$ for Definition 35 evaluated over it.

Theorem 111 (duality)**.** *For every state σ and package d:*

(a) (soundness) if u contributes a row to $\mathrm{stale}_{\sigma}(d)$ *then* $\mathrm{adm}_{\sigma}(u, d)$*; hence* $\mathrm{stale}_{\sigma}(d) \neq \emptyset$ *implies* $\mathrm{prp}_{\sigma}(d) \neq \mathsf{none}$ *and* $\bigsqcup\{\, b : (p, u, \ell, b) \in \mathrm{stale}_{\sigma}(d) \,\} \leq \mathrm{prp}_{\sigma}(d)$*;*

(b) (completeness) if (R) holds and $\mathrm{rsv}_{\sigma}(u, d)' = \mathrm{rsv}_{\sigma}(u, d)$ *for every u with* $\mathrm{owed}_{\sigma}(u, d) \neq \emptyset$*, then u contributes a row exactly when* $\mathrm{adm}_{\sigma}(u, d)$*, and*

$$\mathrm{stale}_{\sigma}(d) \neq \emptyset \iff \mathrm{prp}_{\sigma}(d) \neq \mathsf{none}, \qquad \bigsqcup\{\, b : (p, u, \ell, b) \in \mathrm{stale}_{\sigma}(d) \,\} = \mathrm{prp}_{\sigma}(d).$$

Proof. Every row of u carries $b = \mathrm{pb}(u)$, and $\mathrm{prp}_{\sigma}(d)$ is the join of $\mathrm{pb}(u)$ over admitted units (Definition 38), so both parts reduce to comparing "u contributes a row" with $\mathrm{adm}_{\sigma}(u, d)$.

(a). A row means every guard passed. `bumpOf(u) == none`, `b == none` and `u.depth == 0` are Item (A1). `sources is empty` on the full source set is the first half of Item (A2), and `D in sources` is the $d \notin \mathrm{src}_{\sigma}(u)$ of Definition 28, read from the other end: it is the same clause as seeding `seen` with the sources. `sources is empty` after the rebinding is Item (A4). `level` is $\min\{\delta(\{P\}, d) : P \in \texttt{reaching}\} = \delta(\mathrm{owed}_{\sigma}(u, d), d)$, because `shortestPathsUp` gives

$\delta(\{P\}, d)$ for each P and a non-reaching source has distance ∞; and $\text{owed}_\sigma(u, d) \subseteq \text{src}_\sigma(u)$ gives $\delta(\text{src}_\sigma(u), d) \leq \delta(\text{owed}_\sigma(u, d), d) = \texttt{level} \leq \text{pd}(u)$, which with $d \notin \text{src}_\sigma(u)$ is the second half of Item (A2) by Lemma 29. `D not in resolve(u.propagateScope)` is Item (A3); `cancelledFor` is Item (A5). Finally $\text{sch}_\sigma(u)' \subseteq \text{sch}_\sigma(u)$, so $\text{rsv}_\sigma(u, d)'$ implies $\text{rsv}_\sigma(u, d)$, which is Item (A6). So every guarded unit is admitted, and the join over a subset of the admitted units is at most the join over all of them.

(b). Conversely let $\text{adm}_\sigma(u, d)$. Item (A1), Item (A2)'s first half, Item (A4), Item (A3) and Item (A5) pass the corresponding guards as above, and $d \notin \text{src}_\sigma(u)$ passes `D in sources`. By (R) every owed source reaches d, so `reaching` $= \text{owed}_\sigma(u, d) \neq \emptyset$, and `level` $= \delta(\text{owed}_\sigma(u, d), d) = \delta(\text{src}_\sigma(u), d) \leq \text{pd}(u)$ by Item (A2). By the second hypothesis Item (A6) passes `resolvableBy` over the owed sources. So u contributes a row, the guarded set is the admitted set, non-emptiness is equivalent to $\text{prp}_\sigma(d) \neq$ `none`, and the joins agree. □

Remark 112 (where the two formulations disagree). SPEC §13.7b says the two formulations MUST agree and that disagreement is an implementation bug. As written they do not agree, and the disagreement is in the text, not in an implementation; three shapes, all on Figure 1, all with a multi-source unit and a target that got ahead of one source.

- *An owed source that does not reach the target.* Unit `feat(core,api)^: x` at C; `cli` depends on `core` only. Let `core` have delivered C to `cli` and `api` have no release carrying C. SPEC §9.2 reaches `cli` at level 1 through `core`, finds $\text{owed}(u, \texttt{cli}) = \{\texttt{api}\}$, and bumps `cli` with provenance `api`, a package it does not depend on. The audit rebinds `sources` to {`api`}, finds `reaching` empty, and reports nothing. Definition 24 then also demands that `api` precede `cli` in the publish order whenever `cli` is tagged at h, a constraint no edge supports.
- *An owed source further away than a delivered one.* Unit `feat(core,ui)^: x` at C, target `@acme/theme`, which depends on `ui` and reaches `core` at distance 2. Let `ui` have delivered C to `@acme/theme` and `core` not. SPEC §9.2 admits at level 1 with $\text{owed}(u, \texttt{@acme/theme}) = \{\texttt{core}\}$; the audit computes `level` $= 2 > 1$ and reports nothing.
- *Resolvability over different sets.* The same unit with `core` on stable and `ui` on `beta`, target `@acme/theme` on stable, `core` delivered and `ui` owed. SPEC §9.2 hoists `anyStable` over the full source set and admits; the audit tests {`beta`} against a stable target and rejects, which is what SPEC §9.3a intends.

In each case the audit under-reports relative to SPEC §9.2; by Theorem 111(a) it never over-reports. The first shape is the more serious, since SPEC §9.2's answer is the questionable one: a bump owed by a package the target cannot reach, and a well-formedness constraint over a non-edge. A definition of $\text{owed}(u, D)$ restricted to sources that reach D would remove the first two shapes and make (R) a theorem; the third needs SPEC §9.2 and SPEC §13.7b to agree on which source set resolvability is evaluated over. All three are flagged for the specification.

Example 113 (the audit, on both failed runs)**.** At σ_1 of the first shape of Example 39, where `core` has published and `cli` has not, the audit for $d =$ `cli` walks up from `cli`, finds `core` at `dist`[`core`] $= 1$, finds $\text{owed}_{\sigma_1}(u, \texttt{cli}) = \{\texttt{core}\}$, and returns the single row

$$\text{stale}_{\sigma_1}(\texttt{cli}) = \{\, (\texttt{core},\ u,\ 1,\ \texttt{patch}) \,\},$$

whose join is `patch` $= \text{prp}_{\sigma_1}(\texttt{cli})$, as Theorem 111 requires; the unit has one source, so (R) and the resolvability condition hold outright. Run the same audit for `ui`, which did publish, and it is empty: $\text{dlv}_{\sigma_1}(\texttt{core}, C, \texttt{ui})$ empties the owed set. At σ_1 of the second shape, where `cli`

proceeded and is tagged at C, the audit returns the same row for `cli`, although $C \notin W^{\circ}_{\sigma_1}(\texttt{cli})$: this is the row the CCME 2.0.0 audit, which iterated the target's fresh window, could not produce, and it is the question an operator actually asks after a failed run (which of my packages are behind, and behind what), answered without recomputing a plan.

The clauses the proof leans on are the ones SPEC §13.7b warns an implementation drifts on, and it is worth naming what each drift produces. Excluding d from its own unit's sources: without it, a package that both changes and consumes a sibling in one unit audits as behind itself. Measuring `level` from the whole set rather than per ancestor: taking `dist`$[p]$ for each p separately over-reports the level and so under-reports staleness, exactly in the diamond cases. Iterating the target's window rather than every retained tuple: the consumer that got ahead is invisible. And resolving `inherit` to the unit's own bump before testing it against `none`.

Proposition 114 (the tag-level screen is necessary, not sufficient)**.** *For packages P, D with baselines, say the screen of* SPEC *§13.7b* fires *when* $\mathrm{basec}_\sigma(P) \not\preceq \mathrm{basec}_\sigma(D)$*. Then:*

(a) the screen fires iff $\neg\mathrm{dlv}_\sigma(P, \mathrm{basec}_\sigma(P), D)$*: it is Definition 10 with the unit's commit replaced by P's own baseline commit;*

(b) if $\mathrm{adm}_\sigma(u, d)$*,* $P \in \mathrm{owed}_\sigma(u, d)$ *and* $c_u \preceq \mathrm{basec}_\sigma(P)$ *(the source has released the work and its baseline reaches that release), then the screen fires for* (P, d)*;*

(c) the converse of (b) fails.

Proof. (a): $\mathrm{dlv}_\sigma(P, \mathrm{basec}_\sigma(P), D)$ asks for $t \in T_\sigma(P)$ with $\mathrm{basec}_\sigma(P) \preceq t \preceq \mathrm{basec}_\sigma(D)$; any such t gives $\mathrm{basec}_\sigma(P) \preceq \mathrm{basec}_\sigma(D)$ by transitivity, and conversely $t = \mathrm{basec}_\sigma(P)$ is a release commit of P. (b): if $\mathrm{basec}_\sigma(P) \preceq \mathrm{basec}_\sigma(d)$ then $t = \mathrm{basec}_\sigma(P)$ witnesses $\mathrm{dlv}_\sigma(P, c_u, d)$, contradicting $P \in \mathrm{owed}_\sigma(u, d)$. (c): a provider may be legitimately ahead of a consumer with nothing owed: the units between them may all declare `^none` or depth `+0`, may be excluded by `Propagate-Scope`, may reach d only beyond their declared depth, or may travel edges of a kind not in `propagation.kinds`. Each gives a state where the screen fires and no bump is admitted. □

The screen is therefore a way to *find* candidates cheaply, by comparing two tags rather than walking a window, and never a way to decide releases; using it to decide would manufacture bumps that no commit asked for. Part (a) is why it screens: a consumer owed something has a provider release its own release does not reach. Part (b) needs the provider to have released the work; a debt from a provider that has not released the commit at all is not the screen's business, since that provider is then in the plan on its own account (Lemma 84). There is also a case the screen cannot see at all: P and d released at the same commit, in the same run, with d published first. Ancestry cannot distinguish them and no rule over tags can. That case is prevented rather than detected, by publishing in dependency order (Definition 44) and by refusing a run that would release a provider at the baseline commit of a consumer it still owes without that consumer following (Definition 24); where the consumer follows and fails, Proposition 26 is the reported remainder.

Remark 115 (historical source of the eligibility guard)*.* CCME 1.0.0 omitted the resolvability guard, so wherever SPEC §9.3a suppressed a bump the two formulations disagreed. Proof obligation 140 preserves the counterexample; CCME 2.0.0 made the repair normative, and the current text keeps it while adding the delivery rebinding whose consequences Remark 112 records.

9 Cost

The current SPEC (3.1.0-rc.2) makes SPEC §13.11 normative for semantic equivalence and informative for cost and technique. An implementation must produce the plan SPEC §13 defines, but need not use or attain any bound below. The accounting assumes decoded, indexed inputs with fixed-width or interned identifiers and adjacency and precedence inputs already in their specified order; comparison, decoding, sorting, allocation, adapter, record matching and emitted output costs remain, and hash-table operations are expected constant time under the chosen hashing assumptions. This section transcribes the phase table, proves the transformations it relies on and the parser bound SPEC §18.3 depends on, and states the space ledger; it makes no global optimality claim.

Notation follows SPEC §13.11 so that the table can be read beside it. P packages, E workspace dependency edges, H commits and A parent edges in the history reachable from h, C commits in the union of all pending windows, U units in those commits, N total bytes of their messages and changed paths, T reachable tags, and M tag-record and package-format matches examined while partitioning the inventory (at worst $P \cdot T$). k is the number of distinct commits carrying a boundary: a stable baseline, a package's newest baseline of any channel where that is another commit, or the release of a provider that bounds a consumer's owed window (SPEC §13.3), of which there are none while no consumer has got ahead of a provider. m is the number of distinct marker commits, the commits some ancestry question of SPEC §13 is asked about: the k boundaries, every `cancel` commit, and every commit carrying an `Edits`, `Deletes` or `Reverts` footer. R is the work of resolving scopes and changed paths, I the number of unit-to-package incidences (up to $P \cdot U$), Z the retained or emitted contribution and provenance incidences and Z_v those examined while constructing them, I_w the stored or emitted package-window memberships (up to $P \cdot C$). F_c counts correction footer selectors, J_{corr} the package-target incidences examined for correction containment and application, I_c cancel-scope incidences, P_c packages named by a cancel, and J_c cancellation queries across direct admission and both propagation axes. $\mathrm{bw}(x) = \max(1, \lceil x/w \rceil)$ for word size w, so a zero-marker phase still pays for traversal. F is the number of publish failures and O_{out} the size of emitted output. In the per-target row only, D is the number of targets one unit reaches, S its source-set size and Σ its resolved scope-set size. For the polyrepository profile, Q is the number of repositories, H_q and A_q one repository's reachable commits and parent edges, G the control gitlink transitions examined, K_q the distinct boundary revisions used in repository q, m_q the markers in q, V the package memberships in shared-version groups, X the (consumer release tag, repository) boundaries resolved under SPEC §27.11, Y the longest route between two peers and B the distinct (repository, revision) trees read. For the execution profile, T_k is the run's tasks, D_k their precedence edges, W_n the worker nodes, P_o the packages whose outputs another task consumes, and L_r the coordination refs alive in one transport repository.

Phase	Literal	Achievable	Note
Load workspace (SPEC §13.1)	$O(P + E)$	$O(P + E)$	
Load tags (SPEC §13.2)	$O(T \log T + M)$	$O(T + M)$	running maximum per package; hash (package, version) for `E191`
Pending windows (SPEC §13.3)	$O(P \cdot (H + A))$	$O((H+A)\,\mathrm{bw}(m)+I_w)$	one marker pass, Proposition 117; $O((k+1)(H+A)+I_w)$ with a walk per boundary, Proposition 116
Parse and resolve (SPEC §13.4)	$O(N + R + I)$	$O(N + R + I)$	lexing and resolution have different cache keys; Proposition 130

Phase	Literal	Achievable	Note
Corrections (SPEC §13.4b)	repeated ancestry and scope scans	$O(H + F_c + J_{corr} +$ wildcards $\cdot\, I + O_{out})$	indexed ancestry charged in the marker pass; package-target probes counted even without wildcards
Cancellation predicates (SPEC §13.5, SPEC §9.2)	a cancel scan per query	$O(I_c + (P_c + J_c)\,\mathrm{bw}(\text{cancels}))$	per-package cancel masks; ancestry charged in the marker pass
Direct bumps (SPEC §13.6)	$O(U + I)$	$O(U + I)$	consume resolved incidences
Holds (SPEC §13.6a)	$O(U + I)$	$O(U + I)$	consume resolved incidences
Propagation graph walks (SPEC §13.7)	$O(U \cdot (P + E))$	input-dependent	reuse a walk only when its full inputs match; Theorem 85, Proposition 122
Propagation materialisation	repeated set unions	$O(Z_v + Z)$	in-place indexed insertion; only unique required output has the lower bound $\Omega(Z)$
Per-target predicates (SPEC §9.3a)	$O(D \cdot (S + \Sigma))$	$O(D + S + \Sigma)$	per unit; hoist the source-channel set and scope; Proposition 125
Channel resolution (SPEC §13.8)	$O(P \cdot U)$	$O(U + I + P)$	invert unit-to-package incidences; Proposition 126
Versions and plan (SPEC §13.9, SPEC §13.10)	repeated member and record scans	$O(P + I + Z + O_{out})$	consume built aggregates; scan each disjoint version group once
Publish order (SPEC §19.2)	$O(P^2 + E)$	$O(E + P \log P)$	a comparison heap over the ready set; Proposition 127
Blocking closure (SPEC §19.3)	$O(P \cdot (P + E))$	$O(P + E)$ per run	one multi-source reverse traversal; Proposition 128
Build readiness (SPEC §19.2a)	$O(P \cdot (P + E))$	$O(P + E)$	a pass-through node per non-building package, or a memoised index; Proposition 129
Polyrepository snapshots (SPEC §27)	repeated control scans	$O(G + \sum_q (H_q + A_q))$	index control gitlinks once; walk each source snapshot once
Polyrepository windows (SPEC §27)	$O(P \cdot \sum_q (H_q + A_q))$	$O(\sum_q (H_q + A_q)\,\mathrm{bw}(m_q) + I_w)$	one marker pass per repository; $O(\sum_q K_q(H_q + A_q) + I_w)$ with a walk per boundary
Publication input closure (SPEC §27.2)	$O(P \cdot (P + E + V))$	$O((P + E + V)\lceil Q/w \rceil)$	condense the group-augmented graph, one bitset union per edge
Link evidence (SPEC §27.11)	$O(X \cdot (Q + Y))$, $X \cdot Y$ tree reads	$O(Q + X \cdot Y)$, B tree reads	root the link tree once; read each (repository, revision) tree once
Link settlement (SPEC §27.11)	one commit per route hop	one commit per recording repository	at most $Q - 1$ commits, two in a star
Record comparison (SPEC §13.2)	$O(T)$ per repository	$O(T)$ per repository	one inventory read under the lock; Definition 42
Task graph and readiness (SPEC §28.3)	$O(T_k \cdot (T_k + D_k))$	$O(T_k + D_k)$	indegree counters decrement each successor once
Placement (SPEC §28.2)	$O(T_k \cdot W_n)$	$O(T_k \cdot W_n)$	W_n is small; a free list per compatibility class is rarely worth its bookkeeping
Output transfer (SPEC §28.5)	up to $T_k \cdot P_o$ transfers	at most $P_o \cdot W_n$ transfers	one per distinct (output set, node); verified bytes are not fetched again
Coordination polling (SPEC §28.4)	$O(L_r)$ refs per tick	refs addressed to the polling node	the name prefix routes; one fetch per changed tip; dead runs tax every tick until removed

Multiplicative costs, corrections, cancellation, provenance and retained indexes can each dominate and must be charged even when their final result is empty. The candidate-traversal row is not a bound on the complete planner: a cached walk is reusable only when its source set,

depth and edge kinds are identical, and it must retain each unit's own distance, fresh-window membership, cancellation, channel eligibility, self-source exclusion and provenance. Per-unit predicate checks and output construction add work, and this report does not prove that all such work fits the bucket traversal bound. The polyrepository and execution rows are sums over repositories and counts of tasks respectively; neither is a globally linear end-to-end claim.

9.1 Windows

Proposition 116 (window bucketing)**.** *$\{W_\sigma(p) : p \in \mathcal{P}\}$ has at most $k_s + 1$ distinct members, where $k_s \le k$ is the number of distinct defined values of* $\mathrm{scom}_\sigma(\cdot)$*. Computing* $\mathrm{reach}(\cdot)$ *once per distinct boundary commit, once more for packages with no stable baseline, and testing membership by lookup costs $O((k+1)(H+A))$ reachability work and $O(I_w)$ for the memberships stored or emitted.*

Proof. By Definition 9, $W_\sigma(p)$ depends on p only through $\mathrm{scom}_\sigma(p)$; packages sharing a stable baseline commit share a window, and packages with no stable baseline share $\mathrm{reach}(h)$. So the map from the k_s stable boundary commits, plus the undefined case, onto distinct windows is surjective, giving the count. Each traversal visits at most H commits and A parent edges, there are at most $k+1$ of them when every boundary of SPEC §13.3, newest-baseline and owed-window boundaries included, gets one, and each membership stored or emitted is charged once in I_w. □

The traversals range over the reachable history, so the bound is in $H + A$ and not in C: proving that a commit is outside a window may require walking commits that never enter the union. Commit-graph generation numbers can reject candidates and bound a walk but cannot prove ancestry across branches; a positive answer still needs a reachability query or a precomputed index. k counts distinct boundary commits, not release runs, and can exceed P, since a release may record packages at different commits and a provider's releases can bound several consumers' owed windows.

Proposition 117 (one marker pass, SPEC §13.11)**.** *Give each of the m markers one bit, and visit the reachable history once, children before parents: when a commit is visited, set its own marker bit if it is a marker, then OR its mask into the mask of each of its parents. When the pass ends, bit x of* $\mathrm{mask}(c)$ *is set exactly when $c \in \mathrm{reach}(x)$. The pass costs $O((H+A)\,\mathrm{bw}(m))$ word operations, against $O(m\cdot(H+A))$ for a walk per marker, and dense rows occupy $H\cdot\lceil m/w\rceil$ words.*

Proof. $c \in \mathrm{reach}(x)$ exactly when $c = x$ or some child c' of c satisfies $c' \in \mathrm{reach}(x)$, since a path from x down to $c \neq x$ ends with an edge from a child of c to c. Claim: when a commit is visited its mask is final. By induction along the visiting order: every child of c is visited before c, each child's mask is final at its visit by the induction hypothesis, and each child ORs that final mask into $\mathrm{mask}(c)$ at its visit; nothing else writes $\mathrm{mask}(c)$ after the children, apart from c's own bit set at its visit. So at the end bit x of $\mathrm{mask}(c)$ is set iff $c = x$ or bit x is set in some child's final mask, which by the same induction is iff $c = x$ or some child is in $\mathrm{reach}(x)$, that is, iff $c \in \mathrm{reach}(x)$. Each commit is visited once and each parent edge relaxed once, with one $\lceil m/w\rceil$-word OR per relaxation and one traversal step even when $m = 0$, giving the cost. □

Children-before-parents is the reverse of the order SPEC §25 requires of an adapter, which returns parents first. Afterwards $c \in \mathrm{after}(b)$ is one clear bit, a cancellation test is a non-empty intersection of $\mathrm{mask}(c)$ with the cancel markers whose scope contains the package, and the proper-ancestor test of SPEC §13.4b is one set bit together with $c \neq x$. Under SPEC §27

the pass runs once per repository over that repository's own markers, with repository identity in every key (Definition 75). A pass confined to the union of the pending windows and the parent edges inside it sets the same bits for the union's commits, because a window is closed under descendants and no path leaves the union and returns to it; what remains in $H + A$ is finding the union.

Corollary 118 (the fresh-window identity)**.** *With* $\text{after}(b) = \text{reach}(h) \setminus \text{reach}(b)$*, a commit* c *is in* $W^{\circ}_{\sigma}(p)$ *exactly when* $c \in \text{after}(\text{scom}_{\sigma}(p))$ *and* $c \in \text{after}(\text{basec}_{\sigma}(p))$*. Every window is therefore a function of one boundary commit, a fresh membership test is the conjunction of two lookups, and no reachability set is keyed by a* (*stable, fresh*) *pair; the identity holds whether or not the newest baseline descends from the stable one.*

Proof. By Definition 9, $W^{\circ}_{\sigma}(p) = (\text{reach}(h) \setminus \text{reach}(\text{scom}_{\sigma}(p))) \setminus \text{reach}(\text{basec}_{\sigma}(p)) = \text{reach}(h) \cap \overline{\text{reach}(\text{scom}_{\sigma}(p))} \cap \overline{\text{reach}(\text{basec}_{\sigma}(p))} = \text{after}(\text{scom}_{\sigma}(p)) \cap \text{after}(\text{basec}_{\sigma}(p))$, with an empty subtracted set where a boundary is undefined. Nothing in the derivation relates the two boundaries to each other. □

Window storage may still be shared only inside one plan and one fixed snapshot, between packages whose boundary identity is equal, and the shared set must be immutable; a materialised fresh window is shared only when both boundaries are equal. Storing $W_{\sigma}(p)$ explicitly per package costs $O(P \cdot C)$ memory; $k + 1$ dense bitsets of H bits, or $O((k + 1) \cdot C)$ bits over the union once it is known, or a sparse $O(I_w)$, each make admission a lookup and none turns the history traversal into $O(C)$.

Proposition 119 (early stop, SPEC §13.11)**.** *Let the pass of Proposition 117 visit commits in an order known to be children-before-parents without a full walk, decreasing commit-graph generation number for instance. Once every unvisited commit that has a visited child carries all* k *boundary bits, every unvisited commit lies in no window and is named by no tuple of* SPEC *§13.4, so the pass may stop. The stop is sound only when no package lacks a stable baseline, and only after every commit named by an* `Edits`*,* `Deletes` *or* `Reverts` *footer has been visited.*

Proof. h has the greatest generation number among reachable commits, since the number increases strictly along child edges, so h is visited first. Let c be unvisited and take any path $h = c_0 \to c_1 \to \cdots \to c_j = c$. It has a first unvisited commit c_i with $i \geq 1$, whose child c_{i-1} on the path is visited, so c_i carries every boundary bit: $c_i \in \text{reach}(x)$ for every boundary x, and c, an ancestor of c_i, is in every $\text{reach}(x)$ too. Every train window is $\text{reach}(h) \setminus \text{reach}(x)$ for a stable boundary x when every package has one, every fresh window is a subset of a train window, and every owed window of SPEC §13.3 excludes $\text{reach}(x)$ for the provider release x that bounds it, which is among the k boundaries; so c is in no window. A package without a stable baseline keeps $\text{reach}(h)$ as its window, which no boundary bit excludes, and the diagnostics `E210` and `W213` of SPEC §13.4b read the ancestry of a correction target that lies outside every window; each is the stated condition. □

Remark 120 (the delivery test)*.* The delivery test of Definition 19 is one window bit for a target that has not released past the unit's commit, which is every target in the ordinary case. A target that has costs one ancestry question per release of each source past that commit, answered from the marker pass when those releases are markers, and such targets exist only after a consumer got ahead of a provider. The owed windows those targets add are counted in k, and are empty while no consumer has got ahead.

9.2 Safe traversal reuse

The superseded bucketing proposal grouped units by window class and directive tuple and traversed from the union of their sources. That key is insufficient: unit identity affects admission, cancellation, scope, inherited channel, self-source exclusion and output provenance. SPEC §13.11 identifies identical graph-walk inputs as the one safe memoisation key, and additionally permits one unbounded walk per (source package, edge kinds) whose union across the sources of one unit, minus those sources, at the least level per target, is exact within that unit. Other optimisations still require an equivalence argument. The following historical result characterises why the naive merge across units is unsafe.

Definition 121 (the naive merged candidate set, SPEC §13.11)**.** For $S \subseteq \mathcal{P}$ let $\delta^+(S, d)$ be the least length of a chain of *at least one* edge from a member of S to d, and $R(S, \ell)^+ = \{d : \delta^+(S, d) \leq \ell\}$. This is the naive positive-edge candidate set; SPEC additionally requires attribution to an eligible unit and that unit's self-source exclusion.

Proposition 122 (limit of naive merged bucketing)**.** *Let $u_1, \ldots, u_m$ share a depth ℓ and let $S_i = \mathrm{src}_\sigma(u_i)$, $S = \bigcup_i S_i$. Then*

$$\bigcup_i R(S_i, \ell) \;\subseteq\; R(S, \ell)^+,$$

the two sides agree on every $d \notin S$, and they differ exactly on the packages

$$X \;=\; \{\, d \in S \;:\; \delta^+(S, d) \leq \ell \;\; \textit{and} \;\; \forall i,\; d \in S_i \;\textit{or}\; \delta(S_i, d) > \ell \,\},$$

which $R(S, \ell)^+$ admits and no individual traversal does. In particular, if no $\mathrm{src}_\sigma(u_i)$ contains a package reachable from that same set within ℓ edges, then $X = \emptyset$ and bucketing is exact.

Proof. Inclusion: if $d \in R(S_i, \ell)$ then $d \notin S_i$ and $\delta(S_i, d) \leq \ell$, so the witnessing chain has length at least 1 and starts in S, giving $\delta^+(S, d) \leq \ell$.

Agreement off S: let $d \notin S$ and $\delta^+(S, d) \leq \ell$. The witnessing chain starts at some $p \in S_i$, and $d \notin S_i$, so $d \in R(S_i, \ell)$.

The difference: for $d \in S$, membership of $R(S, \ell)^+$ says only that some chain of length $1..\ell$ reaches d from S, whereas membership of $\bigcup_i R(S_i, \ell)$ requires some i with $d \notin S_i$ and $\delta(S_i, d) \leq \ell$. Negating the latter gives the stated X. □

Example 123 (the bucket that over-admits)**.** On Figure 1, take the single unit

$$u \;=\; \texttt{fix(core,ui)\^{}major:\ \ x},$$

so $S_1 = \{\texttt{core}, \texttt{ui}\}$ and $\ell = 1$. Per Example 30, $R(S_1, 1)$ excludes `ui`, which takes the unit's own `patch` directly. But $\delta^+(S_1, \texttt{ui}) = 1$ by the edge `ui` → `core`, so `ui` $\in R(S_1, 1)^+$: the merged rule gives `ui` the propagated `major` as well, publishing it two levels higher than the specification says, with no diagnostic to show for it. Here $X = \{\texttt{ui}\}$.

Remark 124 (what X costs, and what SPEC §13.11 says about it). The warning box in SPEC §13.11 is about the opposite error, subtracting the merged source set S, which withholds the bumps of a package that is a source of one unit and a legitimate target of another, and now requires a per-unit self-source test. Merely using $R(S, \ell)^+$ over-admits on X: a unit whose own scope-set contains both a package and a dependent of it within the depth bound. Then the dependent, which SPEC §9.2 excludes from that unit's targets because the unit bumps it directly, receives the unit's *propagated* bump as well, a difference whenever `Propagate` names a value above the unit's own bump, as in `fix(core,cli)^major`. Proof obligation 143 preserves the 1.0.0 counterexample that motivated the current rule.

9.3 Hoisting

Proposition 125 (the resolvability predicate is loop-invariant)**.** $\mathrm{rsv}_\sigma(u, d)$ *as computed by the source scan*

$$\exists p \in \mathrm{src}_\sigma(u) : \ \mathrm{chan}_\sigma(p) = \mathsf{stable} \ \vee \ \mathrm{chan}_\sigma(p) = \mathrm{chan}_\sigma(d)$$

is equal, for every d, to the membership test of Definition 35 against the precomputed set $\mathrm{sch}_\sigma(u)$*. Evaluating it once per unit and testing membership per target costs $O(D + S)$ where the literal form costs $O(D \cdot S)$; hoisting c_u and* $\mathrm{ps}(u)$ *likewise takes the unit from $O(D(S + \Sigma))$ to $O(D + S + \Sigma)$.*

Proof. A set contains $\mathrm{chan}_\sigma(d)$ exactly when some $p \in \mathrm{src}_\sigma(u)$ has $\mathrm{chan}_\sigma(p) = \mathrm{chan}_\sigma(d)$, and contains stable exactly when some p has $\mathrm{chan}_\sigma(p) = \mathsf{stable}$; the disjunction of the two membership tests is therefore the disjunction of the two existentials, which is the source scan. The set has one entry per *distinct* channel among the sources. The predicate reads d only in that one comparison, and Item (A3) and c_u do not read d at all, so all three may be computed before the target loop. □

The cost matters because of where it lands: a unit carrying `^^` across a wide scope-set admits thousands of targets from a source set of one or two packages, and the literal reading re-walks those sources thousands of times to re-derive an answer that never changed. Hoisting is not an optimisation to be justified by profiling; it is what the predicate's own dependency structure already says. The bound covers the hoisted scope and channel predicates only; cancellation intersections cost up to bw(cancels) per queried target, and materialising provenance charges the source insertions in Z_v even when deduplication leaves Z unchanged.

9.4 Inverted channel resolution

Proposition 126 (inversion)**.** *Building the candidate lists by pushing from units to the packages their scope-sets resolve to yields, for each p, the same list as scanning the window for directives naming p, provided units are pushed in* SPEC *§11.6 precedence order. It costs $O(U+I+P)$ against the literal $O(P \cdot U)$; the incidence term cannot be omitted because those appends are output work.*

Proof. A package p appears in the pushed list of u exactly when u carries a `Channel` directive, $p \in \mathrm{scope}(u)$, and $c_u \in W^\circ_\sigma(p)$, which is the pulled form's membership condition read in the other direction, so the two lists have the same elements. They have the same order because the push visits units in the order the pull would sort them into. The push performs one pass over units, each contributing $|\mathrm{scope}(u)|$ appends against scope resolutions SPEC §13.4 has already paid for, and one flat pass over packages. □

Two obligations come with the inversion and neither is optional. Units must be pushed in SPEC §11.6 order, or the first-match-wins loop below silently picks a different directive, a determinism bug that surfaces only when one package is named by two commits. And the `W186` count is over the whole candidate list, not the prefix examined before the winner was found, so the list must be complete before it is read; an implementation that fuses the two passes and evaluates a package as soon as its first candidate arrives loses `W186`.

9.5 Deterministic order and failure closure

Proposition 127 (the publish heap)**.** *The order of Definition 44 is computable in $O(E + P \log P)$ with a comparison heap over the ready set, against $O(P^2 + E)$ for scanning the ready set for its least name at every step. No comparison-based structure improves the bound for unsorted names: a workspace with P unsorted names and no edges makes the sequence a sort of its names.*

Proof. Kahn's algorithm [11] visits each package and relaxes each edge once; with the zero-indegree packages kept in a heap keyed by name, each package enters and leaves the heap once at $O(\log P)$ and each relaxation is $O(1)$, giving $O(E + P \log P)$. A linear scan of the ready set at each of P steps costs $O(P)$ each. For the lower bound, with no edges every package is ready at once and Definition 44 returns the names in byte-wise order, so any method that decides the order by comparisons performs a comparison sort of P keys, which needs $\Omega(P \log P)$ comparisons. This is a worst-case statement for unsorted comparison keys and not an instance-optimality result: where name ranks or sorted names are already provided, the reduction proves no new lower bound, and any other structure must provably return the same least element at every step. □

The historical bound of $O(P+E)$ for an arbitrary topological order predates the byte-wise tie-break SPEC §19.2 now requires; Proposition 45 proves that the required order is well defined and machine independent, not that it is linear.

Proposition 128 (the blocking closure)**.** *The blocked set of Definition 50 is computable in $O(P + E)$ traversal work per run, plus one read of each planned package's recorded causes, by adding each failure to an incremental multi-source reverse traversal over `publish.blockingKinds` on the full workspace graph and marking each newly reached consumer once. The literal `run` of* SPEC *§19.3, which tests each planned package by a fresh transitive walk, is $O(P \cdot (P + E))$ whatever the number of failures, and a walk from each failure is $O(F \cdot (P + E))$.*

Proof. A package transitively depends on a failed or blocked package exactly when it transitively depends on a failed one, since a blocked package itself depends on a failed one; so the set `bad` of `run` is non-empty for d exactly when d is reached from F over the reversed edges. Marking a package on first reach and never expanding it twice visits each package and each reversed edge at most once across all failures, which is $O(P+E)$ however the failures interleave with publication. Whether a reached package is blocked or proceeds is then the own-cause test, which reads the causes the plan already retains for that package and excludes those from failed or blocked providers; packages are examined in publish order, so every provider's status is known when its dependent is examined. Recording every failed ancestor or path adds output-sized work, and the first cause and the `W194` order still follow the deterministic publish and failure order rather than queue or hash iteration order. □

Proposition 129 (build readiness in linear time)**.** *The build-stage edges of Definition 48 are computable in $O(P + E)$: give every package that does not build a pass-through node, direct one relay edge from provider to consumer for every dependency edge not declared `none`, and take the build constraints as reachability in that relay graph. A search per building pair through the packages between them is $O(P \cdot (P+E))$. After transitive reduction at most two task edges per dependency edge survive.*

Proof. A path in the relay graph from P's node to C's node is exactly a dependency path from C to P with no `none` edge, since relay edges exist for precisely the edges not declared

`none` and pass-through nodes relay without constraint; so build(P) precedes build(C) in the relay graph exactly as Definition 47 requires. Building the relay graph touches each package and each edge once. Scheduling then follows the relay edges, which the executor already does for the publish stage, so no per-pair search is needed. Alternatively a memoised index of the nearest building packages each package reaches without a `none` edge is built in one visit per package in reverse topological order, at the cost of the set unions it stores, which are output-sized. For the last claim, each dependency edge $C \to P$ contributes at most one build-stage constraint (build-to-build or publish-to-build, by its relation) and one publish-stage constraint, and a constraint carried through intermediate packages is implied by the relayed edges, so the reduction keeps at most two per dependency edge. □

9.6 The profile rows

The polyrepository rows are sums over repositories and deliberately not globally linear in fleet history: distinct boundaries can require distinct reachability sets, output can contain $P \cdot U$ scope incidences, and propagation can still materialise Z contributions. Snapshots: the control gitlink transitions are indexed in one pass per fixed control snapshot and each source snapshot is walked once, $O(G + \sum_q (H_q + A_q))$, never once per consumer. Windows: one marker pass per repository, $O(\sum_q (H_q + A_q)\,\mathrm{bw}(m_q) + I_w)$, with the boundaries of Definition 75 counted per repository in K_q as revisions and not as (stable, fresh) pairs. Publication input closure: the graph augmented with shared-version-group membership can contain cycles, so its strongly connected components are condensed in $O(P+E+V)$ and visited in reverse topological order with one bitset union per condensed edge, $O((P + E + V)\lceil Q/w \rceil)$; a dense representation costs $O(P\lceil Q/w \rceil)$ words and an expansion into name lists up to $O(P \cdot Q)$. Link evidence and settlement: the link tree is rooted once in $O(Q)$, each (repository, revision) tree is read at most once per plan, and a package settles with one commit per recording repository, at most $Q - 1$ and two in a star. Record comparison: one inventory per repository under the lock, $O(T)$ each (Definition 42). Every index and cache key preserves repository identity, no cache synthesises ancestry between repositories, and ancestry and walk caches are bounded by configured memory or an eviction policy over all their tiers, because a cache of every queried pair grows quadratically in H_q; recomputation after eviction preserves semantics and may lose the cached time bound, a trade-off an implementation states.

The execution rows count tasks and nodes. Task graph and readiness: a counter of unfinished predecessors per task, decremented once per completed predecessor, makes readiness $O(T_k + D_k)$ over the run, where rescanning every task after each completion is $O(T_k \cdot (T_k + D_k))$. Placement: choosing a compatible free node per task is $O(T_k \cdot W_n)$, and W_n is small enough that a free list per compatibility class is rarely worth its bookkeeping. Output transfer: one transfer per distinct (output set, node) pair bounds the count by $P_o \cdot W_n$ where a transfer per consumer task would reach $T_k \cdot P_o$, and verified bytes are not fetched again (Definition 69). Coordination polling: the branch-name prefix of Definition 68 lets a node list only the refs addressed to it and fetch once per changed tip, where listing every live ref is $O(L_r)$ per tick, and the refs of dead runs tax every tick until removed. None of these rows bounds transfer bytes or command durations, which Section 5 treats separately.

9.7 Space

Memory is counted in machine words, except input and output byte buffers, with indexed inputs retained.

Representation	Retained or auxiliary space	Qualification
Package and history adjacency	$O(P + E + H + A)$	input indexes; not repeated per package
Parsed messages and resolved incidences	$O(N + U + I)$	byte buffers plus unit and incidence records; cached resolution has its own key
Dense history marker rows	$O(H\lceil m/w \rceil)$	plus traversal state; zero markers need no rows
Explicit package windows	$O(I_w)$	optional materialisation; shared boundary indexes can avoid it
Cancel masks	$O(P_c\lceil \text{cancels}/w \rceil)$	plus scoped-package lookup and diagnostic state
One propagation BFS	$O(P)$ auxiliary	on top of shared adjacency; retained walk caches have a separate budget
Direct channel candidates	$O(P + I)$ as lists, or $O(P)$ summaries	summaries require the deferred-output rule of SPEC §13.8; diagnostics and provenance are additional
Contribution and provenance records	$O(Z)$	streaming reduces retention only where later phases and output permit
Heap publish order, blocking membership	$O(P)$ auxiliary	shared graph excluded; all-cause or path diagnostics add output-sized storage
Publication input closure bitsets	$O(P\lceil Q/w \rceil)$	shared component sets may reduce storage; expanded lists cost up to $O(P \cdot Q)$
Link evidence cache	retained pins plus keys	B counts trees, not their entries or bytes; dense hub revisions can be large

No implementation should add every alternative in this ledger by default. Peak simultaneously live allocations, construction scratch space, immutable input buffers, retained output and cache keys are what count; batching the marker computation limits scratch space but does not bound retained marker columns, and a transposed per-marker layout has the same $H \cdot m$ logical bits as the row layout with neither universally better. Linear scanning is worst-case time-optimal when every input byte must be inspected, explicit output needs time at least its size, a single adjacency traversal and the blocking closure have tight linear bounds, and the publish heap matches the comparison-sorting lower bound for unsorted names. These are conditional, phase-specific statements: neither $\Omega(Z)$ output work nor a dense $H \times m$ or $P \times Q$ representation proves that every instance requires that representation or that the complete planner is jointly time- and space-optimal.

9.8 Parsing

Lexical parsing and resolution have distinct cache keys. Lexing is pure in the repository identity, immutable record identity, parser and schema identity and parsing configuration. Scope and changed-path resolution additionally depends on the workspace package and path index, graph and resolution configuration. A lexical cache hit therefore does not cache R or I, and record immutability does not imply a whole-plan cache. File ownership is a longest-prefix probe of the path's own ancestor directories, a literal scope term is one map lookup, a glob whose only $*$ is its last byte selects a contiguous range of the sorted names in $O(\log P)$ plus its matches, and a scope-set depends on its commit only through . and, under SPEC §27, through the owning repository, so it may be resolved once per distinct text; none of this changes I, which is output. Vectors 82h to 82j exercise direct-channel resolution, inherited-channel provenance and per-unit admission; they do not exhaustively validate these cache keys.

Proposition 130 (linear parsing, SPEC §20.7)**.** *A conforming parser reads a message of n bytes in $O(n)$ time and $O(1)$ auxiliary space beyond the parsed result, with no input causing superlinear behaviour.*

Proof sketch. SPEC §20 specifies every step as a forward index scan over the byte array with no backtracking: separator detection compares a fixed token at line starts; header parsing advances a cursor through type, scope-set, inline directives, `!` and `:` , each transition determined by the current byte and a bounded amount of already-scanned state; footer detection tests each line against a fixed set of key shapes. Each byte is examined a bounded number of times, and the working state is a fixed number of indices and flags. No construct requires re-examining a prefix, which is the property that distinguishes it from a backtracking regular-expression engine and is why SPEC §18.3 can promise bounded work on untrusted commit messages. □

This is the one bound in the table stated for the parser rather than the planner, and it is here because SPEC §18.1's threat model includes commit messages authored by anyone who can open a pull request. A superlinear parser would make that a denial-of-service surface in CI; Proposition 130 is why the specification prescribes an index scan and relegates regular expressions to an appendix.

9.9 What none of this may change

Every representation above is internal. The plan, the diagnostics and their order must be identical to the literal reading (SPEC §17.2, Theorem 106), and an implementation that trades a different plan for speed does not conform however fast it is. Appendix B.7a of SPEC tests exactly the transformations that are almost equivalence-preserving, a hoist that collapses a multi-channel source set, an inversion that loses SPEC §11.6 order, a fused pass that miscounts `W186`, a skipped channel pass that leaves a channel unset; it is not exhaustive, and an optimisation it does not name is still the implementer's to justify.

10 Proof obligations

This section keeps two ledgers. The first, Section 10.1, holds the obligations open against the current specification, CCME 3.1.0-rc.2 at repository commit `252216ba`, and against the reference implementation at the same commit: places where the normative text contradicts itself, where the text states a boundary of a guarantee rather than a rule, and where the engine departs from the text. Each states the claim, a minimal scenario, the consequence for the results of this report, the repair or the recorded status, and the case and test that pin it. They are ordered by severity, the silent loss of a release first and diagnostic metadata last. The second ledger, Section 10.2, is the audit of CCME 1.0.0 at `713f1a0b` that this report was first written for. Its six obligations are provenance: the released 2.0.0 text incorporated their repairs and the current text keeps them, so they are reproduced unchanged.

Every SPEC reference in Section 10.1 is to the current text. In Section 10.2, except where a paragraph explicitly names CCME 2.0.0 or a current theorem, every numbered reference and quoted guarantee refers to the 1.0.0 snapshot, and present tense describes the historical rule being executed.

10.1 Current obligations

The sources consulted are SPEC itself, its dated design history and changelog, the release notes of the engine, the integration test plan, and the test sources of the engine and of its integration suite. Where the design history names a departure, the obligation quotes it; where it does not, the obligation says what the code does and where.

10.1.1 The owed window and `E201` are specified but not implemented

Proof obligation 131 (a consumer that sits out the run in which its provider delivers is stranded)**.** SPEC §13.4a admits a bump-axis contribution for a target until the source has *delivered* it. SPEC §13.3 adds, for every edge $D \to P$ where D has a baseline, an owed window `owedWindow(P, D)` to the union of windows parsed in SPEC §13.4, so that a debt survives the release of the provider; and SPEC §19.3 refuses with `E201` a run that would release a provider at the baseline commit of a consumer it still owes unless the consumer releases after it in the same run. The reference implementation implements the delivery test and the reconciliation of a proceeding consumer, and neither the owed window nor `E201`.

Source. The design history entry of 22 September 2026, "Delivery discharges a propagated contribution", ends: "The dispat engine implements the delivery admission and the reconciliation of a proceeding consumer; the owed windows and `E201` are not yet implemented, and a consumer that sits out the run in which its provider releases the owed commit is therefore still stranded in that engine, which its release notes list as a departure." The engine's sources at the same commit contain no `E201` and no owed window; `admission.go` carries `isDelivered` and `owedSources`, the delivery half. The release notes at this commit do not yet carry the departure: `services/dispat/CHANGELOG.md` ends at `v1.11.0-rc.3` of 20 September, two days before the entry, and `announce/rc/notes.yaml` does not mention it.

Scenario. One commit C_1 carries `feat(core)^: x` and `feat(cli): y`, with `cli` consuming `core`. Run 1: `core` fails to publish; `cli` has a cause of its own, proceeds at `2.1.0` with its manifest naming `core`'s baseline `1.4.0`, and is tagged at C_1 (SPEC §19.3; vector 80d). A commit C_2 lands. Run 2 releases `core@1.5.0` at C_2 in a run `cli` sits out, held or deselected. Now $W(\texttt{core}) = \emptyset$ and $W(\texttt{cli}) = \{C_2\}$: C_1 lies in no window of either package, so the unit is never parsed, `cli` is never planned, and its manifest names `core@1.4.0` for ever. The delivery test would answer correctly if it were asked, since `core`'s release commit C_2 is not reached by `cli`'s baseline C_1; the owed window is what asks it. Vector 80d states the case in its last sentence: "`core` published at a later commit in a run `cli` sat out leaves `C1` in `cli`'s owed window (§13.3), so the next run plans `cli`'s catch-up."

Consequence. In the specification, Theorem 87 holds as stated: the owed window keeps the contribution admissible until `cli` releases at or after `core`'s release carrying C_1. In the engine, the theorem holds only while some pending window still holds the commit, which is every case but this one. The loss is silent in the plan. The tag-level screen of Proposition 114 does fire, because `core`'s stable commit is not an ancestor of `cli`'s baseline, so a scheduled SPEC §13.7b audit reports `W195`; nothing releases it.

Repair. Add the owed windows to the parsed union and `E201` to the publish protocol, as written. Until then the operator-side remedy is the one SPEC §19.3 states for the refused run: release the consumer in the same run as the provider, or pin it with an explicit `Release-As`.

Pin. Vector 80d, last two cases. The tests `TestAdmissionCatchesUpAConsumerThatProceededOnItsOwn` and `TestAdmissionCatchesUpAConsumerThatOvertookAHeldProvider` of the integration suite and `TestAdmissionCatchesUpAConsumerThatOvertookItsProvider` of `internal/plan` pin the delivery half at an unchanged h; no test in either suite exercises the run the consumer sits out.

10.1.2 Two releases on one commit: a stated boundary of completeness

Proof obligation 132 (the same-commit failure case of `E201`)**.** SPEC §19.3 states: "If the consumer is published after the provider and fails, the same state arises, and `E201` reports it with its remedy: an explicit `Release-As: <version>` on the consumer (§8.6), because no later plan can compute the debt." The protocol reports this state; it does not resolve it. Completeness (Theorem 87) therefore has a boundary the specification itself states.

Scenario. The state of proof obligation 131 after run 1, and run 2 at the same $h = C_1$ with both packages in the plan: `core@1.5.0` publishes and is tagged at C_1, then `cli@2.1.1` fails. Both `core`'s release commit and `cli`'s baseline are C_1. By SPEC §13.4a, `delivered(core, C1, cli)` asks whether some release commit t of `core` carrying C_1 lies in the reach of `cli`'s baseline commit; $t = C_1$ does, so nothing is owed, although `cli` still names `core@1.4.0`. SPEC §13.4a says why no rule over tags can do better: "Two releases on one commit have no order (§13.7b), so `delivered` cannot tell a target that released at commit `t` after its source from one that released there in an earlier run whose source then released at `t` too."

Consequence. Theorem 87 discharges a contribution when d releases at a commit whose ancestry contains c_u. Ancestry is a partial order on commits and says nothing about two records on one commit, so the theorem is exact only under the hypothesis that a provider is never released at the baseline commit of a consumer it owes unless that consumer releases after it in the same run and succeeds. SPEC §19.3 enforces the first half before publication and reports the second with a remedy. The closing remark of Proposition 114, that this case is prevented rather than detected, holds with the qualification that the prevention is a refusal and the residue is a report.

Status. A boundary, stated. In the engine it is unreported, because `E201` is not implemented (proof obligation 131); the state arises exactly as described and leaves no diagnostic. Vector 80d names the refused run; no vector or test exercises the failure after the provider's publication.

10.1.3 The version summary still contradicts the graduation rule

Proof obligation 133 (SPEC §13.9 does not preserve SPEC §11.5's train target)**.** SPEC §11.5 states the graduation version as "`applyBump(S, E)`, the same `target` as §11.4, with no prerelease suffix", where SPEC §11.4 defines E as "the effective bump accumulated over the pending window (§13.3)", the train window $W(P)$. SPEC §13.9's stable branch instead reads "`next = applyBump(stableBaseline(P), effective(P))`", and `effective(P)` is `max(direct(P), propagated(P))` (SPEC §13.7) with `direct(P)` taken "over surviving tuples for `P` whose commit is in `Wfresh(P)`" (SPEC §13.6). On a channel-only graduation the fresh effective bump is `none` although an older minor or major change established the train; read literally, the summary loses the aggregate the specific rule requires. This report follows SPEC §11.5; the summary should delegate to it or carry a separately named train target.

What changed since 2.0.0. The obligation as first recorded had a second half: SPEC §11.5 said `E185` fires when the target is not greater than the "baseline core", which would have refused every ordinary graduation. The current text resolves it: "Graduation never lowers a version: if `target` is lower than the core of the baseline, `E185` is raised. A `target` equal to that core is the ordinary graduation, because `1.3.0` ranks above `1.3.0-rc.1`." The same section now also carries the channel-entry patch on the way out of a train (`W204` on graduation, vector 39b), which removes the dead end in which a train entered by the patch could not graduate. Both are implemented. The first half of the obligation stands: SPEC §13.6's own second paragraph says that "§11.4 then uses the full `W(P)` to compute that admitted prerelease's aggregate target", and the window table of SPEC §13.3 assigns the aggregate to $W(P)$, but SPEC §13.9 is the section an implementer transcribes and it names neither.

Consequence. Theorem 89 proves that fixed version inputs give the same result on retry; it does not decide between two formulas. Definition 21 follows SPEC §11.5. A second implementation transcribing SPEC §13.9 alone graduates a channel-only train to its stable baseline, which SPEC §13.9's own last guard then refuses as `E195`.

Pin. The last row of SPEC §11.4's table ("`1.3.0-rc.1`; nothing new, `E` is still the window's `minor`; `stable`; `1.3.0`; graduation") and edge case 61 of SPEC §15.5: "Graduation with no pending bumps: publishes the accumulated `target`." The engine follows the specific rule: `computeVersion` in `plan.go` applies the bump held over the whole train to the stable baseline, and `TestTrainGraduationCountsPublishedWork` asserts that "the bump of the already-published breaking change must still be in force even though it alone would not have re-released the train"; `TestGraduationUpdatesSpanTheTrain` and `TestTrainEnteredByTheEntryPatchGraduates` pin the record and the patch.

10.1.4 One edge-kind set orders, blocks and refuses cycles

Proof obligation 134 (no `orderKinds` or `blockingKinds` in the engine)**.** SPEC §13.1 requires the graph to be acyclic "over the edge kinds of `propagation.kinds` and `publish.orderKinds`", and adds that a cycle "existing only through [`devDependencies`] is not a cycle for this purpose and does not trigger `E200`, which is the common and legitimate case". SPEC §19.2 orders over `publish.orderKinds`, "which includes `optionalDependencies`"; SPEC §19.3 blocks over `publish.blockingKinds`, "which excludes `optionalDependencies`". The engine has neither key. Every declared edge, whatever its kind, enters the one graph that the cycle check, the publish order and the failure closure read; only propagation honours a kind set.

Where. `config.go` builds one `model.Dependency` per declared edge, carrying its kind and dropping none; `loadWorkspace` in `plan.go` adds each to the graph and reports `E200` on any cycle of it; the publish order is that graph's topological order; and the blocking closure of `release/executor.go` reads a per-relation boolean, `isBlocking`, not the edge's kind. The kind set of `parser.propagation.kinds` is consulted by the propagation walk alone, which is what the integration test plan states as the purpose of `TestPropagationKindsSelectTheEdgesTraversed`. Neither the design history, the changelog, the release notes nor the documentation records the difference; the documentation of `propagation.kinds` describes the walk only.

Scenario. Two packages that list each other under `devDependencies`, a test fixture depending back on the package it exercises. The specification accepts the workspace (vector 100a, edge case 99a); the engine refuses it with `E200` before any plan. Separately, `docs-site` depending on `ui` under `optionalDependencies` with `ui`'s publish failing: the specification publishes `docs-site` (vector 79, edge case 100); the engine blocks it with `W194` unless its relation declares `isBlocking: false`.

Consequence. The departure is on the safe side. The engine's graph is a superset of the one SPEC §19.2 orders, so any topological order of it is an order the specification accepts and Proposition 45 is unaffected; the blocked set of Definition 50 is computed over a superset of `blockingKinds`, so it is a superset of the specified one and the $F \cup B$ of Theorem 107 is larger, never smaller. Nothing the specification forbids is published. What is lost is admission: a workspace with a development-only cycle cannot release at all, and an optional dependent waits for a provider it does not need.

Repair. Either implement the two keys with the specified defaults, or record the departure where the design history records the others and restrict `E200` to the runtime kinds. The second is a one-line change to the graph the cycle check reads.

Pin. Vectors 79 and 100a and edge cases 99a and 100 are the conformance cases; none has a test, and both vectors fail against the engine as it stands. `TestFatalDependencyCycle` of the integration suite and `TestCycleIsRejected` of `internal/plan` pin `E200` over runtime edges; `TestExecutionDevDependencyOutputReachesConsumer` shows a `devDependencies` edge ordering builds under SPEC §28, which is what vector 22 of SPEC §28.8 asks.

10.1.5 The fleet recorder rejects a retry that annotates the same commit

Proof obligation 135 (the create-only rule's exception is implemented on one of two paths)**.** SPEC §19.1 states: "A release tag is created, never replaced. Writing it to the authoritative store MUST fail when the store already holds that tag name, unless the stored tag names the same commit for the same package and version, which is the retry of an uncertain write (§19.4)." The single-repository recorder implements both halves; the fleet recorder of SPEC §27 implements the first and not the exception.

Where. `gitx/records.go` leases every record on the name being absent and, when the store refuses, reads the stored tag back peeled: the same commit is "this run's own write arriving twice and is a success", another commit is a published record left where it is (`E221`). Its inventory compares peeled lines because "two runs annotating the same commit write different tag objects, and a comparison on those would call one release two". The fleet recorder's `PushRelease` in `gitx/polyrepo_write.go` pushes each source tag with a plain unforced `git push`, which Git rejects whenever the remote holds a different tag object under that name, the same commit included; the rejection surfaces as `E335` after the publication.

Scenario. One run's tag push is applied by the remote and its acknowledgement is lost, or two checkouts of one source repository record the same version at the same head in turn. The next push of that name carries a new annotation of the same commit. In a single repository the push is reported as `W223`, "tag already exists on the remote at this release's commit,

skipped"; in a fleet it is refused, the run reports `E335`, and the dependent legs are blocked (SPEC §27.7).

Consequence. Nothing incorrect is recorded: the refused write is the identity the store already holds, and the next run reads it as released. The cost is a failed run and blocked consumers for a retry the specification defines as successful. Corollary 93 is unaffected, since the ledger holds one record either way.

Status. Open for the fleet path; resolved for the single-repository path by the design history entry of 21 September 2026, "Record authority under the lock". Neither the design history, the changelog nor the test plan records the fleet path's difference. `TestReleaseRecordsCreateOnlyPush` pins the single-repository behaviour ("the same commit is the retry of a lost write and succeeds"); `TestReleaseRecordsInAComposedWorkspace` pins the fleet's comparison under the lock, not its push. Cases: SPEC §28.8 vectors 27 and 28; edge case 76a.

10.1.6 The failure category travels only from a distributed lock loss

Proof obligation 136 (the class of SPEC §28.9 is attached at the new sites only)**.** SPEC §28.9 requires implementations to "expose stable machine-readable categories, named by the identifier each row states". The design history entry of 22 September 2026 states the engine's position: "the existing `E220` to `E222`, `E335` and `E336` carry `native-recording-or-lock` as their class; the field is attached where a distributed run loses its lock, and not yet at the older recording and lock sites, which is a departure to be closed by attaching it there."

Consequence. A reader that switches on the class, which is what SPEC §28.9 designs the class for, sees a lost lock in a distributed run and not the same lost lock in a local one, nor a refused record. No release outcome depends on it. `execution/codes.go` names the mapping in one place, so the repair is to emit the field at the five older sites.

Pin. Vector 14 of SPEC §28.8 and `TestExecutionLockLossStopsNewEffects` of the integration suite cover the site that carries the field; row EXE-12 of the requirement matrix asserts `E226` with its `execution-authority` class. No test asserts the class on a local `E221` or `E336`.

10.1.7 Profile departures and inapplicable vectors

Proof obligation 137 (what the engine records as not implemented or done otherwise)**.** The design history entry of 22 September 2026 lists, beside proof obligation 136, the remaining departures of the engine from SPEC §28 and the vectors that do not apply to it. They are recorded here so that the table of Section 11 reads against them.

- "Not implemented, so the conditional vectors do not apply: reuse of verified outputs across runs (vector 20) and the rollback profile of section 26 (vector 26)." SPEC §28.8 states the rule the entry relies on: a vector whose condition does not hold for an implementation "is neither satisfied nor violated". `TestExecutionOldRunReceiptGrantsNothing` and `TestExecutionTaskReceiptDischargesNothing` pin the halves of those vectors that hold regardless.
- "Departures from the profile as written: the platform check of section 28.2 runs before dispatch for the packages a run releases and, for a prepared provider, at its placement,

where an unsatisfiable platform fails the provider's consumers." SPEC §28.2 asks that unsupported target platforms "fail preflight". `TestExecutionPreparedProviderObeysItsOwnPlatforms` and `TestExecutionOrchestratorOnlyBuildIsNotHeldToTheWorkersPlatforms` pin the engine's timing.

- SPEC §28.4 "now also permits a durable mailbox ref or a history-free snapshot, and neither is implemented". These are permissions, not requirements; the entry records the cost of the mode in use, a first transport of a whole history.
- The specification's own header keeps the VCS adapters of SPEC §25 and the explicit rollback of SPEC §26 "pending implementation". The forward-publish projection this report models is unaffected; SPEC §13.7c says its guarantees are claims about that projection.

10.1.8 Textual defects noticed in passing

Four defects of the text, none of which changes a rule, are recorded for the specification's owner because they affect citation. Appendix B numbers two vectors "82c": the consumer that gets ahead of a held provider (B.7) and the hoisting of `resolvableBy` over a mixed source set (B.7a); this report cites them as "82c (B.7)" and "82c (B.7a)". In SPEC §19.3 the bullets "continue publishing packages that are not blocked" and "not release a package at the baseline commit of a consumer it still owes" are run into the preceding bullet's line rather than set as items. The design history says the release notes list the departure of proof obligation 131, and at this commit they do not. And the window table of SPEC §13.3 cites SPEC §13.6 as a consumer of the aggregate train window, while the formula of SPEC §13.6 reads the undelivered one and only its prose defers the aggregate to SPEC §11.4, the seam proof obligation 133 sits on.

10.2 Historical CCME 1.0.0 obligations

Formalising CCME 1.0.0 at `713f1a0b` exposed six failure classes. Each obligation below states the claim, a minimal counterexample, the consequences, a repair, and a conformance vector. Three concern SPEC §13.7c, one SPEC §13.7b, one SPEC §9.3, and one the optimisation advice of SPEC §13.11.

None is a defect in the reference implementation. In two cases (proof obligations 138 and 140), the implementation already uses a mechanism absent from the specification. A second implementation following only the written contract would therefore be incorrect. Source references below are relative to `services/dispat/internal/plan/` and cite the line numbers of the released `services/dispat/v1.7.2` tag. Those sources have since moved: at `252216ba`, `containedInBaseline` is at `plan.go` line 2507, `Plan.StaleSources` at line 1280, and the per-unit traversal of `propagate.go` is served from a walk cache (line 76) whose composition is checked against the literal walk (Section 11).

Released CCME 2.0.0 resolves them as follows: proof obligation 138 by W_{fresh}; proof obligations 139 and 142 by the full retry invariant and fixed-state wording; proof obligation 140 by applying channel eligibility in the upward audit; proof obligation 141 by stating axis independence in the bump-to-channel direction only; and proof obligation 143 by retaining each unit's own source exclusion when bucketed. The counterexamples remain regression evidence for those normative conditions. The regression vectors named below as "the released 2.0.0 form", 133 to 138, are in the current text the edge cases 133 to 138 of SPEC §15.7, one row each. Except where a paragraph explicitly names CCME 2.0.0 or a current theorem, every numbered SPEC § reference and quoted guarantee in this subsection refers to the CCME 1.0.0 snapshot at `713f1a0b`; present tense describes the historical rule being executed, not the

released 2.0.0 text.

10.2.1 The prerelease window never discharges

Proof obligation 138 (the 1.0.0 G4/G6 claims fail on a prerelease train)**.** Under the 1.0.0 single-window rule, ledger discharge and convergence fail after a successful prerelease publish. Current Theorems 92 and 98 use W_{fresh} and do not make that claim.

Counterexample (bump axis). This is SPEC vector 95b, run twice. Baselines `core@1.5.0-beta.0` and `cli@2.0.1-beta.0`, both with an older stable baseline; `cli` depends on `core`; one commit `fix(core)^: x`.

Run 1 plans `core@1.5.0-beta.1` and `cli@2.0.1-beta.1` and publishes both. Run 2, at the same h: by Lemma 78 neither window changed, because Definition 9 measures from the last *stable* tag and no stable tag was written. Under the 1.0.0 direct and propagated rules, `core` and `cli` each receive `patch` again, both packages are planned again, and SPEC §11.4 increments the counters to `beta.2`. Run 3 gives `beta.3`. The plan never empties, and `E199`, the convergence check of SPEC §19.6, which admits no exclusion but held packages, fires after the first fully successful run of any prerelease train.

Counterexample (channel axis). Theorem 102 says the channel a package arrived on is not proposed for it again. It does not say nothing is proposed. Let p have stable baseline `1.2.0` and a window containing an older `release(p)%rc` and a newer `release(p)%beta`. Run 1: SPEC §11.6 precedence gives the newer directive, p is released `1.2.1-beta.0` (`W186` noting the conflict), and the operator's stated intent is honoured. Run 2: the window is unchanged, so both directives are still candidates; `%beta` now proposes nothing (`W199`, Theorem 102) and the *older* `%rc`, which SPEC §11.6 had ruled had lost, wins by default and p is released `1.2.1-rc.0`. Run 3: `%rc` is now inert and `%beta` proposes `beta` again, computing `1.2.1-beta.0`, which does not exceed the baseline `1.2.1-rc.0`, so the run stops with `E195`. A repository that did nothing wrong is wedged, having published a version onto a channel its newest directive did not ask for.

What broke in 1.0.0. Its G4 (no double release), G6 (convergence), and G7 channel-settling claim (the channel axis settles) all fail, and with them SPEC §17.2's idempotency clause and SPEC §19.6's convergence check, for every package on a prerelease train, which is to say for the entire feature the prerelease flow exists to support.

The repair. The window answers one question, what has p not yet released, and SPEC §13.3 correctly measures it from the stable tag because SPEC §11.4 recomputes a train's target over the whole train. The missing distinction is that the same set is then used to answer a second question: is anything still *owed*? Separate them. Define the *fresh window*

$$W^{\circ}_{\sigma}(p) \;=\; W_{\sigma}(p) \setminus \mathrm{reach}(\mathrm{basec}_{\sigma}(p)),$$

which for a package whose baseline is stable equals $W_{\sigma}(p)$, and for one on a train excludes the commits its own prereleases already published. Then:

(R1) Definition 19 discharges on $W^{\circ}_{\sigma}(p)$, so a package whose prerelease shipped the work counts as a source (which SPEC §13.4a already intends: the artefact is public);

(R2) plan membership (Definition 21) requires a contribution from a commit in $W^{\circ}_{\sigma}(p)$, while the version computation (SPEC §11.4) keeps reading $W_{\sigma}(p)$, so the train's target still accumulates over the whole train;

(R3) candidate collection in Definitions 33 and 34 is restricted to $W^{\circ}_{\sigma}(p)$, which is what stops the superseded directive of the second counterexample from voting again.

With (R1)–(R3), Lemma 78 strengthens to $W^{\circ}_{\sigma'}(p) = \emptyset$ for *every* released p, prerelease or not, because $\mathrm{basec}_{\sigma'}(p) = h$. Fresh discharge repairs the prerelease counterexample. Current Theorems 92, 94 and 98 separately state the full retry-invariant conditions needed for persistence, non-widening, and bounded convergence; (R1)–(R3) alone do not imply them.

Evidence. The reference implementation has exactly this predicate, under a different name and undocumented in SPEC: `containedInBaseline` (`plan.go`, lines 1836–1855) reports whether a commit has already been published by the package's baseline tag, and its callers implement (R1) in `sourcePackages`, (R2) by gating `NewWork` on it, and (R3) in both `resolveChannels` and `propagateChannels`. Its comment states the consequence the specification does not: without it the engine "would re-warn ... on every run until graduation".

Regression vectors. Vector 133 is the released 2.0.0 form of the bump-axis case. Historically, (i) vector 95b, run twice: the second run's plan must be empty. (ii) The two-directive window above, run three times: run 2 must be empty, and `E195` must not arise.

10.2.2 Window-scoped suppression lapses, so the admitted set can grow

Proof obligation 139 (the 1.0.0 G5 claim lacks suppression stability)**.** The 1.0.0 proof assumed "same sources, same depth, same graph at h", although window-scoped suppression can change the source set. CCME 2.0.0 makes the needed stability part of the full retry invariant used by current Theorem 94.

The root cause is a design decision taken three times, in three sections, each time for a good local reason: suppression is scoped to the window. A `cancel` stops counting once the package releases (SPEC §13.4a); a hold does the same (SPEC §13.4a); and a correction "reaches only undischarged work", so a `Deletes` is a no-op for a package that has released its target (SPEC §7.4.2, Remark 17). Each rule is right on its own terms. Together they mean that the set of units a package acts as a propagation source for is *monotone in the state*, which is precisely what **G5** says it is not.

Counterexample (cancellation). `cli` depends on `core`. Three commits: A carrying `feat(core)^: streaming reader`, then B carrying `cancel(core)`, then C carrying `fix(core): unrelated`, in that ancestry.

Run 1: $\mathrm{canc}(A, \texttt{core})$ holds and $A \in W_{\sigma_1}(\texttt{core})$, so `core` is not in $\mathrm{src}_{\sigma_1}(u_A)$ and u_A propagates to nothing; `core` releases `1.4.3` from C alone (SPEC vector 41). $\Pi(\sigma_1) = \{\texttt{core}\}$: `cli` is not planned.

Run 2: `core` is tagged at h, so $W_{\sigma_2}(\texttt{core}) = \emptyset$, so $\mathrm{disch}_{\sigma_2}(\texttt{core}, A)$ holds, so `core` re-enters $\mathrm{src}_{\sigma_2}(u_A)$ by the discharge disjunct of Definition 19. Now u_A propagates: $A \in W_{\sigma_2}(\texttt{cli})$, `cli` is not cancelled, and the bump is admitted. $\Pi(\sigma_2) = \{\texttt{cli}\} \not\subseteq \Pi(\sigma_1)$.

Counterexample (corrections). The same shape without any `cancel`. Let B instead carry `chore(core): drop it` with `Deletes: <A>`. In run 1 the correction discards A's record for `core`, so u_A has no source and reaches nothing; `core` still releases, on C. In run 2, W_{σ_2}(`core`) is empty, so by SPEC §7.4.2 the correction is a discharged no-op for `core` (`W209`), A's record stands again, and u_A propagates to `cli` exactly as before. This route is worth stating separately because `W209` reads as reassurance (the correction did not need to do anything) when it is in fact the moment the suppressed propagation returns.

What broke in 1.0.0. Its unrestricted G5 claim and the "nothing is added" step of its G6 proof failed. Theorem 98's conclusion survives here (`cli` releases in run 2 and run 3 is empty) but the variant argument does not, and the security reading of **G5** does: whether `cli` is released at all depends on whether `core`'s publish succeeded. Inducing a publish failure changes the set of packages a commit eventually releases, which is the inference SPEC §18.1 relies on **G5** to forbid.

The repair, and why the obvious one is wrong. The tempting fix is to make cancellation sticky, never letting a cancelled package become a source, by defining $\mathrm{disch}_\sigma(p, c) \wedge \neg\mathrm{canc}(c, p)$. It is wrong: it breaks SPEC vector 72, where `cancel(core)` lands *after* `core` published the work and the consumers' catch-up must still proceed. Ancestry cannot separate the two cases, since in both the cancel is in the history at h and in both `core` carries a tag at h. The difference is only whether the released version counted the unit, and that is not recoverable from the state.

The recommendation is therefore to restate the guarantee rather than change the algorithm: **G5** holds for every unit none of whose sources is suppressed at h, which is hypothesis (H2), and the widening in the remaining case is intended behaviour, since neither a `cancel` nor a `Deletes` touches code (SPEC §10.1, SPEC §7.4.2) and the work of A is in fact inside the published `core@1.4.3`. What the specification should not do is claim a subset property that the discharge rule of SPEC §13.4a deliberately violates, and SPEC §18.1 should not rest a security argument on the unrestricted form.

Evidence. The cancellation counterexample was run against the reference implementation, which follows SPEC §13.4a literally and reproduces it: with the history above and `core` tagged at h in the second state, the first plan releases `core` alone and the second releases the consumer as well, reporting it as a *catch-up from core*. The engine is doing what the specification says; it is **G5** that does not describe it.

Regression vector. Vector 134 is the released 2.0.0 form. In the historical pair, each history above is run twice: run 2 plans `cli`, the plan is documented as a superset of run 1's, and the `W193` naming `core` as the origin is present so that the widening is visible to whoever reads the plan rather than silent.

10.2.3 The staleness dual omits the resolvability test

Proof obligation 140 (SPEC §13.7b's two formulations disagree)**.** SPEC §13.7b states that `staleSources(D)` "is non-empty exactly when §9.2 assigns `D` a non-`none` propagated bump", and that disagreement is an implementation bug. The procedure tests scope, depth, window, and cancellation, but has no counterpart to Item (A6).

Counterexample. SPEC vector 95 (also edge case 60m): `feat(core)^%beta: x` where `core` moves onto `beta` and its consumers stay on `stable`. SPEC §9.3a suppresses every consumer's bump with `W208`, so $\mathrm{prp}_\sigma(\texttt{ui}) = \texttt{none}$, and SPEC is explicit that "`feat(core)^%beta` releases `core` alone". But `staleSources(ui)` passes every guard it has and returns the row $(\texttt{core}, u, 1, \texttt{patch})$. The audit reports `ui` as behind a dependency for which no release is owed, and by SPEC §13.7b's own words the implementation is now buggy by definition.

The repair. Add the hoisted predicate of Proposition 125 to the procedure, immediately after the `cancelledFor` test:

```
srcChan = { channel[P] : P in sources }        # ğ9.3a, as in ğ9.2
if not ('stable' in srcChan or channel[D] in srcChan):  continue
```

With it, Theorem 111 holds unconditionally and its hypothesis can be dropped.

Evidence. The reference implementation cannot exhibit the disagreement, because `Plan.StaleSources` (`plan.go`, line 1217) returns the rows recorded by the downward traversal rather than re-deriving them from an independent upward walk. The duality is therefore true there by construction and untested; an implementation that follows SPEC §13.7b literally, as a second implementation would, disagrees on every prerelease workspace.

Regression vector. Vector 135 is the released 2.0.0 form; historically, vector 95 was extended: assert that `staleSources(ui)` is empty, and that the randomised down-versus-up cross-check SPEC §13.7b recommends is run on a workspace containing at least one prerelease train.

10.2.4 Axis independence is one-directional

Proof obligation 141 (the "vice versa" in **G8**)**.** SPEC §13.7c states **G8** as "suppressing a bump under SPEC §9.3a never suppresses a channel, and vice versa", and proves it by observing that the two passes share only $\mathrm{chan}_\sigma(\cdot)$, which phase 3 reads and never writes. That argument proves one direction only, and the other is false.

Counterexample. Let `core` and its consumer `app` both sit on `stable`, and let one commit carry `feat(core)^%beta++1`: the caret propagates a bump to `app` and the `++1` propagates the `beta` channel to it, which is exactly the pairing SPEC §9.3a prescribes for taking consumers along. Suppose `app`'s own window also contains `release(app)%rc`. By Definition 34 a direct directive beats every propagated one, so $\mathrm{chan}_\sigma(\texttt{app}) = \texttt{rc}$ and the propagated `beta` is discarded. Phase 3 then computes $\mathrm{sch}_\sigma(u) = \{\texttt{beta}\}$, finds $\mathrm{chan}_\sigma(\texttt{app}) = \texttt{rc} \notin \mathrm{sch}_\sigma(u)$ and no source on `stable`, and suppresses the bump with `W208`. Had the propagated channel been applied, `app` would have been on `beta` and the bump admitted. The suppression of a channel proposal caused the suppression of a bump.

The repair. State **G8** as Theorem 104 does: the channel assignment is computed by phases 1–2 and reads no phase-3 output, so no bump suppression can affect it. The dependency in the other direction is real, is the entire content of SPEC §9.3a, and should be documented as such rather than denied. The operational consequences SPEC §13.7c draws from **G8** are unaffected: they only ever use the direction that holds.

Regression vector. Vector 136 is the released 2.0.0 form. The historical workspace above asserts that `app` is released on `rc` with no propagated bump and a `W208`, and that the `W208` is a consequence of the channel precedence rule rather than of anything the caret did.

10.2.5 Inherited channels read a baseline that release moves

Proof obligation 142 (SPEC §9.3's replay-independence claim)**.** SPEC §9.3 states of the `inherit` resolution that "the result is deterministic, does not depend on which earlier run published the origin". The second half is false: resolution rule 2 reads $\mathrm{ch}(\mathrm{base}_\sigma(q))$, and the origin's own release moves it.

Counterexample. `app` depends on `core`; `core` sits at baseline `1.5.0-beta.0` over stable baseline `1.4.2`; `app` at `2.0.0`. Two commits: C_1 carrying `feat(core)^++1: x` (no explicit channel value, so the propagated channel is `inherit`) and C_2 carrying `release(core)%stable: graduate`.

Run 1: `core`'s own channel is `stable` (the direct directive), but the inherited value for `app` is $\mathrm{ch}(\mathrm{base}_{\sigma_1}(\texttt{core})) = \texttt{beta}$, so `app` is planned at `2.0.1-beta.0` while its provider graduates to `1.5.0`. Suppose `core` publishes and `app` fails. Run 2: $\mathrm{ch}(\mathrm{base}_{\sigma_2}(\texttt{core})) = \texttt{stable}$, the proposal is inert (`W199`), and `app` is planned at `2.0.1`. Both runs plan `app`, at different versions, with every completed release stable: the 1.0.0 cross-run version-stability claim fails. CCME 2.0.0 instead fixes inherited baseline-channel state as one component of its full retry invariant, as required by current Theorem 89. Run against the reference implementation, the two plans are exactly as computed here (`app`: `beta` → `stable`, `2.0.1-beta.0` → `2.0.1`).

What breaks, and what does not. Determinism (Theorem 106) is unaffected: each plan is still a pure function of its state. What fails is the *cross-run* half of the specification's sentence, and with it channel stability for unreleased dependents of a graduating origin. Note also that run 1's own output is operationally odd: a consumer is dragged onto a train in the same run that ends it. Both effects have one cause, phase 1 reading baselines so that it cannot depend on phase 2's output (SPEC §9.2); the price of that layering is that `inherit` answers with the origin's pre-graduation channel.

The repair. Scope the specification's sentence to a fixed state (determinism per run, which is what Theorem 106 proves), and state the drift explicitly. Resolving `inherit` against the origin's *resolved* channel instead would remove the drift but requires a fourth phase, which SPEC §9.2's no-circularity argument was written to avoid; the fresh window of proof obligation 138 does not remove it either, since the commit remains pending for the failed dependent. The honest fix is documentation plus the vector below.

Regression vector. Vector 137 is the released 2.0.0 form: the two commits above, two runs: run 2 must plan `app` at `2.0.1` on `stable`, and the plan should carry the `W199` that records why the run-1 proposal lapsed.

10.2.6 Bucketed traversal over-admits a unit's own sources

Proof obligation 143 (the 1.0.0 bucketing remedy has its own edge)**.** CCME 1.0.0 SPEC §13.11 warned that subtracting the merged source set from a bucketed traversal silently withholds bumps, and prescribes admitting "any node reached across at least one edge within

the depth bound". By Proposition 122 that prescription over-admits the set X: a package that is itself a source of the unit that reaches it.

Counterexample. One unit, `fix(core,cli)^major: x`, where `cli` depends on `core`. SPEC §9.2 seeds `seen` with both sources, so `cli` is not a propagation target and takes the unit's own `patch`. Under the merged rule, `cli` is reached from `core` across one edge inside the depth bound and takes the propagated `major` as well: a two-level difference in the published version, with no diagnostic.

The repair. Bucket the traversal, then re-decide the few targets that are themselves sources. Since the bucket key fixes the propagated bump, the only question for such a d is whether *some* unit in the bucket admits it, that is whether $\exists i : d \notin S_i \wedge \delta(S_i, d) \leq \ell$. The candidates are confined to S, which is small, and the per-unit source sets are already materialised, so the check is cheap. Equivalently, and more simply, exclude from bucketing any unit whose own source set contains a package reachable from that set within the depth bound.

Evidence. The reference implementation does not bucket: it traverses once per unit (`propagate.go`, line 401), so it is unaffected. The obligation is on the advice, which a performance-minded second implementation would follow.

Regression vector. Vector 138 is the released 2.0.0 form. Historically, SPEC Appendix B.7a, whose vectors test exactly this class of almost-equivalence, gains one: `fix(core,cli)^major` must publish `cli` as a patch, bucketed or not.

11 Conformance

Each result below is testable, and most are tested. The table maps every result to the specification section it formalises, the conformance cases that exercise it, and the test that stands behind it. Cases are numbered as the specification numbers them: a bare number is a vector of SPEC Appendix B (B.7 for partial failure and catch-up, B.7a for optimisation equivalence, B.9 for the channel axis, B.12 for shared-version groups), "ec" a row of the edge-case tables of SPEC §15, and "§27.10", "§27.12" and "§28.8" the numbered vectors of those sections. Appendix B numbers two vectors 82c, so the table says which. A test name set alone is in `services/dispat/internal/plan`; "(int.)" marks the black-box integration suite `tests/integration`, whose plan states the claim each test proves and, for the execution tests, the vector each one is written against. The point of the mapping is that a result with no case and no test is a claim resting on the proof alone, and the last two columns are where that shows. The last block of the table runs the other way: protocol requirements that have cases and tests but no numbered result in this report.

Result	SPEC	Cases	Test
Theorem 85 (**G1**)	13.7c, 9.2	67, 68	`TestPropagationDepthIsShortestPath`

Result	SPEC	Cases	Test
Theorem 87 (**G2**)	13.7c, 13.4a	66, 74, 76, 80d, 82c (B.7); ec 87, 88	`TestCatchUpAfterConsumerFailure`, `TestCatchUpForNeverReleasedConsumer`, `TestAdmissionCatchesUpAConsumerThatOvertookItsProvider`, `TestAdmissionOwedSetNamesOnlyTheProvidersStillOwed`; `TestAdmissionCatchesUpAConsumerThatProceededOnItsOwn`, `TestAdmissionCatchesUpAConsumerThatOvertookAHeldProvider` (int.)
Theorem 89 (**G3**)	13.7c	70; ec 84, 137	`TestCatchUpOnATrainWithPublishedOwnWork`, `TestSpec2SourceGraduationChangesInheritedRetry`
Theorem 92 (**G4**)	13.7c	66; ec 83, 89, 133	`TestCatchUpDischargesExactlyOnce`
Theorem 94 (**G5**)	13.7c	67, 68; ec 92, 134	`TestCatchUpNeverWidensBlastRadius`, `TestSpec2SuppressionLapseCanExposeCatchUp`
Theorem 98 (**G6**)	13.7c, 19.6	77, 77a, 80; ec 133	`TestPrereleaseTrainConvergesAfterRelease`, `TestPropagatedTrainWorkConverges`
Proposition 99, Lemma 97	13.7c, 19.3	80d; ec 88	`TestAdmissionCatchesUpAConsumerThatProceededOnItsOwn` (int.); no test of the optional-edge case of Remark 100
Proposition 26	19.3, 13.4a	80d (last case)	— (`E201` unimplemented, proof obligation 131)
Proposition 101	13.10a	77, 77a; ec 98	`TestFixedConvergesWhenNothingPending`
Theorem 102 (**G7**)	13.7c, 9.3	97c; ec 60r	`TestChannelAxisConverges`, `TestTransitionIsIdempotent`
Theorem 104 (**G8**)	13.7c, 9.3a	94, 95, 97; ec 136	`TestPropagatedStableNeverGraduates`, `TestCaretAloneDoesNotDragConsumersOntoATrain`, `TestSpec2RejectedBumpDoesNotSuppressChannel`
Theorem 106	17.2	ec 82; §28.8 5	`TestConflictingDirectChannelsPickTheNewest`, `TestConflictingPropagatedChannelsPickTheNewest`, `TestPlanDigestIsStableAcrossRecomputation`
Corollary 108	17.2	ec 82, 83	`TestNoCatchUpWhenConsumerIsFresh`
Theorem 107	17.2, 19.3	70; ec 84, 87, 91	`TestCatchUpAfterConsumerFailure`; `TestPlanConsumerFailureCatchesUpAfterProviderPublished`, `TestPlanProviderBuildFailureBlocksConsumerThenHeals` (int.)
Theorem 111	13.7b	ec 135	`TestStaleSourcesAgreesWithPropagation`, `TestSpec2RejectedBumpProducesNoStaleSourceRow`
Proposition 114	13.7b	—	`TestPossiblyBehind` (the screen's shape only; see below)
Proposition 45	19.2	69, 80a; ec 93, 103	`TestOrderChainRunsInTopologicalOrder`, `TestPublishOrderHoldsThroughAPackageThatIsNotReleasing` (int.)
Proposition 49	19.2a	80b, 80c	`TestStageRelationOrdersBuildsThroughAPackageThatDoesNotBuild`, `TestStageRelationBuildsOverlapWhilePublishesFollowTheProvider`, `TestStageRelationBooleansKeepTheirMeaning`, `TestStageRelationNoneCarriesNoBuildOutputs` (int.)
Proposition 67	17.2, 28.3	§28.8 1, 3, 5, 9	`TestExecutionAbsentKeepsLocalReleaseUnchanged`, `TestExecutionModesPlanIsIndependentOfWorkers`, `TestExecutionPlanDigestIgnoresPlacementAndTime`, `TestExecutionPlanIsIndependentOfWorkers`, `TestExecutionPreparationLeavesThePlanDigestUnchanged` (int.)
Proposition 117	13.11	—	`TestAncestryIndexAgreesWithAWalk`, `TestUnionReadPlansExactlyAsPerBoundaryReads`

Result	SPEC	Cases	Test
Proposition 116	13.11, 27.9	§27.10 20; §27.12 29	`TestPlanningQueriesGitOncePerPackage`, `TestPlanningSharesHistoryForDifferentTagsAtTheSameCommit`, `TestComposedUnionReadPlansExactlyAsPerBoundaryReads`, `TestDistinctBoundaryWindowsRetainCanonicalPayloadOnce`; `TestPolyrepoPrereleaseAndStableWindowsStayRepositoryLocal` (int.)
Proposition 122	13.11	82c (B.7a) to 82k; ec 138	`TestCachedWalkIsTheLiteralWalk`, `TestWalkResultCannotReachItsSharedTail`, `TestSpec2SelfSourceCheckIsPerUnit`
Proposition 125	9.3a, 13.11	82c (B.7a), 82d	`TestCaretAloneDoesNotDragConsumersOntoATrain`
Proposition 126	13.8, 13.11	82e, 82f, 82k	`TestDirectChannelBeatsPropagated`, `TestConflictingDirectChannelsPickTheNewest`
Proposition 130	20.7	99	`FuzzParse` (`pkg/ccme`)
Protocol requirements with cases and tests but no numbered result			
Blocking closure (Definition 50)	19.3	70, 80a; ec 91, 104	`TestPublishOrderBlocksAConsumerBehindAFailedProviderItDoesNotNameDirectly`, `TestOrderProviderFailureSkipsTheWaitingConsumer`, `TestStageRelationBlockingSkipsAConsumerWithItsOwnChanges` (int.)
Records under the lock	13.2, 19.1	§28.8 27; ec 76a	`TestReleaseRecordsRefuseACloneWithoutTheRemoteTags`, `TestReleaseRecordsNeverMoveAPublishedTag`, `TestReleaseRecordsRefuseATagAtAnotherCommit`, `TestReleaseRecordsCreateOnlyPush`, `TestReleaseRecordsInAComposedWorkspace` (int.)
Output identity	28.5	§28.8 8, 10, 17, 25	`TestExecutionIgnoredOutputsReachConsumersOnOtherWorkers`, `TestExecutionBranchTransportCarriesEveryDeclaredOutput`, `TestExecutionDefectiveOutputFailsThePrerequisite`, `TestExecutionPartialOutputSetStaysUnready`, `TestExecutionCraftedOutputSetsNeverReachAConsumer`, `TestExecutionOutputBytesIgnoreTheCheckoutAttributes` (int.)
Ready/go handshake	28.6	§28.8 13, 30	`TestExecutionPublishRunsOnWorkersUnderAuthorization`, `TestExecutionUnauthorizedPublisherNeverStarts`, `TestExecutionCraftedPublicationAuthorizations`, `TestExecutionLostPublishReplyIsNeverRepublished` (int.)
Lock loss	28.6	§28.8 14	`TestExecutionLockLossStopsNewEffects`, `TestExecutionKilledWorkerBlocksDependentsAndKeepsCapacity`, `TestExecutionLostLockIsRememberedForEveryLaterTask`, `TestExecutionLockLostDuringTheHookWithdrawsTheWaitingPublisher`, `TestExecutionUnansweredPublisherRetainsTheLock` (int.)
Capacity on evidence	28.2	§28.8 21	`TestExecutionQueuedWorkIsNotATimeout`, `TestExecutionWorkerCapacityHoldsAcrossRuns`, `TestExecutionTaskTimeoutLeaksTheSlot`, `TestExecutionQueuedAssignmentIsReplacedAndThenRefused` (int.)

Result	SPEC	Cases	Test
Consumer boundaries	27.6, 27.11	§27.10 11, 12, 19; §27.12 10, 13, 28	`TestPolyrepoReleaseCheckpointProvidesNextConsumerBaseline`, `TestPolyrepoCrossRepositoryBaselineRequiresEvidence`, `TestPolyrepoSinceControlRevisionProjectsSourceGitlinks`, `TestChoreographyReadsTheBoundaryFromTheReleasedLink`, `TestChoreographyRelaysTheBoundaryAlongTheRoute`, `TestChoreographySettlesTheProviderRevisionBeforePublishing` (int.)

Several rows deserve comment.

Theorem 111 has a test, and the test cannot fail. As proof obligation 140 records, `Plan.StaleSources` returns the rows the downward traversal recorded, and it still does: at `252216ba` it returns `r.Sources` of the release, so `TestStaleSourcesAgreesWithPropagation` compares a quantity with itself. That is a reasonable engineering choice (an independent upward walk would be a second implementation of the same rule, with its own bugs) but it means the duality is asserted rather than checked here, and it is the reason proof obligation 140 went unnoticed. `TestSpec2RejectedBumpProducesNoStaleSourceRow` exercises the observable eligibility result of edge case 135; the randomised cross-check SPEC §13.7b recommends is the thing that would catch a disagreement, and it needs a workspace with a prerelease train to do so.

Proposition 114 has no case and a test that checks the screen's shape only. `TestPossiblyBehind` asserts that a provider's tag outside the consumer's ancestry flags the consumer, that a never-released provider owes nothing, that the ancestor relation clears the flag, and that no claim is made without ancestry. Nothing checks the converse the proposition is about, that a flagged consumer may owe nothing; the proposition rests on its proof.

Theorem 98 is the row whose test predates the normative fresh-window rule. `TestPrereleaseTrainConvergesAfterRelease` opens with the words "the reported bug": a package that entered a beta train and published `1.0.0-beta.0` re-appeared in every subsequent plan as `beta.1`, `beta.2`, and so on. That is exactly proof obligation 138, found in operation rather than on paper, and fixed in the engine. CCME 2.0.0 then specified W_{fresh} and the current text keeps it, so a second implementation following the text discharges the prerelease work as the engine does.

Proposition 122 had cases and no test when the engine traversed once per unit. It no longer does: `propagate.go` serves every walk from a cache keyed by source set, depth and edge kinds, holding each package's unbounded walk and composing a unit's walk from its sources' walks when the unit names at most eight of them, and walking outright otherwise. The vectors of Appendix B.7a are therefore no longer passed trivially. `TestCachedWalkIsTheLiteralWalk` compares every route the cache can take against the literal walk of SPEC §9.2, target for target and in order, over 150 random workspaces that are dense on purpose, since diamonds and sources that reach one another are where a composition goes wrong. The set X of Proposition 122 cannot arise, because the self-source exclusion is applied per unit after composition, which `TestSpec2SelfSourceCheckIsPerUnit` pins as edge case 138.

Proposition 117 has no case: the cost workload at the end of Appendix B.7a is informative and states no threshold. Its test is the definition itself: `TestAncestryIndexAgreesWithAWalk` checks the bit-parallel marker pass against a walk per ancestry question over random DAGs whose rank and topology are drawn independently, and `TestUnionReadPlansExactlyAsPerBoundaryReads` runs the planner with the union read hidden, one window per boundary, as

it ran before the pass existed, and requires the same plan.

The rows of Proposition 67 and of the last block rest on the integration plan's claims. Each execution test names in its comment the vector of SPEC §28.8 it is written against, and the plan's Goal 57 states the claim each proves; the section has thirty vectors, of which 20 and 26 do not apply to the engine (proof obligation 137). Vectors 28 and 29 of that section, the create-only record after a lost lock and the documented recovery of an abandoned one, are not named by any test, and the lock-loss row does not claim them.

Three gaps are the ones the obligations name. Vectors 79 and 100a, the optional dependency that does not block and the cycle that exists only through `devDependencies`, have no test and fail against the engine (proof obligation 134). The last two cases of vector 80d, the run the consumer sits out and the refusal with `E201`, have no test and the engine does not implement them (proof obligation 131). And the retry of a fleet record that annotates the same commit has no test on the fleet path (proof obligation 135).

12 Evaluation: the executed campaigns

The proofs of Section 6 say what the algorithm does; this section reports what the reference implementation [5] was observed to do, in the campaigns that were actually executed, with the binary, the environment and the counting unit of each one stated separately. Five questions organise the account. **RQ1** asks whether the consumer-relative admission window retains the residual obligations that a source-relative window loses. **RQ2** asks whether versions and completion records stay stable through repeated failure, repair and identity conflict, and how far the plans agree with an independent model. **RQ3** asks whether execution respects artefact availability and CI gates. **RQ4** asks what planning costs as the graph, the intent and the history grow. **RQ5**, new in this report, asks what the distributed execution profile of SPEC §28 measured on two real build graphs, and what it did not.

The discipline of the section is that every number is transcribed from a retained file that the sentence or the table note names, that no campaign is pooled with another, and that failed and discarded attempts appear beside the completed ones. Deterministic designed matrices are reported as counts with their denominators and receive no p-values.

12.1 Evidence boundaries and units

Seven bodies of evidence are used. Table 6 pins each one to its binary, its environment and its counting unit. The units differ and are not interchangeable: a *history* is one generated repository planned once; an *episode* is one fixture driven through fault, retry and repair; a *state* or a *transition* is an enumeration unit of the finite model; a *command record* is one invocation the verifier replayed; a *predicate* is one verifier check applied repeatedly to fixtures; a *release* is one `dispat release` from invocation to exit under GNU `time`. Predicate counts are implementation checks, not independent observations, and are never summed across campaigns.

Table 6: Evidence boundaries. Sources: `experiments/README.md` and `experiments/results/compare-v140-v160.md` (historical differential); `output/experiments-v182-summary.json` and `experiments/results/v182-20260907/README.md` (1.8.2); `paper/saner/evidence/EXPERIMENTS-1.9.0-20260909.md` (1.9.0); `paper/saner/evidence/EXPERIMENTS-1.10.0-20260909.md` and `experiments/saner-2027/campaign-1.10.0-20260909/summary.json` (1.10.0); `experiments/saner-2027/campaign-mechanisms-20260910/RESULTS.md` (mechanisms); `/Users/yohimik/Projects/linux-build-test/dispat/gce/PARITY-20260922.md` and `PARITY-20260922-round2.md` beside it (kernel); `.ci/dispat/gce/REPORT.md` on branch `origin/dispat-gce` of `/Users/yohimik/Projects/llvm-project` (LLVM).

Campaign	Binary under test	Environment	Counting unit and record
Historical differential, from 30 August 2026	Released `dispat 1.4.0 (linux_arm64)` from the official `yohimik/dispat-alpine:1.4.0` image (release commit `5e3a2337`); repeated on 1.6.0 (`f9c2acf2`), 1.7.0 and 1.7.2	Docker on an Apple-silicon host; one container at a time	History (generated instance); `experiments/results/v140`, `v160`, `v170`, `v172` and the `compare-*.md` notes beside them
1.8.2 E1 to E5, 7 September 2026	`dispat 1.8.2 (linux_-arm64)`, CLI commit `a7011cbcb9e88e52e7336f6fc8f1b207a8e7a311`, image `yohimik/dispat-alpine@sha256:4c0ad4b8268dd10dbb136008798c1b07ffc1c0c7c534b8465a9f5edcab0e7bfd`	Docker, two CPUs and 4 GiB per container, contended host	History (E1), repetition (E2), release row (E3), episode and timing cell (E4), check (E5); `experiments/results/v182-20260907`, 753 files under its manifest
1.9.0 companion, 9 September 2026	`dispat 1.9.0 (linux_-arm64)`, SHA-256 `be3370c472cdbfe66fe4256d3fc1ff78ff19bdd5f041ef222187a52767fa4f35`, runner image `sha256:97363c9dbac292599987e66241d6da56abd7540df965b24bc20d20573fcac31f`	Docker Desktop Linux ARM64, two CPUs and 4 GiB, no network except private registries	History, episode, execution, CI repetition, predicate; `experiments/saner-2027/campaign-next-release-prepared/runs`, one failed and one completed attempt
1.10.0 companion, 9 September 2026	`dispat 1.10.0 (linux_-arm64)`, SHA-256 `03d1ba5fbaef35b69582e9489f40b5438f81ae6926e02ea6d02716b5b73d92fd`, runner image `sha256:6f92932fbba5d33d61d777073f5fc155133063f4c3d4b2f950e3e5b9ed9a263e`	As above	As above; `experiments/saner-2027/campaign-1.10.0-20260909`, archive of 9,157 files
Mechanism matrix, 9 September 2026	The same 1.10.0 binary and runner image as the row above	Docker, network disabled, two CPUs, 4 GiB	Episode, state, transition; `experiments/saner-2027/campaign-mechanisms-20260910`, 16,464 fixture files archived
Kernel farm rounds, 22 September 2026	Round 1: a development build, SHA-256 `a7172a7feb21e27ce882050120b9e04df1e6d4b4ee5f3341ab8a99ba155ba70a`. Round 2: the feature branch at `f0c9e959`, static linux/amd64, SHA-256 `63441cc66f68d07b5efbf9a42edd5559304d153d48b82b1dfb037349e20606ed`	Google Compute Engine [19]: `c3-standard-22` and `t2d-standard-8`, region us-central1	Release (GNU `time` elapsed), per-cell script seconds; `dispat/gce` of the fork at `c699bf3d`, copied under `experiments/distributed-20260922/evidence/kernel-gce`

Campaign	Binary under test	Environment	Counting unit and record
LLVM pair, 22 September 2026	Attempts 1 to 8: the feature branch at `f0c9e959`; attempt 9: `1.11.0-rc.4+main.272772cb` as printed in its release log	Google Compute Engine: `e2-standard-4` orchestrator, two `n2-custom-14-49152` workers, zone us-central1-a	Release; the fork's `.ci/dispat/gce/REPORT.md` and the retained logs under `experiments/distributed-20260922/evidence/llvm-gce/logs`

None of these campaigns measures a release-candidate feature outside **RQ5**. The parser in every campaign through 1.10.0 is CCME 2.0.0 (`paper/saner/evidence/EXPERIMENTS-1.10.0-20260909.md`); the kernel and LLVM rounds run the SPEC §28 execution profile of the 1.11.0 release candidate and are the only evidence about it.

12.2 RQ1: the consumer-window mechanism

The consumer-relative window of Definition 9 is the mechanism behind **G2**. The mechanism campaign tests it two ways, from `experiments/saner-2027/campaign-mechanisms-20260910/PROTOCOL.md`: a finite projection of the model, and a counterfactual computed on the completion states the binary actually produced.

The finite projection. The projection enumerates every DAG consistent with one fixed order of four named vertices, each of four single intent sources, every completion set reachable by repeatedly recording an enabled package, and every subset of the remaining packages designated as failed. It checks exact residual obligations, no widening, no double ledger delivery, and strict decrease exactly when progress occurs. It then applies the source-window mutant, which discards a unit when its originating package completes, to the same states. The counts, from the `executor.model` object of `experiments/saner-2027/campaign-mechanisms-20260910/summary.json`, are 64 graphs, 256 source instances, 772 reachable completion states, 2,270 failure-subset transitions of which 1,054 made progress, 0 projection violations, and 260 states in which the source-window mutant lost obligations. States and transitions are enumeration units, not samples. The projection is a stable-channel, one-intent model of four vertices; it is not a mechanised proof of the full language, of arbitrary graphs or of implementation refinement (`RESULTS.md`, section "Finite projection").

The binary-derived counterfactual. The counterfactual takes each episode's original obligation set and its observed completion tags and asks what the source-window rule would have discarded. It runs no modified binary and compares no competitor (`RESULTS.md`, section "Source-window counterfactual"). Table 7 transcribes the strata of `summary.json`.

The figures the conference account quotes are the fault-matrix row: 111 obligations lost in 84 of the 120 fault episodes, with 28 of 40 episodes at each of the three boundaries and 47 of 70 in each channel. The ten no-fault controls lose nothing, which is the expected null. The ten foreign-identity cases lose one obligation each and are reported separately because they test a different intervention (Section 12.3). The counts are the sensitivity of a deficient analytical rule under designed cases, not a win rate.

*Answer to **RQ1**.* In every state the projection reached and in every episode the binary completed, the consumer-relative window retained exactly the residual set, and a source-

Table 7: Source-window counterfactual by stratum. Every figure is the `episodes`, `source_mutant_losing_episodes` or `source_mutant_lost_obligations` field of the corresponding `strata` entry in `experiments/saner-2027/campaign-mechanisms-20260910/summary.json`; the same figures appear in `RESULTS.md` beside it. Units are episodes and obligations.

Stratum	Value	Episodes	Losing episodes	Lost obligations
Class	fault matrix	120	84	111
	no-fault control	10	0	0
	foreign identity	10	10	10
Boundary	build	40	28	41
	before publication	40	28	35
	after artefact write	40	28	35
	foreign	10	10	10
	none	10	0	0
Channel	stable	70	47	61
	beta	70	47	60
Graph	chain	28	20	38
	diamond	28	20	25
	fan-out	28	20	20
	two disconnected pairs	28	14	14
	chain with shortcuts	28	20	24
All		140	94	121

relative window would have lost obligations in 260 of the 772 states and in 84 of the 120 fault episodes.

12.3 RQ2: retry and identity stability

The mechanism matrix. The matrix crosses five four-package topologies (chain, diamond, fan-out, two disconnected pairs, chain with shortcuts) with stable and beta baselines, four fault targets and three injection boundaries, giving 120 fault episodes; ten no-fault controls and ten foreign-identity cases bring it to 140. Each fault episode executes the same fault twice, repairs it and runs a final no-op; the order is fixed by seed 20260910; build and publish concurrency are one (`PROTOCOL.md`). The run took 113.653827 s, from 2026-09-09 20:39:05.988761 to 20:40:59.642604 UTC, and passed 140 of 140 episodes with no failed, invalid or excluded case (`summary.json`, `execution` and `executor`). Table 8 gives the five checks.

Table 8: Mechanism checks of the 140-episode matrix, from the `mechanism_checks` object of `experiments/saner-2027/campaign-mechanisms-20260910/summary.json`. The 130 denominator is the 120 fault episodes plus the ten foreign-identity cases; the ten no-fault controls do not enter it (`RESULTS.md`).

Check	Passed / denominator
Repeated fault preserved the residual plan and tags	130 / 130
Successful repair left an empty plan	130 / 130
Exactly one durable artefact write per obligation	140 / 140
Foreign artefact identity conflict was refused	10 / 10
Final no-op left the plan empty and tags unchanged	140 / 140

Per stratum, 40 of 40 episodes passed at each boundary, 70 of 70 in each channel and 28 of 28 on each graph (`summary.json`, `strata`). The after-write injector lets the supplied local publisher write the artefact durably and then return a nonzero exit; it simulates neither

SIGKILL nor a network partition nor power loss. The foreign-identity cases test that adapter's immutable version-and-content identity, not an npm or OCI registry (summary.json, interpretation). An independently implemented offline checker reconstructed every plan from the raw command output, checked tags, publisher events and the four stored artefact identities per case, and re-enumerated the finite model; it reported no discrepancy over 140 episodes (summary.json, offline_consistency_check). A negative check on a corrupted copy failed as intended (RESULTS.md, "Evidence integrity").

Reference agreement on the companion campaigns. The 1.10.0 companion campaign planned 300 generated histories, seeds 90001 to 90300, against the restricted core oracle: 300 accepted matching rows, 0 semantic rejections, 0 invalid executions, 0 rows with differences (experiments/saner-2027/campaign-1.10.0-20260909/summary.json, core). The corrected 1.9.0 campaign reports the same 300 of 300 on the same seeds (paper/saner/evidence/EXPERIMENTS-1.9.0-20260909.md). The oracle is the historical model of the conference paper's restricted profile; 300 of 300 on a fixed sample is agreement on that sample, not a universal conformance result.

The recovery stage of each companion campaign drives 20 episodes over graphs of 20 and 100 packages with failure probabilities .1 and .3 and seeds 100 to 104. In 1.10.0 all 20 converged with progress on every run and at most four runs, distributed as nine episodes in two runs, five in three and six in four, over 251 raw command records with no invariant error (summary.json, recovery). In the corrected 1.9.0 run the same 20 episodes converged in at most four runs over 248 command records, distributed nine, six and five (EXPERIMENTS-1.9.0-20260909.md). The two campaigns are not pooled. runs/run-v1.10.0-linux-arm64-20260909/recovery-comparison.json locates the difference in one episode, graph size 100, probability .3, seed 100: the first attempt left 20 packages pending in 1.10.0 against 18 in 1.9.0, so the later seeded deny sets diverged and the remaining counts ran 20, 2, 1, 0 against 18, 2, 0, which accounts for one extra release, tag and status triplet (EXPERIMENTS-1.10.0-20260909.md). The traces do not attribute the difference to the 1.10.0 feature; fixed seeds do not fix a concurrent executor's schedule.

The 1.8.2 full-profile campaign. The 7 September 2026 campaign on the released 1.8.2 binary planned 3,000 generated instances in four shards: core seeds 1 to 1000 and 5001 to 5500, and full-profile seeds over the same ranges (output/experiments-v182-summary.json, E1.shards). Both core shards agreed exactly, 1,000 of 1,000 and 500 of 500, with 0 invalid executions. The full profile deliberately generates regressive prereleases: of its 1,500 instances, 503 were refused with a validated E195 diagnostic and 997 were accepted; of the accepted, 546 matched the historical oracle; four refused dossiers also had no row difference, so 550 comparisons were diagnostic-clean; the divergent package rows numbered 1,914, classified by the campaign's heuristic classifier as PG1 862, PG2 375, PG2x 14 and PG3? 663 with none unclassified (output/experiments-v182-summary.json, E1.totals and E1.divergence_classes; experiments/results/v182-20260907/README.md). Per shard the full-profile figures were 343 refusals, 353 accepted-clean and 356 diagnostic-clean rows with classes 572, 240, 9 and 442 for seeds 1 to 1000, and 160, 193, 194 and classes 290, 135, 5 and 221 for seeds 5001 to 5500 (E1.shards). The canonical manifest binds the four shards to extractor SHA-256 3d5eef3ef2bf87de7876ddf6d04e852028b72091dd2417ce4943c96e48011e52 (experiments/results/v182-20260907/e1/canonical-manifest.json). A refused diagnostic plan is not an executable release plan, the oracle implements the historical CCME 1 model, and the PG3? label is heuristic, so these counts are taxonomy evidence rather than CCME 2.0.0 conformance (output/experiments-v182-summary.json, limits).

The first 1.8.2 attempt is retained at `experiments/results/v182-20260907-attempt1`: its strict exit checker classified the expected `E195` refusals as invalid executions, and the corrected campaign reran every seed from scratch rather than patching the records (`experiments/results/v182-20260907/README.md`). Seed 580 of the core profile, which produced an invalid fixture under 1.8.1, succeeded under 1.8.2 in a changed fixture environment (tmpfs and a pre-run `git fsck`); planner source is unchanged between the two releases, so the success is not a product fix (`README.md`; `E1.normalized_changes_from_v181` lists that one core seed and 503 full-profile seeds).

A separate check in the same campaign ran SPEC Appendix B vectors 133 to 138 against the released 1.8.2 binary: 6 of 6 passed (`output/experiments-v182-summary.json`, `CCME2`). Vector 135 exercises the observable downward eligibility result and is not an independent implementation of the upward audit, so the proof remains the support for Theorem 111.

The historical differential campaigns. The oracle of E1 (`experiments/e1/oracle.py`) implements the model of this report as it stood against the CCME 1.0.0 snapshot; the generator (`experiments/e1/gen.py`) produces layered instances and the realiser (`experiments/shared/realize.py`) turns each into a real repository (`experiments/README.md`). On the released 1.4.0 binary the core profile agreed 1,000 of 1,000 on seeds 1 to 1000 and 500 of 500 on seeds 5001 to 5500. The full profile over seeds 1 to 1000 left 412 instances clean and classified every divergent row as PG1 409, PG2 240, PG2x 58 or PG3 390, and over seeds 5001 to 5500 left 222 clean with PG1 182, PG2 135, PG2x 23 and PG3 200, with zero unclassified (`experiments/README.md`, "v1.4.0 campaign, remaining results"; `experiments/results/compare-v140-v160.md`). The first shard's 1,097 divergent rows are the four predicted classes and none outside them: the 240 PG2 prerelease-train discharges and the 58 PG2x suppression-lapse and cascade variants are proof obligations 138 and 139 observed at scale; the 409 PG1 channel-entry patches are the deliberate one-patch rule of SPEC §11.4 (`W204`), which the report model does not transcribe; and the 390 PG3 bump-axis liveness rows arise because the oracle implements the companion article's compact admission, which drops clause (A1). The 1.6.0, 1.7.0 and 1.7.2 campaigns reproduce the per-seed rows on all 3,000 instances with only the version string differing (`experiments/results/compare-v140-v160.md`, `v170/compare-v160-v170.md`, `v172/compare-v170-v172.md`). These records validate the CCME 1.0.0 model against those binaries; they do not validate CCME 2.0.0 admission, whose window is the subject of **RQ1**.

Orphan recovery and mid-release pushes. E2 drives a six-package npm fixture against a Verdaccio registry behind a failing proxy that answers 502 to named PUTs (`experiments/README.md`). In the 1.8.2 campaign both repetitions recorded the failed publish untagged, a same-version recovery and a final plan releasing nothing (`output/experiments-v182-summary.json`, `E2`). E5's three fresh 1.8.2 cells passed 8 checks for the orphan scenario, 12 for a clean concurrent push and 14 for a conflicting concurrent push (`E5`); the two push scenarios distinguish a completed release tag from a concurrent source change on selected interleavings, not on a systematic sweep. Its nine competitor cells were not rerun and remain historical (`limits`).

Historical competitor observations. The competitor phases of 30 August 2026 pin the newest versions published on that date: Lerna 10.0.1, Nx 23.1.2, Turborepo 2.10.12 with Changesets 3.0.1, and release-please 17.11.2 (`experiments/README.md`, "Competitor tool pins"). Each phase was run twice with identical protocol-relevant transcript lines. Lerna tagged at version time, before publish; the denied `cli` left `cli@1.0.1` tagged and orphaned,

and recovery was blocked by `EUNCOMMIT` until manual cleanup. Nx also tagged before publish and propagated a patch to all five dependents of a minor change; its recovery is `nx release publish` again, which skips already-published versions by registry query. Changesets tagged after publish, left no tag for the denied package, recovered by registry query, and gated propagation by version range; its release intent is a changeset file consumed at version time, so the plan is not a function of Git history and cannot be replayed. Turborepo neither versions nor publishes. release-please has no local publish step and was observed only through the E3a replay (`experiments/README.md`, "E2 extension results"). These are observations of the pinned configurations, not properties of every mode those tools expose.

Historical replay. E3a replays five releases each of two public repositories under a reconstructed layout and policy. The 1.8.2 campaign reproduced the established profile: 36 of 42 released package-versions exact for `google-cloud-node` and 7 of 64 for `conventional-changelog` (`experiments/results/v182-20260907/README.md`, and `E3` of `output/experiments-v182-summary.json`, which records `metrics_equal_v181` true for all ten rows). The per-release rows are in `experiments/results/e3a/gcn.json` (released 1, 22, 11, 7, 1 and exact 1, 17, 11, 6, 1) and `experiments/results/e3a/cc.json` (released 20, 8, 9, 6, 21 and exact 2, 3, 1, 0, 1). Neither project used the modelled intent language, and the mismatches may reflect the reconstruction as much as the projects' conventions; the experiment shows the limit of policy transfer and does not claim those releases followed CCME.

*Answer to **RQ2**.* Across the designed graph, channel, target and fault factors of the 140-episode matrix, recorded progress survived repeated failure without version drift, repair emptied the plan, and the identity check refused every foreign artefact. The restricted oracle agreed on all 300 histories of each companion campaign and on all 1,500 core instances of the 1.8.2 campaign; the 1,914 divergent full-profile rows and the 1,097 rows of the historical campaign are retained and classified, and they mark where the restricted models stop rather than where the implementation fails.

12.4 RQ3: availability boundaries and CI

The companion campaigns execute two registry topologies and one CI fixture. The three-package topology is an npm package, a Docker base image and a Docker application; the four-package extension is `assets` (npm), `ui` (npm), `base` (Docker) and `app` (Docker), where `ui` reads the predecessor's local `dist` output and performs no registry install (`experiments/saner-2027/campaign-next-release-prepared/CAMPAIGN-PLAN.md`). Each topology repeats five configurations six times in a frozen seeded order: mixed local-output scheduling, fully serialised controls, a missing barrier, an injected base-publication failure with repair, and an injected app-build failure with repair.

Per topology the 18 designed failures are six of each negative configuration; all 12 repairable executions completed at unchanged planned versions; the six missing-barrier cases intentionally have no repair; the other 24 completed cases performed no build or publication on their final unchanged run (`EXPERIMENTS-1.10.0-20260909.md`). In the three-package mixed fixtures the base build overlapped npm publication in all six observations, by 0.110716 to 0.151483 s. In the four-package mixed fixtures the `ui` build overlapped the deliberately delayed `assets` publication by 0.070128 to 0.073725 s, and the base build started 6.292152 to 6.393487 s before the `ui` publication ended and completed before that publication began in all six cases. The latter is an admissible schedule: a build that reads local output need not wait for its provider's publication, and the fixture's delays expose availability rather than measure throughput. The

Table 9: The 1.10.0 companion campaign. Executions, initial outcomes, repairs, no-op reruns and predicate counts are the `mixed` and `integration` objects of `experiments/saner-2027/campaign-1.10.0-20260909/summary.json`; the intervals are the six-decimal figures of `paper/saner/evidence/EXPERIMENTS-1.10.0-20260909.md`, whose nine-decimal originals are the `overlap` arrays of the same `summary.json`. Predicates are repeated verifier checks, not samples.

Topology	Executions	Initial successes	Designed failures	Repairs	No-op reruns	Predicates
Three-package	30	12	18	12	24	522
Four-package	30	12	18	12	24	798
CI fixture	3 repetitions					81

three CI repetitions each cover baseline publication, a specification change, a consumer change, an injected failing test, a missing required test, repair and an unchanged run; the three failing-test and three missing-test negatives blocked the release stages, the failing-test attempts wrote no release tag, repairs propagated the specification release to its consumer, and unchanged runs executed the selected tests without release stages.

The corrected 1.9.0 campaign. The completed 1.9.0 run, `run-v1.9.0-linux-arm64-20260909-corrected`, has the same 12, 18, 12 and 24 counts per topology, the same 522, 798 and 81 predicate counts, overlaps of 0.101969 to 0.128675 s in the three-package fixtures and 0.068203 to 0.077008 s between `ui` and `assets`, with the base build starting 6.298757 to 6.311191 s before `ui` publication ended (`paper/saner/evidence/EXPERIMENTS-1.9.0-20260909.md`). Its provenance records all 14 stages passed and a `COMPLETE` marker written at 2026-09-09T10:48:01 UTC (`runs/run-v1.9.0-linux-arm64-20260909-corrected/provenance.json`).

The preserved failed oracle attempt. The first 1.9.0 attempt, `runs/run-v1.9.0-linux-arm64-20260909`, ran from 05:35:36 to 07:37:14 UTC, passed its first thirteen stages and failed the fourteenth, `mixed-4node-verification`, with exit code 1 (`provenance.json` of that run, `status: FAILED`). Its verifier log ends in the assertion "`mixed-2: base local build overlaps ui publication`" (`mixed-4node-verification.log`): the original oracle required the base-build and `ui`-publication intervals to overlap strictly, and the traces showed the valid schedule in which the base build completed before `ui` publication began. The predicate was corrected before the full rerun to the dependency-relevant condition, that the base build starts before `ui` publication ends, with overlap and completed-before-start both recorded descriptively (`CAMPAIGN-PLAN.md`). The failed attempt has no `COMPLETE` marker and is not counted as passing evidence; its recovery stage had reached a maximum of five runs over 251 raw command records where the corrected run reached four over 248, which is why the two are not pooled (`EXPERIMENTS-1.9.0-20260909.md`). The 1.10.0 campaign reuses the corrected protocol byte for byte, protocol manifest SHA-256 `a3366e2fbdc590be7e8f6268bc7390146669cf7a0395837c2531a0af5554dcaa`, with no further oracle change (`campaign-1.10.0-20260909/summary.json`).

A separately frozen diagnostics addendum in the 1.10.0 campaign passed 16 of 16 purpose-selected cases for the standalone diagnostics command; it is engineering evidence outside the 14-stage campaign and its sample counts (`EXPERIMENTS-1.10.0-20260909.md`).

*Answer to **RQ3**.* In these fixtures, on one Linux ARM64 environment, a configured barrier distinguishes local build availability from required publication, and both a failed and an

absent required test prevent release; the campaign measures neither concurrency at scale nor onboarding effort.

12.5 RQ4: planning cost

The E4 timing cells plan generated workspaces of P packages with U intent units, in three graph shapes, with an optional padding history of commits that touch no package. Table 10 transcribes the eight cells of the 1.8.2 campaign.

Table 10: Planning cost, 1.8.2 campaign: the eight rows of `experiments/results/v182-20260907/e4/timing.csv`, verbatim. Seconds are the median, minimum and maximum of the cell's repetitions; MiB is the maximum resident set size. The container was capped at two CPUs and 4 GiB on a contended Apple-silicon Docker host (`experiments/results/v182-20260907/README.md`).

P	Shape	U	Padding	Edges	Median s	Min s	Max s	MiB
100	layered	200	0	185	0.017	0.017	0.017	15
1,000	layered	200	0	2,002	0.250	0.235	0.252	21
10,000	layered	200	0	19,919	19.272	18.992	20.000	92
1,000	chain	200	0	999	0.254	0.241	0.258	26
10,000	chain	200	0	9,999	19.354	18.933	19.645	118
1,000	star	200	0	999	0.244	0.237	0.247	19
1,000	layered	1,000	0	2,002	0.290	0.280	0.302	26
1,000	layered	200	5,000	2,002	0.664	0.640	0.665	30

The medians rise from 0.017 s at 100 layered packages to 19.272 s at 10,000, and the 10,000-package chain records 19.354 s and 118 MiB, the largest of both columns. Multiplying the intent five-fold at 1,000 packages moves the median from 0.250 to 0.290 s; adding 5,000 padding commits moves it to 0.664 s, the history term of Section 9 showing through. The fixture generator uses `git commit-tree` for long padding histories only after a small fixture matched the ordinary generator's commit IDs, refs, messages and log output, and every fixture passed `git fsck` (`experiments/README.md`, gotcha 18 and the paragraph following it).

The loaded-host repetitions. The earlier campaigns measured the same headline cells under a different harness and report much larger figures: in the 1.4.0 campaign the headline cells were re-measured in five containers each, giving about 0.6 s at 100 packages, 2.8 to 6.7 s at 1,000 and 110 to 149 s at 10,000, with the spread attributed to bimodal container placement on the host, and the 1.6.0 comparison records medians of medians of 0.61, 6.49 and 143.1 s against 0.60, 6.43 and 142.4 s (`experiments/README.md`, "v1.4.0 campaign, remaining results"; `experiments/results/compare-v140-v160.md`). Those figures are not comparable with Table 10: the runner then retained every generated workspace until the container exited and divided the resident set size twice, and it was rewritten before the 1.8.x campaigns (`experiments/README.md`, gotcha 17). The two sets are reported side by side as what each harness measured; neither is a cross-version benchmark.

The E4 recovery cells of the 1.8.2 campaign converged in 24 of 24 episodes within four runs, 31 total runs in the first repetition and 32 in the second (`output/experiments-v182-summary.json`, `E4.recovery`); the 1.4.0 campaign's 24 episodes converged with a maximum of five runs and the 1.6.0 campaign's with four (`compare-v140-v160.md`).

*Answer to **RQ4**.* On the capped host the observed cost grows sharply between the 1,000- and 10,000-package cells and with history length; the design isolates neither a general asymptotic

bound nor an end-to-end figure, and the loaded-host repetitions show how far the same cells move with the harness.

12.6 RQ5: distributed execution

The execution profile of SPEC §28 places build and publish stages on worker nodes that share a Git repository with the orchestrator as their only transport. Two build graphs were measured on 22 September 2026, both on Google Compute Engine [19]: the Linux kernel image-build farm of kerneltests.org [15], whose cells are independent, and the LLVM monorepo, whose cells consume one another's outputs. The specification's own record of the implementation and of both measurements is the entry "2026-09-22: Distributed execution implemented" of `/Users/yohimik/Projects/dispat/specs/ccme-spec/DESIGN-HISTORY.md`, which this subsection follows in substance and cites for the figures it states. The claim the measurements support is narrow and is stated once here: on these fixtures, once each, a distributed release took no longer than the same recipe on one node. No speedup of the profile or of the engine is claimed beyond that. The raw observations of both graphs are copied, unedited, into `experiments/distributed-20260922/evidence/` (the evidence tree below), whose `MANIFEST.json` records every file's origin and SHA-256 and whose `CROSS-CHECK.json` traces the headline figures of the two fork reports and the specification entry to those raw files: 39 checks, 37 agreeing, 0 disagreeing, and two additional kernel trials that are disclosed there and reported below rather than counted as headline figures.

Acceptance oracles. Each fork carries its own comparison script, and a run counts only if the script accepts it. The kernel fork's `dispat/compare.py` accepts a measured run only if its cases, kernel revision, compiler and assembler identities match the retained public reference exactly, every cell succeeded, no configuration is missing or duplicated, and the elapsed time is no greater than the reference (`dispat/README.md` and `dispat/PARITY.md` of `/Users/yohimik/Projects/linux-build-test`). The LLVM fork's `.ci/dispat/gce/compare.py` compares the install tree of every package file by file between the single-node and the distributed checkouts, with the install prefix masked, the executable bit and symlink targets recorded, and the toolchain archive compared by member list, sizes and modes; it prints the receipts' own fingerprints beside the masked comparison and exits nonzero on any difference. Neither script measures speed on its own; the kernel gate folds elapsed time in, the LLVM pair reports it beside the product comparison.

12.6.1 The kernel farm

References. The public reference for each architecture group is the build step of the upstream builder on kerneltests.org for kernel v7.3-rc4 at `93f51579e7df24878021409441 8f205253383cc5`, read from the public build pages and retained in `dispat/gce/parity-20260922-reference.json` of the fork and in `experiments/distributed-20260922/evidence/kernel-gce/round1-codex/baseline.json`: alpha 623 s (build 332), hexagon 111 s (build 330) and m68k_nommu 128 s (build 331), with `alpha-linux-gcc (GCC) 16.2.0` and `m68k-linux-gcc (GCC) 16.2.0` over GNU assembler 2.47.20260726 from the kernel.org crosstool builds [18], and `Ubuntu clang version 18.1.3 (1ubuntu1)` for hexagon. The exact case lines are four for alpha, three for hexagon and five for m68k_nommu. These are individual build-step durations measured on the farm's own machines while sharing them with another build, not the wall time of a farm run, and they are other hardware: they bound acceptance and are not compared numerically beyond the gate. For context only, the first fork report records that the farm's 57 upstream builders took 23.0 machine-hours in

3 h 52 min of wall time for v7.3-rc4 on six machines (`dispat/gce/REPORT.md`); that figure was read from public pages and enters no comparison here.

The first run. The first real run, on the `dispat-prototype` branch with a worker-nodes build at commit `1146c210`, three VMs (an `e2-standard-4` orchestrator and two `n2-custom-14-28672` spot workers) and kernel.org crosstool gcc 14.3.0, is recorded in `dispat/gce/REPORT.md`; its orchestrator and worker logs are copied under `experiments/distributed-20260922/evidence/kernel-gce/first-run`. It is not parity evidence: the compiler is not the reference compiler, the exported pass, fail and skip counts were not retained, the wrapper accepted the script's exit code as success although the farm script accumulates errors and exits zero, and the raw logs were not found in the fork itself (`dispat/PARITY.md`). What it established was the coordination profile and two engine defects. Four groups finished; `m68k` was lost when Compute Engine preempted its worker 18 minutes into the build; the run was restarted twice because the worker's idle timer stopped a node five seconds after a 38-minute build and stopped a fresh node after 34 s. Claim-to-result was 2,250 s for alpha against the 623 s reference, 324 s for hexagon against 111 and 246 s for m68k_nommu against 128, ratios of 3.5, 2.3 and 1.8 of the script body to the reference step; the probe round trip was 4 s, assignment to claim 37 s cold and 3 s warm, and a result reached the orchestrator in 5 s (`dispat/gce/REPORT.md`). The copied logs bear these figures out: `gce-timings.py` run over `first-run/gce-logs` reads the worker's claim and completion stamps as 00:04:47 to 00:42:20 for alpha, 01:07:28 to 01:12:54 for hexagon, 01:07:27 to 01:11:33 for m68k_nommu and 01:11:40 to 01:11:54 for um, within three seconds of the report's claim-to-result figures at the logs' one-second resolution (`CROSS-CHECK.json`, the first-run entries).

Round 1: every candidate failed the gate. Round 1 (`dispat/gce/PARITY-20260922.md`) installed the reference toolchains, ran the development binary of SHA-256 `a7172a7f...` named in Table 6 on an `e2-standard-2` orchestrator, a `c3-standard-22` worker (11 cores, 22 threads) and a `t2d-standard-8` worker (8 cores, 8 threads), and measured a grouped control and four candidates, each once. The gate is GNU `time` elapsed of `dispat release` from invocation to exit, including source transport, assignment and claim polling, worker checkout, the complete build script, result transport and reporting; VM provisioning, toolchain setup and one-time source preparation are outside it.

Table 11: Kernel round 1, from the results table of `/Users/yohimik/Projects/linux-build-test/dispat/gce/PARITY-20260922.md`; the release seconds are also the `phase-timing.json` of each candidate in `experiments/distributed-20260922/evidence/kernel-gce/round1-codex/curated-linux-parity-evidence.tar.gz`. Script seconds are the complete build-script body per cell. One trial per candidate. The reference is the kerneltests.org build step.

Candidate	Placement and script seconds	Release s	Reference s	Gate
split hexagon	c3: defconfig 69; t2d: tinyconfig 54, allnoconfig 76	170.07	111	fail, elapsed only
split m68k_nommu	c3: m5272 35, m5407 34, m5249 35; t2d: m5307 65, m5475 49	159.03	128	fail, elapsed only
batched hexagon	c3: allnoconfig+tinyconfig 65; t2d: defconfig 147	165.30	111	fail, elapsed only
batched m68k_nommu	c3: first two cases 66; t2d: last three 167	196.23	128	fail, elapsed only

`compare.py` accepted the cases, toolchain identities and revision of every candidate and rejected only the elapsed time. The cold grouped control ran the three groups sequentially on

the C3 worker in 1,797.75 s of release, of which the three script bodies were 1,608 s (hexagon 130, m68k_nommu 154, alpha 1,324); alpha's script alone exceeded its 623 s reference, so no removal of dispatch overhead could have carried it. The first control transported the full 1.4 GB repository and 1.9 GB working tree, its source phase taking 45 s of which the recorded push was 44.392 s; an image-only export reduced the transport to about 512 KB and later pushes recorded 0.031 to 0.050 s. Per cell, warm kernel fetches took 0.446 to 0.571 s and worktree checkouts 7.026 to 8.327 s; assignment to claim took 1 to 4 s and completion to coordinator receipt 1 to 5 s, with a 0 s scheduler queue gap. About 89% of the control's elapsed time was inside the build scripts (`PARITY-20260922.md`). The failed setup attempt `batched-hexagon-setup-failure` is retained apart from the timed candidates; distcc was prepared and never measured. The round's raw evidence is the curated archive named in that file, SHA-256 `2fcc02ec891df09044b45a8b9d783015545e984abafe7eff745ed ac21cf08255`, 145 files, copied to `experiments/distributed-20260922/evidence/kernel-gce/round1-codex/curated-linux-parity-evidence.tar.gz`; the release elapsed figures of Table 11, the control's 1,797.75 s, alpha's 1,324 s of script, the per-group 229, 173 and 1,340 s and the 44.392 s push are the `*/phase-timing.json` members of that archive, as `CROSS-CHECK.json` traces them.

Round 2: calibration. Round 2 (`dispat/gce/PARITY-20260922-round2.md`) first measured the farm's own script bodies outside dispat, on prepared source trees, with `dispat/gce/calibrate.sh`; the raw lines are `dispat/gce/round2/calibration/cal-c3.txt` and `cal-t2d.txt`, and the installed toolchains are confirmed in `cal-c3-toolchains.txt`. On `c3-standard-22`, hexagon defconfig alone at 33 jobs took 73.92 s; the three hexagon configurations started together at 11 jobs each took 120.11, 81.04 and 63.67 s, the group finishing after 120.12 s; the five m68k_nommu configurations started together at 7 jobs each took 128.16, 127.41, 126.65, 127.04 and 103.16 s, the group finishing after 128.17 s; alpha defconfig, allnoconfig and tinyconfig alone at 33 jobs took 68.48, 33.16 and 24.04 s (`cal-c3.txt`). On `t2d-standard-8`, hexagon allnoconfig alone took 87.51 s and the two-batch m68k_nommu split took 236.18 and 295.49 s (`cal-t2d.txt`); round 1 had measured hexagon defconfig at 147 s there. Three facts followed: a single m68k_nommu build leaves part of the C3 machine idle, so several may share it; the three hexagon configurations crowd each other on one machine; and the T2D machine cannot hold hexagon defconfig inside the reference.

Round 2: topologies and results. The budget for the longest cell is the reference less the release's fixed overhead, measured at 3 to 8 s from release start to `worker ready`: 102 s for hexagon and 119 s for m68k_nommu. The topologies, generated into `dispat/gce/round2/par-c3/dispat.hexagon.yaml` and `dispat.m68k_nommu.yaml`, pin cells to the machine that starts the release with `runOnly: orchestrator` and leave the rest to the worker. For hexagon, `execution.concurrency` is 2: defconfig at 24 jobs and tinyconfig at 9 jobs on `par-c3`, allnoconfig at 12 jobs on the `t2d-standard-8` worker `w1`. For m68k_nommu it is 4: the four costliest configurations at 8 jobs each on `par-c3`, `m5475evb` at 12 jobs on `w1`. Both files carry `runOnly: [worker, orchestrator]` at the top so that a worker's cell is built on the worker and recorded by the orchestrator. Table 12 transcribes every timed release of the round: the rejected first candidate, the no-op rerun that planned nothing, the four accepted trials and the two single-node runs. The fork's files are copied unchanged under `experiments/distributed-20260922/evidence/kernel-gce/fork-codex-kernel-parity-c699bf3d/dispat/gce/round2/`, and the paths below are relative to that directory in both places.

`compare.py` accepted the exact case lines, the compiler, the assembler and the revision in all

Table 12: Kernel round 2, every timed release. Elapsed is the `elapsed=` field of the GNU `time` record in `par-c3/*.time` and `solo-c3/*.time` (also `*.elapsed`); script and lease seconds per cell and the `compare.py` verdict with its `speedup` ratio (reference over elapsed) are from `verdicts.txt`; all under `dispat/gce/round2` of the fork and under `experiments/distributed-20260922/evidence/kernel-gce/fork-codex-kernel-parity-c699bf3d/dispat/gce/round2`. The reference is the kerneltests.org build step. Each row is one release.

Run	Cells: script s (lease s)	Elapsed s	Ref. s	Verdict, ratio
hexagon, first candidate	c3: tinyconfig 60 (12.0), defconfig 84 (12.0); w1: allnoconfig 80 (12.0)	111.47	111	reject, elapsed only; 0.9957836189109177
hexagon, rerun of the unchanged configuration	no cell built: every package reported `unchanged`	3.83	111	not compared; planned nothing
hexagon, trial 1	c3: tinyconfig 70 (0.0), defconfig 85 (0.0); w1: allnoconfig 82 (0.0)	95.89	111	accept; 1.1575763896130984
hexagon, trial 2	c3: tinyconfig 71 (0.0), defconfig 84 (0.0); w1: allnoconfig 81 (0.0)	92.22	111	accept; 1.2036434612882239
m68k_nommu, trial 1	c3: m5249evb 95 (1.0), m5272c3 96 (1.0), m5407c3 96 (14.0), m5307c3 97 (15.0); w1: m5475evb 50 (0.0)	115.61	128	accept; 1.1071706599775106
m68k_nommu, trial 2	c3: m5249evb 105 (0.0), m5407c3 105 (0.0), m5307c3 106 (0.0), m5272c3 107 (0.0); w1: m5475evb 51 (0.0)	115.48	128	accept; 1.1084170419120194
hexagon, one node alone	solo: tinyconfig 91 (13.0), allnoconfig 105 (13.0), defconfig 110 (12.0)	122.62	111	reject, elapsed only; 0.9052356874898059
m68k_nommu, one node alone	solo: m5475evb 56 (0.0), m5407c3 91 (0.0), m5307c3 92 (0.0), m5272c3 73 (58.0), m5249evb 72 (66.0)	138.01	128	reject, elapsed only; 0.9274690239837694

seven releases that built cells and rejected three of them on elapsed time alone; every cell reported `passed`, three cases for hexagon and five for m68k_nommu, with no skips and no unrecognised lines (`verdicts.txt`). The eighth release, `par-c3/hexagon-2`, was a rerun of the unchanged hexagon configuration before the release tags were reset: it planned nothing, logged `unchanged` for every package and exited in 3.83 s (`par-c3/hexagon-2.elapsed` and `hexagon-2.release.log.gz`); it is a release with nothing to do and is listed so that no run of the round is omitted. The gate was therefore met twice for hexagon after one rejected candidate, at 95.89 and 92.22 s against 111, and twice for m68k_nommu with no rejected candidate, at 115.61 and 115.48 s against 128. The rejected first candidate reached 111.47 s against the 111 s reference with a 12.0 s lease wait on each of its three cells; it preceded the leased-tree change described below. Their margins, 13 to 17% and 10%, exceed the 3.7 s spread between the two hexagon trials and say nothing about other groups (`PARITY-20260922-round2.md`, "Limits"). `dispat/gce/round2/timing.txt`, produced by `dispat/timing.py` over the coordinator and worker logs, reports the same overheads from the worker's side across the five paired runs: the initial source push at 0.000 s with 0.009 to 0.024 s of Git work, assignment to claim in 0 to 3 s, the per-cell farm checkout in 0 to 1 s, a finished cell reaching the coordinator in 0 to 5 s, and the release's own Git subprocesses at 7.205 to 7.231 s in every run.

What the harness changes bought. Three changes separate round 1's split hexagon at 170.07 s from round 2's trial 2 at 92.22 s on the same two machine shapes (`PARITY-20260922-round2.md`, "What each harness change bought"). First, a leased source tree replaced the per-cell worktree: a node prepares one pristine tree per build slot during setup and a cell leases one under an advisory lock after checking its revision and cleanliness, which cost 0.0 s in every warm run against round 1's 7.026 to 8.327 s per cell. The check is not free on a tree nothing has read: the first candidate recorded 12.0 s of lease on all three cells and m68k_nommu trial 1 recorded 14.0 and 15.0 s on two slots used for the first time (`verdicts.txt`); the startup script now performs that read once per slot, a change made after these runs and credited with none of the figures. Second, recording a worker cell on the orchestrator: in the first candidate the worker's build finished at +106 s and the release exited at 111.47 s because the publish stage went back to the worker for one more poll, a four-second authorisation and a report; `runOnly: [worker, orchestrator]` removed 5.5 s from the end of every release, and trials 1 and 2 exit within a second of their last build. Third, a generator that writes the topology down (`-orchestrator`, `-concurrency`, `-cell-jobs`, `-batch`, `-publish-on-orchestrator`) so that placement, slots, jobs and batching are regenerated rather than edited. None of the three is an engine change; the first is the fork's, the second a configuration the engine already offered, the third tooling.

The same-hardware pairs. To say what the second machine is worth, each group ran once more on a single `c3-standard-22` with no worker and nothing else changed: the same cells, per-cell jobs, build-slot setting and recipe, with `runOnly: orchestrator` and no `workers` key (`dispat/gce/round2/solo-c3/dispat.hexagon.solo.yaml` and `dispat.m68k_nommu.solo.yaml`). The single node reports the same processor as `par-c3`, Intel Xeon Platinum 8481C at 2.70 GHz, 11 cores and two threads each, in zone us-central1-b because the shape had become unavailable in us-central1-a. Both single-node runs built every case correctly and were rejected on elapsed time alone: hexagon at 122.62 s and m68k_nommu at 138.01 s, ratios 0.9052356874898059 and 0.9274690239837694 of the reference (`verdicts.txt`). Against the paired trials the second machine, a slower one of another family, is what carries both groups under the gate.

Two features of the single-node runs are stated rather than hidden. First, `execution.concurrency` did not bound a release that names no worker: hexagon was configured with two build slots and started all three cells at once, m68k_nommu with four and started all five; such a release emits no assignment, claim or probe, which also removes the 3 to 8 s of preflight the paired runs pay, so the single-node figures are the best the machine can do with the recipe, not a throttled version of it. Second, the lease and freshness check dominated the m68k_nommu run: five cells checked five trees at the same moment on one disk, and two recorded 58 and 66 s of `source_lease_seconds` with build scripts of only 73 and 72 s (`solo-c3/results/build-m68k_nommu-m5272c3_defconfig.g5NdBk/result.json` and `build-m68k_nommu-m5249evb_defconfig.LEf7Hy/result.json`, field `timings.source_lease_seconds`, in the fork's `dispat/gce/round2` and in the evidence tree's copy); the three hexagon cells recorded 12.0 to 13.0 s (`verdicts.txt`). The 138.01 s figure is therefore pessimistic by some part of that cost. The conclusion does not rest on the single-node runs alone: the calibration measured the same shape building each whole group outside dispat and without any lease, the hexagon trio in 120.12 s and the five m68k_nommu cells in 128.17 s of script body (`cal-c3.txt`), both at or above their references before a second of release overhead.

alpha, and why it was not run. The alpha group was not attempted. Its `allmodconfig` is one configuration of one cell and cannot be split. The calibration put the other three alpha cases at 68.48, 33.16 and 24.04 s on `c3-standard-22`, 125.68 s together, and round 1 had measured all four as one script body on the same shape at 1,324 s; the subtraction gives `allmodconfig` a lower bound of about 1,198 s, lower because round 1's small cases shared a build directory with it. With about 9 s of fixed overhead the budget for the cell is about 610 s, a factor of 1.96 over what the largest machine the project's 32-vCPU quota allows can do. The fork estimates that a `c3-standard-88` would clear the budget at 60% scaling efficiency and a `c3-standard-44` would not at the measured 0.78; both are estimates and neither was provisioned (`PARITY-20260922-round2.md`, "alpha: the capacity this project does not have"). alpha is therefore a group the experiment could not measure, not one it measured and failed.

What the engine cannot express, and what it logged. The engine has two placement classes and no per-worker pinning; it places a ready package on any free node without regard to speed. The two classes sufficed here only because each group needed exactly two machines and the class pinned to the release's own machine was the fast one; a second `t2d-standard-8` worker could receive hexagon defconfig, at 147 s there, and lose the group. In the m68k_nommu trials the worker finished at +56 and +66 s and sat idle for 59 and 49 s while the orchestrator finished four cells; the fork estimates a perfect split near 100 s of build against the 107 s measured (`PARITY-20260922-round2.md`, "The placement dispat cannot express"). The worker logged one `E226` "assignment rejected, unreadable" warning per release for the snapshot branch, five lines in all and the only warnings a worker produced; a release refused to plan without a pushable `origin` for its lock tag, so source and coordination refs now share the mailbox repository; a package `env` may not carry a key with the `DISPAT_` prefix, so the node-describing variables moved to the node's environment; a second run of an unchanged configuration planned nothing and exited in 3.83 s (`par-c3/hexagon-2.elapsed`, the row of Table 12), so repeated trials begin by removing the release tags; and the preflight probe, a poll round trip whose back-off reaches 5 s while a worker is idle, is the largest fixed overhead left at 3 to 8 s. The round's five VMs lived 10 to 22 minutes each and were deleted; the list-price estimate for the round is about USD 1.48 (`PARITY-20260922-round2.md`, "Cost").

12.6.2 The LLVM pair

Fixture and machines. The question is whether a dispat release of the fork's own recipe, with its build stages delegated to workers, produces the same products as the same release on one machine and takes no longer. The fork is `yohimik/llvm-project`, branch `dispat-gce`, LLVM `main` at `a9f4d3ff` (24.0.0git); the report is `.ci/dispat/gce/REPORT.md` at the branch tip `952c2de4`, and every fork figure below is from it unless another file is named. The recipe `.ci/dispat/build.py` runs with `LLVM_FAST=1` (assertions, examples, docs and benchmarks off), `LLVM_SKIP_CHECKS=1` (no `check-*` targets), the X86 target only, `LLVM_BUILD_JOBS=14`, `LLVM_LINK_JOBS=2` and `LLVM_TOOLCHAIN_PROVIDERS=llvm,lld,polly`; the same values appear in `.ci/dispat/gce/dispat.gce.yaml` (two workers, mailbox the orchestrator's own bare `origin.git`) and `dispat.local.yaml` (the same recipe with no `execution` object). Baselines `dispat/<pkg>/v24.0.0` are tagged at `a9f4d3ff`, so every plan is 24.0.0 to 24.1.0. The machines are `lb-orch`, an `e2-standard-4` with 80 GB, and `lb-w1` and `lb-w2`, `n2-custom-14-49152` (7 cores, 14 threads) with 150 GB, zone us-central1-a, on-demand, all on Ubuntu 24.04.5 with g++ 13.3.0, cmake 3.28.3 and ninja 1.11.1. The recipe's identity record embeds the compiler, cmake and ninja versions, the source revision and the recipe's own hash into every receipt (`build.py`, `identity`), and a consumer refuses a provider whose receipt does not match the tree it installed (`verify_provider`).

Nine attempts. Table 13 lists every attempt. Six have a retained orchestrator log and five of those a GNU `time` record, copied from the measuring session's scratchpad into `experiments/distributed-20260922/evidence/llvm-gce/logs/`; the mapping of file to attempt is the `llvm_attempt_mapping` object of `experiments/distributed-20260922/evidence/MANIFEST.json`, and the wall times below were read from those `.time` files. Attempts 1, 3 and 6 have no orchestrator log; attempts 1 and 6 are covered only by cumulative worker logs, attempt 3 by nothing, its log having been overwritten on the node (`MANIFEST.json`).

The single-heavy fixture of attempts 1 to 6 was replaced by the matrix because one package held nine tenths of the work: the matrix adds `llvm-aarch64`, the same sources built for the AArch64 target as a package that depends on nothing (`.ci/dispat/packages/llvm-aarch64.yaml`), so that a single node must serialise two heavy builds and a pool need not, and it sets `LLVM_BUNDLE_COMPRESSION=none` on both sides because a single-threaded xz of the whole install tree was a 16-minute serial step on either side and says nothing about placement (`REPORT.md`, "The matrix fixture").

The matrix result. Table 14 is the pair. The distributed release took 27:48.94 against 41:12.02 on one node, five packages published and exit 0 on both sides, the same recipe and the same per-package settings. The distributed side had been measured once before with the same placement, attempt 7, at 28:45.63 to the same point plus a bundle that failed on the dispat defect; that attempt is not a second repetition of the accepted pair.

The design-history entry adds the transfer volumes: `llvm`'s declared outputs, 1.4 GB in 2,647 files, were pushed once and fetched by two consumers and the orchestrator, `llvm-aarch64`'s about the same, `lld`'s 420 MB and `polly`'s 22 MB likewise, and the bundle's 1.85 GB never travelled; `lld` took 1 min 47 s and `polly` 2 min 13 s on the opposite workers once `llvm`'s outputs had reached them (`DESIGN-HISTORY.md`, entry of 2026-09-22). What the pair shows is the mechanism SPEC §28.7 names as the profile's required gain, at work on a real dependent graph: `lld` and `polly` consumed `llvm`'s outputs on nodes that never built `llvm`.

Products. The bundle on the orchestrator passed the recipe's own integrity check for every provider it merged: the receipt each worker wrote describes, file by file, the tree the orchestrator installed, so the transport is byte-faithful. `compare.py` then finds the two sides differ in bytes, and every difference is of the reproducible-builds kind: every tool of equal size differs first at its `.note.gnu.build-id` and then only in the compiled-in VCS revision, since LLVM embeds the checkout's commit (`6c1a5fcd` on the single node, the snapshot commit `dbc1e8a7` on a worker, which builds from the prepared input state rather than from the trigger commit) and the build id is a hash over that content; `llvm-config` and `lld` also embed the repository string, which differs in length between the two checkouts; `LLVMPolly.so`, `libPolly.a`, `libPollyISL.a` and `liblldCommon.a` differ by a few kilobytes of RPATH and `__FILE__`-strings that carry the checkout's absolute path; and `libLLVMCore.a`, which embeds none of these, is byte-identical (`REPORT.md`, "Products"). Byte identity throughout is a recipe matter, `-DLLVM_APPEND_VC_REV=OFF`, a file prefix map and a relative RPATH, which this recipe does not set. The claim the pair supports is therefore: the same five products, built by the same recipe on two placements, verified by the recipe's own receipts, differing only in the revision, build-id and path strings the recipe chooses to embed.

The single-heavy negative case. Attempts 4 and 5 are the case SPEC §28.7 predicts against distribution. On `w2` alone, `llvm` built in 18:03, `lld` and `polly` in 2:13 in parallel, and the three were published at 20:16; the xz bundle then took 15:53 for a whole release of

Table 13: The nine LLVM attempts, from the attempts table of `.ci/dispat/gce/REPORT.md` on `origin/dispat-gce` of `/Users/yohimik/Projects/llvm-project`. Wall times and exit statuses in the last column are the `Elapsed (wall clock) time` and `Exit status` lines of the named `release-*.time` file under `experiments/distributed-20260922/evidence/llvm-gce/logs/`, and the published and failed counts are the `done` line of the matching `release-*.log`; a wall time with exit 1 includes the failed bundle step; "none" means no orchestrator log was retained.

#	When (UTC)	What happened	Retained log; wall; exit
1	03:54 to 04:38	`llvm` built and published through `w1`; `lld` and `polly` refused its transported tree because the recipe's fingerprint hashed full file and directory modes, which are the creating process's umask. Discarded; the fingerprint now records the executable bit and link targets only.	none
2	04:40 to 05:07	`llvm`, `lld`, `polly` built and published through the workers in 27 min 21 s; the `toolchain` bundle failed at once on an orchestrator with no C++ compiler for the identity record. Discarded; a provisioning omission.	`release-dist2`; no `.time`; `done`: 3 published, 1 failed
3	05:20 to 05:21	Baseline attempt; the clone on `w2` lacked the `dispat/*/v24.0.0` tags, so the plan started from the initials and the recipe refused the major. Discarded; the tags were pushed.	none
4	05:45 to 06:12	Distributed: `llvm` on `w1`, `lld` on `w2`, `polly` on `w1`, all published; the bundle refused `polly`'s transported tree while `llvm` and `lld` passed.	`release-dist3`; 26:47.69; exit 1; 3 published, 1 failed
5	06:13 to 06:49	Baseline on `w2` alone, four packages published: `llvm` 18:03, `lld` and `polly` in parallel to 06:33:24, the bundle 15:53 of single-threaded xz.	`release-local`; 36:09.09; exit 0; 4 published
6	06:51 to 07:14	Distributed again with the recipe at `2b03be6a` (directory records left out of the fingerprint). Interrupted when the fixture changed to the matrix.	none
7	07:16 to 07:45	The matrix, distributed: `llvm` on `w1` 22:23, `llvm-aarch64` on `w2` 23:06, `lld` and `polly` to 07:43:28, four published; the bundle refused `polly` once more. Diagnosed as a dispat defect: captured outputs passed through the checkout's `.gitattributes` text conversion, which rewrote CRLF pairs inside three libraries.	`release-dist5`; 28:45.63; exit 1; 4 published, 1 failed
8	07:46 to 08:27	The matrix on `w2` alone: five published, the two heavy builds sharing the node for 38:38 and 38:51, `lld` and `polly` to 08:27:14 and 08:27:28, the bundle 18 s.	`release-local3`; 41:12.02; exit 0; 5 published
9	08:29 on	The matrix, distributed, with the fixed dispat `1.11.0-rc.4+main.272772cb`: five published.	`release-dist6`; 27:48.94; exit 0; 5 published

Table 14: The LLVM matrix pair, attempts 8 and 9. Per-package figures are the results table of `.ci/dispat/gce/REPORT.md`; the whole-release wall times are the `Elapsed (wall clock) time` lines of `release-local3.time` and `release-dist6.time` under `experiments/distributed-20260922/evidence/llvm-gce/logs/`, and the `done` lines of the matching logs record 5 published and 0 failed on both sides. One repetition per side.

	Baseline, `w2` alone	Distributed, `w1` + `w2` + `lb-orch`
`llvm` (X86)	38:38, sharing the node	22:08 on `w1`, including the 1.4 GB result push
`llvm-aarch64`	38:51, sharing the node	23:22 on `w2`, including its result push
`lld` + `polly`	2:16	2:13 on the two workers, after `llvm`'s outputs reached them
bundle (uncompressed tar)	0:18	0:32 on `lb-orch`
whole release, GNU `time`	41:12.02, exit 0	27:48.94, exit 0

36:09.09. Distributed, `llvm` took 22:11 on `w1` including its 1.4 GB result push, `lld` and `polly` 2:07 and 2:19 on `w2` and `w1`, and the three were published at 26:47.69, a wall time that includes the bundle step failing on the orchestrator's missing compiler and ends in exit 1 (`REPORT.md`, "The single-heavy fixture"; `release-local.time` and `release-dist3.time` in the evidence tree). With one package holding nine tenths of the work, moving a 1.4 GB install tree to consumers that build in two minutes cannot pay for itself; the matrix is the fixture that says what the profile is for.

What the pair measured about the engine. A cold worker's first claim comes minutes after a warm one's, because the node's poll fetches the snapshot branch before it can read the assignment, about 10 minutes for the 3.6 GB history on a cold cache and 6 s warm. Coordination is cheap beside the builds: probe 1 to 4 s, claim 3 to 6 s warm, a result noticed within 5 s, authorisation to publish command 8 to 12 s. Placement is first-free, and the pair's workers are equal, so nothing here says what an unequal pool would do; the kernel rounds do (`REPORT.md`, "What the pair measured about dispat"). The design-history entry records the earlier LLVM coordination measurements the implementation was tuned on: a probe answering in about 4 s, an assignment claimed 3 s after it is written on a warm node and 37 s on a cold one, a result noticed within 5 s, and a first snapshot transport that carried the repository's whole history, 7.3 million objects, in 17 to 20 minutes from a small orchestrator and was paid on every run until the repository's own remote became the mailbox (`DESIGN-HISTORY.md`, entry of 2026-09-22).

Three defects found and fixed. The pair found one defect in the fork's recipe and two in the engine. The recipe's integrity fingerprint hashed full file and directory modes, which are the creating process's umask; a worker built directories 0775 and the engine installs files as 0644 or 0755 and recreates directories, so a faithfully transported tree failed its receipt (attempt 1). The fingerprint now records the executable bit and symlink targets only, and directories not at all (`build.py`, `fingerprint_records`); attempt 4's diagnosis confirmed that every one of the 124 entries `w1` captured was byte- and mode-identical to the file the orchestrator installed. In the engine, a result carrying outputs reported under a fixed 30 s bound that killed the 1.4 GB push; the fix, at `f0c9e959` on the feature branch, "give a result that carries build outputs the transfer window to report", makes the report use `execution.transfer.timeout` (commit subject from `git log` of `/Users/yohimik/Projects/dispat`; `REPORT.md`). And captured outputs passed through the checkout's `.gitattributes`: `polly`'s tree is marked as text, and `git add` rewrote every CRLF pair inside three binaries, so the bundle refused the tree

in attempts 4 and 7; the fix on `main` at `9f649f88`, "capture build outputs as the build wrote them, whatever the checkout's attributes say", hashes captured outputs with no filter and fills the index by hand, and `272772cb` adds the integration test that reads a library with CRLF pairs through a checkout marked as text (`git log`; `DESIGN-HISTORY.md`).

The binaries. Attempt 9 ran `1.11.0-rc.4+main.272772cb`, the version string its release log prints (`release-dist6.log`, field `dispat`). The `binaries` object of `MANIFEST.json` records the deployed binary by its SHA-256 prefix `feba0210d13c90fa` only, with the note that it was built in a checkout carrying uncommitted documentation edits, so that Go stamped it `vcs.modified=true`, and that it differs from a clean stamped build only by that stamp; it is not reproducible from the commit alone. The reproduction kit's recipe `experiments/distributed-20260922/docker/dispat/Dockerfile` rebuilds the pinned binaries from clean checkouts, and the hashes it wrote are in `docker/dispat/verified-20260922/*/dispat.sha256`: `f0c9e959` unstamped gives `63441cc66f68d07b5efbf9a42edd5559304d153d48b82b1dfb037349e20606ed`, byte-identical to the binary of kernel round 2 and LLVM attempts 1 to 8; `272772cb` stamped gives `46bc2cf821dd57479c27bf2a4b7eee63b69e01db6f18fbd6e663ad66e6812ba8` and unstamped `f1dec45d48df4d7fe5a681f5244ce946d71e140ccfbe97c2f8f3a6842f69b196`. The version string in the log and the clean rebuild together are the pin this report can offer for attempt 9.

Caveats, as the specification states them. The design-history entry qualifies both pairs, and the qualifications are repeated here without softening. One repetition per side on both graphs, so variability is unmeasured; the kernel's second node is a slower machine of another family, so each kernel pair is one node against that node plus a second, slower one; the kernel groups have only independent cells to distribute and do not exercise the transfer of dependent outputs; the LLVM matrix's two heavy builds are one project for two targets, a fair matrix but not two projects; the LLVM distributed side had two build nodes against one; and the single-node m68k_nommu figure includes 58 to 66 s of source-lease contention on two of its cells (`DESIGN-HISTORY.md`, entry of 2026-09-22). Two details of the entry disagree with the retained files and are recorded as observed on 22 September, the product side having been told: the entry dates the LLVM pair 2026-09-23, while every retained log and `.start` file is stamped 2026-09-22 UTC, 07:16 to 08:58, which no time zone of the author's moves to the next day; and it names the fixture commit `6c1a5fcd`, where the fork's own commit of the matrix fixture is `d8fb8408` and the report's is `952c2de4`. The driver record `llvm-gce/drivers/pair4.out` in the evidence tree explains the second: the recipe change was applied on the orchestrator as a patch (`llvm-gce/drivers/0001-feat-llvm-aarch64-build-the-AArch64-target-as-a-pack.patch`) and the resulting tip there, `6c1a5fcd`, is the commit under test that the workers built, so the two hashes name the same change on two checkouts, as the fork report says of its earlier pair (`fbc4dcf8` on the orchestrator, `022f7284` locally, "same tree"). Tree identity for the matrix pair was not verified here, because `6c1a5fcd` exists only on the deleted machine.

Public reference timings, as context only. Two public figures were gathered on 21 September 2026 and are retained; they were read from public pages, concern other hardware and other recipes, and are not compared numerically with the pairs. The upstream `llvm/llvm-project` workflow run 34549173201 of 2026-09-11 ran 294.9166666666667 minutes from start to last update, and its `ubuntu-22.04` release-package job took 154.1 minutes on a runner labelled `depot-ubuntu-22.04-16`, of which 95.51666666666667 minutes were the build step and 56.35 the tests (`output/saner-2027/llvm-forecast-20260921/timing-summary.`

`json`, `runs[0]`; the README beside it records the same job as 154.10 minutes and notes that it corrects an earlier approximate figure of 153). That run builds the full release binaries with tests; the pair builds a 40-minute fixture with no checks, so the two are not the same work. For the kernel farm, the figure of 23.0 machine-hours in 3 h 52 min on six machines is already cited above from `dispat/gce/REPORT.md`, with the same caveat.

Answer to ***RQ5****.* On the kernel fixture the distributed release met the public reference gate twice for each of two groups (hexagon 95.89 and 92.22 s against 111, after one rejected candidate at 111.47 s; m68k_nommu 115.61 and 115.48 s against 128, with no rejected candidate) where the same recipe on the same single node did not (122.62 and 138.01 s), each single-node run once; alpha was outside the project's capacity and was not run. On the LLVM fixture the distributed release of a five-package matrix took 27:48.94 against 41:12.02 on one node, produced the same five products under the recipe's receipts, and the negative case predicted by SPEC §28.7 was observed at 26:47.69 against 20:16. On these fixtures, once each, distributed took no longer than single node; nothing more is claimed.

12.7 Threats to validity

Mechanism matrix and finite projection. The matrix is a designed, deterministic set of 140 episodes, not a sample of production releases; the counts are exhaustive over the design and carry no population inference. The after-write injector is an adapter response, not a crash; the identity check is the supplied local publisher's, not a registry's. The projection is four vertices, one intent and the stable channel; the counterfactual runs no modified binary. The runner image cannot be rebuilt bit-for-bit on a fresh machine because `apk add` consults an external repository, so an exact rerun needs the locally present image `sha256:6f92932f...` (`campaign-mechanisms-20260910/README.md`).

Companion campaigns. The oracle of the 300-history stage is the restricted core profile and cannot support a refinement claim; the recovery, topology and CI stages report repeated verifier predicates, which are not independent observations; the artificial delays in the mixed fixtures expose availability and are not throughput; the environment is one Linux ARM64 Docker host and validates nothing about native Windows. The failed 1.9.0 attempt shows that a verifier can encode a wrong expectation, which is why its correction is documented rather than the attempt deleted.

1.8.2 and historical campaigns. The E1 oracle is the CCME 1.0.0 model, so its divergence classes are taxonomy against that model and the PG3? label is heuristic; a refused diagnostic plan is not a release plan. The E3a replay reconstructs layouts and policies the projects never declared. The competitor observations are of pinned versions in one configuration each. Planning-cost figures are host-sensitive, taken on a capped and contended host, and the earlier harness measured the same cells very differently; none is a cross-version or cross-tool benchmark. The seed 580 change between 1.8.1 and 1.8.2 is not attributable to the product.

Kernel rounds. One pair of machine shapes, one region, one kernel revision, two trials per accepted topology and one single-node run per group; the gate compares against public build-step times measured on the farm's own, different, hardware while it shared each machine with another build. The single-node runs are penalised by the cold-metadata lease cost on two m68k_nommu cells and freed of the preflight the paired runs pay; the calibration outside dispat is the corroboration. alpha, every QEMU group and the 57-group matrix are unmeasured; distcc is unmeasured; the raw evidence of round 1 is a curated archive copied

into the evidence tree rather than files in the fork, and the first run's logs, which the fork's `PARITY.md` could not find, are in the evidence tree's `first-run` directory.

LLVM pair. One repetition per side; the distributed side had two build nodes against one; the two heavy builds are the same sources for two targets; the recipe omits the check targets and the assertions and is not the fork's release recipe; six of nine attempts failed or were discarded for the reasons the table gives and the accepted pair is attempts 8 and 9 only; the deployed binary carried `vcs.modified=true` and is recorded by hash prefix only; and the raw logs were copied from the measuring session's scratchpad into the evidence tree, where `MANIFEST.json` records their origin, and are not in the fork.

12.8 Reproduction

The Docker and Terraform kit for the distributed measurements is at `experiments/distributed-20260922/`: `README.md` and `pins.json`; the `docker/` recipes (`dispat`, `kernel-node`, `llvm-node`); `terraform/kernel` and `terraform/llvm` with a shared node module; `scripts/kernel/run.sh` and `scripts/llvm/run.sh`, which drive a bare fleet to `verdicts.txt` and `compare.txt` respectively; and a `local/` Compose check. The `evidence/` directory beside them holds the copied raw observations with their `MANIFEST.json` and `CROSS-CHECK.json`.

What the kit pins, and what was verified on 22 September 2026 on an Apple silicon workstation with Docker Desktop 29.6.1 and Terraform 1.16.3. The experiment binaries rebuild byte for byte from their commits in a digest-pinned `golang:1.26.5-alpine` with the flags of the original builds: `f0c9e959` unstamped gives `63441cc66f68d07b5efbf9a42edd5559304d153d48b82b1dfb037349e20606ed`, the binary the kernel round-2 report names; `272772cb` stamped `1.11.0-rc.4+main.272772cb` gives `46bc2cf821dd57479c27bf2a4b7eee63b69e01db6f18fbd6e663ad66e6812ba8`, the clean form of the LLVM matrix binary, whose deployed copy differs by Go's `vcs.modified` stamp alone (records under `docker/dispat/verified-20260922/`). The node images take Ubuntu 24.04 by digest, every package from the Ubuntu archive snapshot `20260922T000000Z`, and the kernel.org crosstool 16.2.0 tarballs by checksum; the kernel node image reports the identities the fork's oracle requires (Ubuntu clang 18.1.3, GCC 16.2.0 with GNU assembler 2.47.20260726). The four round-2 configurations are regenerated by the fork's own generator with flags that reproduce the retained files byte for byte apart from the mailbox path. Both Terraform roots validate and plan against the original projects (thirteen resources each: network, subnet, two firewall rules, two keys, the signing secret, three nodes and their secret files); the machine shapes, zones, disk sizes, image and lifetimes are those of the measured fleets. `terraform apply` was not run by the author of this section, so the cloud pairs were not re-measured here; the kit is the recipe, and its README states the expected verdicts and the cost.

The pipeline itself was run end to end on the workstation: two containers of the kernel node image, an orchestrator and a worker sharing a mailbox directory, the exported harness at the fork's commit, the generated `hexagon` configuration, one `dispat release`. The worker built `allnoconfig`, the orchestrator `defconfig` and `tinyconfig`, the release exited 0 and the fork's `compare.py` accepted the run: three cases, the kernel revision `93f51579`, the compiler identity, and 63.47 s of elapsed time (`experiments/distributed-20260922/runs/local-kernel-20260922T184954Z/verdict.json`). That elapsed time is a laptop's on an arm64 toolchain and says nothing about the parity of Section 12.6.1; what the run shows is that the placement, the mailbox transport, the cell harness and the oracle work as wired, from the pinned images alone.

A Notation

Symbol	Meaning	Defined
$\mathcal{K}$, h	commits; the fixed HEAD	Definition 1
$\mathrm{reach}(c)$, $c' \preceq c$	ancestors-or-self of c; c' is one of them	Definition 1
$\mathcal{P}$, E, $\mathrm{dep}(p)$	packages; dependency edges; dependents of p	Definition 3
K_{prop}, K_{order}, K_{block}	the propagation, ordering and blocking edge kinds	Definition 3
$\delta(S, d)$	least edge count from a member of S to d	Definition 5
$R(S, \ell)$	the traversal: $\{d \notin S : \delta(S, d) \leq \ell\}$	Definition 28
$\mathbb{B}$, $\sqcup$	$\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}$; their join	Definition 6
$\mathcal{V}$, $\mathcal{C}h$, stable	versions; channels; the stable channel	Definition 7
$\mathrm{ch}(v)$, $\mathrm{core}(v)$, $\mathrm{cnt}(v)$	channel, core triple, prerelease counter of v	Definition 7
$\mathrm{apply}(v, b)$	v with bump b applied	Definition 7
σ, $\sigma \leq \sigma'$	engine state (the reachable tags); inclusion	Definition 8
$T_\sigma(p)$	release commits of p; $h \in T_\sigma(p)$: tagged at h	Definition 8
$\mathrm{base}_\sigma(p)$, $\mathrm{basec}_\sigma(p)$	baseline version of p; its commit	Definition 8
$\mathrm{sbase}_\sigma(p)$, $\mathrm{scom}_\sigma(p)$	stable baseline of p; its commit	Definition 8
$W_\sigma(p)$	train window of p	Definition 9
$W^\circ_\sigma(p)$	undelivered (fresh-admission) window	Definition 9
$\mathrm{dlv}_\sigma(P, C, D)$	P has delivered C to D	Definition 10
$W^{\mathrm{ow}}_\sigma(P, D)$	owed window: what P has not delivered to D	Definition 11
$\mathcal{U}$, c_u, $\mathrm{scope}(u)$	units; a unit's commit; its resolved scope-set	Definition 16
$\mathrm{bump}(u)$	the bump u's type and `!` map to	Definition 16
$\mathrm{pb}(u)$, $\mathrm{pd}(u)$, $\mathrm{ps}(u)$	bump-axis directives of u	Definition 16
$\mathrm{pc}(u)$, $\mathrm{pcd}(u)$, $\mathrm{pcs}(u)$	channel-axis directives of u	Definition 16
$\mathrm{canc}(c, p)$, $\mathrm{held}(p)$	the two suppressors	Definition 19
$\mathrm{disch}_\sigma(p, c)$	p has already released the work in c	Definition 19
$\mathrm{src}_\sigma(u)$	source packages of u	Definition 19
$\mathrm{owed}_\sigma(u, D)$	sources of u that have not delivered c_u to D	Definition 19
$\mathrm{owes}_\sigma(P, D)$	P is owed to D by some admitted unit	Definition 19
$\mathrm{sch}_\sigma(u)$	channels of u's sources	Definition 35
$\mathrm{rsv}_\sigma(u, d)$	d can resolve what u's sources release	Definition 35
$\mathrm{adm}_\sigma(u, d)$	u propagates to d	Definition 37
$\mathrm{chan}_\sigma(p)$	the channel p is released on	Definition 34
$\mathrm{dir}_\sigma(p)$, $\mathrm{prp}_\sigma(p)$, $\mathrm{eff}_\sigma(p)$	direct, propagated, effective bump	Definition 38
$\Pi(\sigma)$, $\nu_\sigma(p)$	the plan; p's planned version	Definition 21
$\mathrm{own}_\sigma(p, X)$	p has a cause of its own relative to X	Definition 23
$\mathrm{wf}(\sigma)$	no provider planned at the baseline commit of a consumer it owes without that consumer following (`E201`)	Definition 24
$\mathrm{run}(\sigma, F)$, $S(\sigma, F)$, $B(\sigma, F)$	one run, failing on F; what it publishes; what it blocks	Definition 25
$\mathrm{stale}_\sigma(d)$	the upward audit of SPEC §13.7b	Definition 110
(H1), (H2), (R), (K)	standing hypotheses	Section 6

Diagnostic codes (`E200`, `E201`, `W208`, and the rest) are those of SPEC §16 and are used here with their registry meanings.

References

[1] *Conventional Commits: Monorepo Extension (CCME) 3.1.0-rc.2.* `specs/ccme-spec/SPEC.md` at `252216ba`, `github.com/yohimik/dispat`, 21 September 2026. The current normative subject, cited throughout as SPEC. GPL-3.0-or-later.

[2] *CCME design history and changelog.* `specs/ccme-spec/DESIGN-HISTORY.md` and `CHANGELOG.md` at `252216ba`. Dated entries stating what the engine implements and where it departs; the entries of 21 and 22 September 2026 are quoted in Sections 10 and 12.

[3] *CCME 2.0.0.* `released CCME 2.0.0` at `d3ce8ae6`. The version the earlier editions of this report formalised; the historical audit uses the `CCME 1.0.0 snapshot` at `713f1a0b`.

[4] *CCME 3.0.1.* `released CCME 3.0.1`. Corrected algorithm and cost guidance between 2.0.0 and the current text.

[5] *dispat*: a release engine for polyglot monorepositories and polyrepositories. `github.com/yohimik/dispat`, commit `252216ba`; latest released candidate `services/dispat/v1.11.0-rc.3`. The reference implementation; the release computation is `services/dispat/internal/plan`, the execution profile `services/dispat/internal/execution`.

[6] *dispat integration test plan and requirement matrix.* `tests/integration/docs/test-plan.md` and `qa-requirements.md` at `252216ba`. Sixty goals, the claim each integration test proves, the regression and bug fences; the source of the integration rows of Section 11.

[7] *Conventional Commits 1.0.0.* `conventionalcommits.org`. CCME is a strict superset.

[8] T. Preston-Werner. *Semantic Versioning 2.0.0.* `semver.org`. Supplies the version precedence of Definition 7.

[9] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels.* RFC 2119, IETF, 1997. The requirement levels SPEC is written in.

[10] H. Garcia-Molina and K. Salem. *Sagas.* In Proceedings of the ACM SIGMOD International Conference on Management of Data, 1987. The pattern the publish protocol of Section 4 implements, with git tags as the log.

[11] A. B. Kahn. *Topological sorting of large networks.* Communications of the ACM 5(11), 1962. The publish order of Proposition 45.

[12] R. L. Graham. *Bounds for certain multiprocessing anomalies.* Bell System Technical Journal 45(9), pages 1563 to 1581, 1966. The scheduling anomaly of Example 63.

[13] R. L. Graham. *Bounds on multiprocessing timing anomalies.* SIAM Journal on Applied Mathematics 17(2), pages 416 to 429, 1969. The list-scheduling bound of Proposition 73.

[14] S. Chacon and B. Straub. *Pro Git*, 2nd edition. Apress, 2014. The commit DAG, reachability and tags of Definitions 1 and 8.

[15] G. Roeck. *kerneltests.org*: the Linux kernel build and boot test farm, `https://kerneltests.org/`, builders `upstream-<arch>-master` builds 330 (hexagon), 331 (m68k_nommu) and 332 (alpha) for v7.3-rc4, read on 22 September 2026 and retained in `dispat/gce/parity-20260922-reference.json` of the fork `github.com/yohimik/linux-build-test`. The public reference times of Section 12.

[16] *linux-build-test*, fork of G. Roeck's farm configuration with the dispat prototype. `github.com/yohimik/linux-build-test`, branch `codex/kernel-parity` at `c699bf3d`; reports `dispat/gce/REPORT.md`, `PARITY-20260922.md` and `PARITY-20260922-round2.md`; oracle `dispat/compare.py`.

[17] *llvm-project*, fork with the dispat recipe. `github.com/yohimik/llvm-project`, branch `dispat-gce` at `952c2de4`; report `.ci/dispat/gce/REPORT.md`; recipe `.ci/dispat/build.py`; oracle `.ci/dispat/gce/compare.py`.

[18] *kernel.org cross toolchains.* `https://mirrors.edge.kernel.org/pub/tools/crosstool/`, release 16.2.0 (GCC 16.2.0, binutils 2.47) for x86_64 and arm64 hosts, with published SHA-256 sums; the alpha and m68k compilers of the kernel farm.

[19] *Google Compute Engine machine families.* `https://cloud.google.com/compute/docs/machine-resource`, read 22 September 2026: `c3-standard-22`, `t2d-standard-8`, `n2-custom-14-49152` and `e2-standard-4`, the shapes of Section 12.

[20] Semyon F. (yohimik). *HOL Guard pull request 2813: opt-in Dispat release-start review.* Open, unmerged pull request, revision `132bb5d105e1234b9f81fddf69267ec488e2990a`, 6 September 2026.